

Pontificia Universidad Javeriana

Facultad de Ingeniería · Arquitectura de Software



TALLER: ESTILOS ARQUITECTÓNICOS Y STACKS TECNOLÓGICOS

Event-Driven Architecture

Topología Broker (coreografía) aplicada a una plataforma de reservas de eventos

Investigación · Análisis arquitectónico · Diseño · Lecciones aprendidas

Flutter
Frontend

.NET / C#
Backend

Elasticsearch
Persistencia

Apache Kafka
Integración

Integrantes: Carlos Pinzón · Nicolás Granados · Juan Esteban Vera

Docente: Andres Sanchez

Curso / grupo: Arquitectura de Software – Grupo 1

Fecha: 29 de Septiembre de 2026

Repositorio: <https://github.com/CarlitosPinzon/EDA-Proyect>

Resumen

Este documento presenta el documento técnico del taller de estilos arquitectónicos. El grupo trabaja con la **Arquitectura Orientada a Eventos** (*Event-Driven Architecture*, EDA) en su **topología Broker**, es decir, una coreografía de servicios que reaccionan a eventos publicados en un bus sin un orquestador central. El stack asignado está compuesto por **Flutter** (interfaz multiplataforma), **.NET 10 con C#** (servicios de backend), **Elasticsearch** (persistencia documental y de búsqueda) y **Apache Kafka** (plataforma de *event streaming* que actúa como bus de integración).

El dominio elegido para el caso práctico es una **plataforma de reservas y venta de tickets para eventos**, un problema clásico de EDA: picos de demanda imposibles de predecir, inventario limitado (asientos o cupos), pagos externos lentos o fallidos y la necesidad de notificar al usuario en tiempo casi real. El diseño toma como referencia la arquitectura pública de **Ticketmaster**, la mayor ticketera del mundo y uno de los usuarios más conocidos de Apache Kafka.

El documento se organiza así: el Capítulo 2 investiga el estilo (definición, clasificación, historia, ventajas, problemas, patrones y casos reales); el Capítulo 3 investiga cada tecnología del stack y su relación con el estilo; el Capítulo 4 desarrolla las cuatro matrices de análisis arquitectónico exigidas (atributos de calidad, principios de diseño, tácticas con sus ADR y mercado laboral). El Capítulo 5 presenta el diseño del ejemplo práctico, llamado TaquillaEDA: arquitectura de referencia, alcance, diagrama de alto nivel, modelo C4 (contexto, contenedores, componentes y código), diagrama dinámico, despliegue en contenedores, modelo de datos, guía de implementación y evidencias de funcionamiento. El Capítulo 6 recoge las lecciones aprendidas durante la investigación, el análisis, el diseño y la implementación, y el documento cierra con las conclusiones.

El sistema se implementó y se publicó en un repositorio público (<https://github.com/CarlitosPinzon/EDA-Proyect>). Además del despliegue en un solo equipo con Docker Compose, se probó en dos equipos (Linux y Windows) con réplicas de Inventario y Pagos en ambos: 100 solicitudes concurrentes por 50 cupos terminaron en exactamente 50 reservas confirmadas y ninguna sobreventa, y al apagar los servicios de un equipo, o desconectarlo de la red en plena carga, el otro asumió su trabajo sin perder reservas.

Palabras clave: arquitectura orientada a eventos, coreografía, Apache Kafka, Elasticsearch, .NET, Flutter, saga, consistencia eventual, ADR, modelo C4, tolerancia a fallos.

Índice general

Resumen	i
1. Introducción	1
1.1. Contexto y objetivo	1
1.2. Asignación del grupo	1
1.3. Dominio del caso práctico	2
1.4. Metodología	2
2. Investigación del estilo: Event-Driven Architecture	3
2.1. Definición	3
2.2. Clasificación del estilo	4
2.2.1. Ubicación en el catálogo de estilos	4
2.2.2. Topologías	5
2.2.3. Los cuatro significados de “orientado a eventos”	5
2.3. Características principales	7
2.4. Historia y evolución	8
2.5. Ventajas y desventajas	8
2.6. Problemas comunes y patrones que los resuelven	10
2.7. Patrones aplicables y cuándo usarlos	11
2.8. Casos de uso: cuándo usar EDA y cuándo no	13
2.9. Casos de aplicación en la industria	13
2.10. Anti-patrones de riesgo para este estilo	14
3. Investigación del stack tecnológico	15
3.1. Flutter (frontend)	15
3.1.1. Definición	15
3.1.2. Características principales	16
3.1.3. Historia y evolución	17
3.1.4. Ventajas y desventajas	17
3.1.5. Casos de uso	17
3.1.6. Casos de aplicación en la industria	18
3.2. .NET y C# (backend)	18

3.2.1. Definición	18
3.2.2. Características principales	19
3.2.3. Historia y evolución	20
3.2.4. Ventajas y desventajas	20
3.2.5. Casos de uso	20
3.2.6. Casos de aplicación en la industria	21
3.3. Elasticsearch (persistencia)	21
3.3.1. Definición	21
3.3.2. Características principales	22
3.3.3. Historia y evolución	23
3.3.4. Ventajas y desventajas	23
3.3.5. Casos de uso	24
3.3.6. Casos de aplicación en la industria	24
3.4. Apache Kafka (integración)	24
3.4.1. Definición	25
3.4.2. Conceptos y características principales	26
3.4.3. Historia y evolución	27
3.4.4. Ventajas y desventajas	27
3.4.5. Casos de uso	27
3.4.6. Casos de aplicación en la industria	28
3.5. Protocolos de integración y componentes adicionales	28
3.6. Relación entre el estilo y las tecnologías	28
3.7. ¿Qué tan común es el stack asignado?	29
4. Análisis arquitectónico	31
4.1. Matriz de atributos de calidad vs. estilo	31
4.2. Matriz de principios de diseño vs. estilo	36
4.3. Matriz de tácticas vs. estilo y stack, y registros de decisión (ADR)	40
4.3.1. Registros de decisiones arquitectónicas (ADR)	43
4.4. Matriz de mercado laboral vs. estilo y stack	46
4.4.1. Fuentes y método	46
4.4.2. Popularidad e indicadores de demanda	47
4.4.3. Salarios	47
4.4.4. Proyección en el mercado laboral	49
5. Diseño: ejemplo práctico y funcional	50

5.1. Arquitectura de referencia: Ticketmaster	50
5.1.1. Por qué Ticketmaster	50
5.1.2. Vista reconstruida de la arquitectura de referencia	52
5.1.3. Lecciones de la referencia que se trasladan al diseño	52
5.1.4. Fuentes específicas de la arquitectura de referencia	53
5.2. Alcance del ejemplo práctico	54
5.2.1. Requisitos funcionales	55
5.2.2. Escenarios de atributos de calidad	55
5.2.3. Entidades de negocio	56
5.2.4. Caso de uso de extremo a extremo	57
5.3. Diagrama de alto nivel (HLD)	57
5.4. Diagrama de contexto (C4 – Nivel 1)	59
5.5. Diagrama de contenedores (C4 – Nivel 2)	60
5.6. Diagrama dinámico (flujo principal)	62
5.6.1. Coreografía completa y compensaciones	64
5.6.2. Máquina de estados de la reserva	67
5.7. Diagrama de despliegue	68
5.7.1. Del taller a producción	71
5.8. Diagramas opcionales	72
5.8.1. System landscape	72
5.8.2. Diagrama de componentes (C4 – Nivel 3): servicio de Inventario	73
5.8.3. Diagrama de código	75
5.8.4. Modelo de datos	76
5.9. Trazabilidad entre decisiones y diseño	79
5.10. Guía de implementación y repositorio	80
5.10.1. Estructura del repositorio	80
5.10.2. Pasos de despliegue	81
5.10.3. Criterios de aceptación de la implementación	82
5.10.4. Evidencias de funcionamiento	82
6. Lecciones aprendidas	85
6.1. Sobre el estilo arquitectónico	85
6.1.1. En una coreografía, el flujo hay que documentarlo porque el código no lo muestra	85
6.1.2. Kafka ordena por partición, no por proceso de negocio	85
6.1.3. Cada agregado necesita una sola autoridad sobre su ciclo de vida	85

6.1.4.	La consistencia eventual se diseña también en la interfaz	86
6.1.5.	Los patrones del estilo no son opcionales	86
6.2.	Sobre el stack tecnológico	86
6.2.1.	Elasticsearch como almacén principal obliga a modelar por agregados . . .	86
6.2.2.	La clave de partición es una decisión de diseño, no de configuración	86
6.2.3.	Evaluar el ecosistema, no solo las funciones	87
6.2.4.	Una herramienta más es un contenedor más que operar	87
6.3.	Sobre la arquitectura de referencia	87
6.3.1.	Distinguir entre hechos documentados y supuestos	87
6.3.2.	Los incidentes enseñan más que los casos de éxito	87
6.4.	Sobre el proceso de trabajo	88
6.4.1.	Los ADR hacen posible cambiar de opinión	88
6.4.2.	Revisar el diseño es más barato que corregir el código	88
6.4.3.	Los datos de mercado se leen como tendencia	88
6.5.	Qué haríamos diferente	89
6.6.	Lecciones de la implementación	89
6.6.1.	El <i>outbox</i> cumple su promesa a cambio de entregar eventos más de una vez	89
6.6.2.	Agregar réplicas no reparte la carga de una localidad muy demandada . . .	90
6.6.3.	La sala de espera solo funciona con clientes que cooperan	90
6.6.4.	Detectar una falla toma tiempo, y ese tiempo se configura	90
6.6.5.	En Kafka, la dirección de conexión la decide el <i>broker</i>	91
6.6.6.	Replicar los servicios no replica la infraestructura	91
6.6.7.	Muchas fallas aparecen en la configuración entre componentes	91
6.6.8.	Un despliegue reproducible necesita verificaciones previas, no solo un comando	91
6.6.9.	Lo que no se ve no se puede demostrar	92
Conclusiones		93
Referencias		95

Índice de figuras

2.1. Comparación de las topologías Broker y Mediator de EDA (elaboración propia a partir de [1])	5
2.2. Línea de tiempo de la evolución de EDA (elaboración propia)	8
3.1. Vista preliminar del rol de cada tecnología en la solución (se detallará con el modelo C4 en el Capítulo 5)	15
3.2. Adopción de tecnologías del stack en la encuesta Stack Overflow 2025 (todos los encuestados) [32]. Flutter no aparece en la categoría web de 2025; en 2024 registró 9,4 % en “otros frameworks”.	30
4.1. Perfil de atributos de EDA frente a un monolito en capas (estimación del grupo inspirada en [1])	36
4.2. Salario anual mediano por rol relacionado con el stack (Stack Overflow Developer Survey 2025) [33]	49
5.1. Vista reconstruida de la arquitectura de Ticketmaster (elaboración propia a partir de [TM1]–[TM9]; no es un diagrama oficial)	52
5.2. HLD	58
5.3. Diagrama de contexto (C4 nivel 1) de TaquillaEDA	59
5.4. Diagrama de contenedores (C4 nivel 2) de TaquillaEDA. Discontinuas naranjas: publicación (pub) y suscripción (sub) de eventos	60
5.5. Diagrama dinámico del CU-01 “Reservar y pagar un ticket” (camino feliz)	62
5.6. Máquina de estados del agregado Reserva (doble borde: estados finales)	67
5.7. Diagrama de despliegue de TaquillaEDA con Docker Compose	69
5.8. System landscape de TaquillaEDA (punteado: sistemas futuros que consumirían los eventos existentes)	73
5.9. Diagrama de componentes (C4 nivel 3) del servicio de Inventario	74
5.10. Diagrama de código (clases principales del servicio de Inventario y del agregado Reserva)	75
5.11. Modelo conceptual de datos con fronteras de propiedad por servicio (ref : referencia por identificador sin clave foránea)	76

Índice de tablas

1.1. Estilo y stack tecnológico asignados al grupo	1
2.1. Clasificación de EDA según varios criterios	4
2.2. Variantes de uso de eventos según Fowler y su uso en el caso	6
2.3. Elementos constitutivos del estilo	7
2.4. Ventajas y desventajas de EDA (topología Broker)	9
2.5. Problemas recurrentes en EDA y patrones de solución	10
2.6. Patrones del catálogo aplicables a EDA y su uso en el caso	11
2.7. Criterios de adopción de EDA	13
2.8. Casos reales de EDA en la industria	13
3.1. Hitos de Flutter	17
3.2. Ventajas y desventajas de Flutter	17
3.3. Hitos de .NET	20
3.4. Ventajas y desventajas de .NET para servicios orientados a eventos	20
3.5. Hitos de Elasticsearch	23
3.6. Ventajas y desventajas de Elasticsearch como almacén de los servicios	23
3.7. Conceptos de Kafka y su función en EDA	26
3.8. Hitos de Apache Kafka	27
3.9. Ventajas y desventajas de Kafka como bus de eventos	27
3.10. Protocolos de comunicación definidos para la solución	28
3.11. Cómo cada tecnología habilita (o tensiona) las propiedades de EDA	29
3.12. Frecuencia de combinación de las tecnologías del stack (estimación del grupo)	30
4.1. Matriz de atributos de calidad vs. EDA (topología Broker) y mitigaciones con el stack	32
4.2. Matriz de principios de diseño vs. EDA (topología Broker)	37
4.3. Matriz de tácticas vs. estilo y stack	41
4.4. Indicadores de popularidad del estilo y del stack (septiembre de 2026)	47
4.5. Matriz de mercado laboral vs. estilo y stack (salarios y proyección)	48
5.1. Hechos públicos sobre la arquitectura de Ticketmaster usados como referencia	50
5.2. De la arquitectura de referencia al diseño de TaquillaEDA	52
5.3. Requisitos funcionales de TaquillaEDA	55

5.4. Escenarios de calidad del ejemplo práctico	55
5.5. Entidades de negocio, relaciones y servicio dueño	56
5.6. Bloques del HLD y su justificación	58
5.7. Elementos del diagrama de contexto	59
5.8. Catálogo de contenedores	61
5.9. Pasos del flujo principal, tópicos y patrones aplicados	62
5.10. Matriz de coreografía: reacción de cada servicio a cada evento	65
5.11. Flujos alternos y mecanismo que los resuelve	68
5.12. Configuración de despliegue por contenedor	69
5.13. Despliegue del taller frente a un despliegue productivo	72
5.14. Mappings principales de los índices	77
5.15. Tópicos de Kafka, claves de partición y grupos de consumidores	78
5.16. Trazabilidad ADR – elementos de diseño – evidencia	79
5.17. Ajustes entre el diseño y la implementación	81
5.18. Criterios de aceptación ligados a los requisitos del taller	82
5.19. Pruebas ejecutadas y resultados observados	83
5.20. Procedimientos de verificación de los demás escenarios	84
6.1. Decisiones que el grupo reconsideraría en un proyecto real	89

Introducción

1.1 Contexto y objetivo

El taller busca que el grupo seleccione, modele e implemente una solución de software sustentando cada decisión técnica en las características del estilo arquitectónico y del stack tecnológico asignado. Este documento técnico reúne los cuatro componentes exigidos: la *investigación* del estilo y del stack, el *análisis arquitectónico* mediante cuatro matrices, el *diseño* del ejemplo práctico con HLD y modelo C4, y las *lecciones aprendidas*. La implementación, desplegada en contenedores, está publicada en un repositorio público; la Sección 5.10 describe su estructura, cómo desplegarla y las evidencias de funcionamiento.

1.2 Asignación del grupo

Tabla 1.1: Estilo y stack tecnológico asignados al grupo

Componente	Elección	Versión de referencia (sept. 2026)
Estilo arquitectónico	EDA – topología Broker	Coreografía de servicios con consistencia eventual
Frontend	Flutter (Dart)	Flutter 3.47 / Dart 3.13 (canal estable, agosto de 2026) [31, 29]
Backend	.NET / C# (ASP.NET Core)	.NET 10 LTS, soporte hasta noviembre de 2028 [20]
Persistencia	Elasticsearch	Serie 9.x (9.5, agosto de 2026); cliente <code>Elastic.Clients.Elasticsearch</code> 9.5 [24, 25]
Integración (asíncrona)	Apache Kafka	Serie 4.x, solo modo KRaft; 4.2 (feb. 2026) con <i>share groups</i> en disponibilidad general [13, 14]
Integración (síncrona)	REST/HTTP + SSE	API REST para comandos y consultas; Server-Sent Events (nativos en .NET 10) para notificaciones <i>push</i>
Componentes adicionales	Docker Compose, API Gateway YARP, Kibana, Kafbat UI, OpenTelemetry + Aspire Dashboard	Contenedores obligatorios, entrada única, observabilidad y administración

1.3 Dominio del caso práctico

La **plataforma de reservas de eventos** permite a un usuario buscar eventos (conciertos, obras, partidos), seleccionar localidades o asientos, reservarlos temporalmente, pagar y recibir su ticket. Las entidades de negocio son **Evento**, **Localidad**, **Reserva**, **Pago** y **Cliente** (el taller exige mínimo tres entidades relacionadas); el inventario se maneja por localidad con cupos, no por asiento numerado. El caso de uso de extremo a extremo propuesto es “*Reservar y pagar un ticket*”, que atraviesa todos los componentes: la app Flutter, la API en .NET, el bus Kafka, los servicios consumidores y los índices de Elasticsearch.

★ Idea clave

Se eligió este dominio porque expone de forma natural los problemas que EDA resuelve (picos de tráfico, desacople entre inventario, pagos y notificaciones) y también los que EDA introduce (consistencia eventual, duplicados, orden de eventos y compensaciones). Esto permite que las matrices de análisis no sean abstractas sino que se apliquen a decisiones concretas.

1.4 Metodología

La investigación combinó fuentes primarias (documentación oficial de Apache Kafka, Microsoft .NET, Elastic y Flutter), literatura de referencia en arquitectura de software (Richards y Ford, Bass *et al.*, Hohpe y Woolf, Fowler, Kleppmann) fuentes de mercado (Stack Overflow Developer Survey 2025, índices TIOBE y PYPL, IEEE Spectrum, portales de empleo colombianos) y fuentes públicas sobre la arquitectura de Ticketmaster (charlas técnicas, casos de estudio y comunicados), usada como referencia del diseño. Las cifras de mercado corresponden a la fecha de consulta (septiembre de 2026) y se convierten a pesos colombianos con la TRM de \$ 3.306,86 por dólar del 27 de septiembre de 2026 [49].

Investigación del estilo: Event-Driven Architecture

2.1 Definición

Definición

La **Arquitectura Orientada a Eventos** (EDA) es un estilo arquitectónico *distribuido y asíncrono* en el que los componentes del sistema se comunican produciendo, detectando y reaccionando a **eventos**. Un evento es un registro inmutable de *algo que ya ocurrió* y que es relevante para el negocio (por ejemplo, **ReservaCreada**, **PagoAprobado**). Los productores publican eventos sin conocer quién los consumirá; los consumidores se suscriben a los eventos que les interesan y ejecutan su propia lógica de forma independiente [1, 3, 7].

Tres ideas sostienen la definición:

1. **El evento es un hecho, no una orden.** Se nombra en pasado (**AsientoLiberado**) y no puede “rechazarse”; a lo sumo, otro componente reacciona emitiendo un nuevo hecho. Una orden (*command*), en cambio, se dirige a un destinatario concreto y puede fallar.
2. **Inversión de la dependencia de comunicación.** En un estilo de solicitud-respuesta el emisor conoce al receptor y espera su respuesta. En EDA el emisor solo conoce el *canal* (tópico); el receptor decide suscribirse. Esto produce desacople *espacial* (no se conocen), *temporal* (no tienen que estar activos a la vez) y de *sincronización* (el productor no se bloquea) [6].
3. **Flujo de negocio emergente.** En la topología Broker ningún componente conoce el flujo completo: el proceso de negocio “emerge” de la cadena de reacciones de cada consumidor. Esa es su mayor virtud (extensibilidad) y su mayor riesgo (visibilidad).

Qué NO es

- **No es simplemente “usar una cola”.** Un sistema que envía mensajes a una cola para que un único trabajador ejecute una tarea es mensajería punto a punto; se vuelve EDA cuando los mensajes representan hechos del dominio que varios interesados pueden consumir sin que el productor lo sepa.
- **No es sinónimo de microservicios.** Los microservicios definen la granularidad del despliegue; EDA define el estilo de interacción. Puede haber microservicios síncronos (REST puro) y monolitos modulares internamente orientados a eventos.
- **No es Event Sourcing.** Event Sourcing es un patrón de persistencia (el estado se reconstruye a partir de la secuencia de eventos). Un sistema EDA puede guardar su estado

de forma tradicional y aun así comunicarse por eventos [7].

- **No es el patrón Observer en memoria.** Observer (GoF) ocurre dentro de un proceso y de forma síncrona. EDA traslada la idea a nivel de sistema distribuido, con un intermediario durable.
- **No garantiza consistencia inmediata.** Por definición el estado entre servicios es *eventualmente* consistente; si el negocio exige atomicidad estricta entre varias entidades, EDA obliga a diseñar sagas y compensaciones.
- **No elimina el acoplamiento.** Lo traslada al *contrato del evento* (nombre, esquema y semántica). Un cambio incompatible en el esquema puede romper a consumidores que el productor ni siquiera conoce.

2.2 Clasificación del estilo

2.2.1 Ubicación en el catálogo de estilos

Según la taxonomía de Richards y Ford [1], los estilos se dividen en *monolíticos* (una unidad de despliegue) y *distribuidos* (varias unidades que se comunican por red). EDA pertenece a los **estilos distribuidos**, junto a SOA, microservicios, *space-based* y *service-based*. En el catálogo del taller aparece dentro de “Estilos distribuidos y cloud-native”, con tres variantes: Broker, Mediator y Streaming. Además, su partición es **técnica** en la topología Broker básica (los procesadores se organizan por la reacción que ejecutan), aunque en la práctica moderna se combina con partición por dominio (cada servicio dueño de un subdominio).

Tabla 2.1: Clasificación de EDA según varios criterios

Criterio	Clasificación	Explicación
Unidades de despliegue	Distribuido	Productores, broker y consumidores se despliegan por separado.
Estilo de comunicación	Asíncrono, basado en mensajes	El emisor no espera respuesta; el intermediario almacena y entrega.
Partición del sistema	Técnica y/o por dominio	Broker básico: por reacción; en práctica moderna: un servicio por subdominio (inventario, pagos).
Coordinación	Coreografía (Broker) u orquestación (Mediator)	Define si existe un componente central que conoce el flujo.
Consistencia de datos	Eventual	El estado converge tras procesar los eventos; no hay transacción global.
Familia en el catálogo del taller	Distribuidos y cloud-native	Comparte grupo con SOA, microservicios, serverless y <i>space-based</i> .

2.2.2 Topologías

Topología Broker (coreografía). No existe un mediador central. El flujo de eventos se distribuye a través de un broker ligero (en nuestro caso, tópicos de Kafka). Cada procesador escucha un evento, ejecuta su lógica y publica un nuevo evento que describe lo que hizo. Es la topología elegida por el grupo: favorece la escalabilidad, el rendimiento y la extensibilidad, a costa de menor control del flujo global y de un manejo de errores más complejo [1].

Topología Mediator (orquestración). Un mediador (orquestrador) recibe el evento inicial y despacha pasos a los procesadores mediante canales dedicados, conociendo el orden y el estado del proceso. Facilita el manejo de errores, la recuperación y los flujos con muchos pasos condicionales, pero concentra acoplamiento y puede convertirse en un cuello de botella.

Event Streaming. El bus no es una cola que se vacía, sino un *log* durable, ordenado por partición y re-leíble. Los consumidores llevan su propia posición (*offset*) y pueden reprocesar la historia. Kafka es la implementación de referencia. Nuestro diseño usa Broker *sobre* una plataforma de streaming, lo que añade la posibilidad de *replay*.

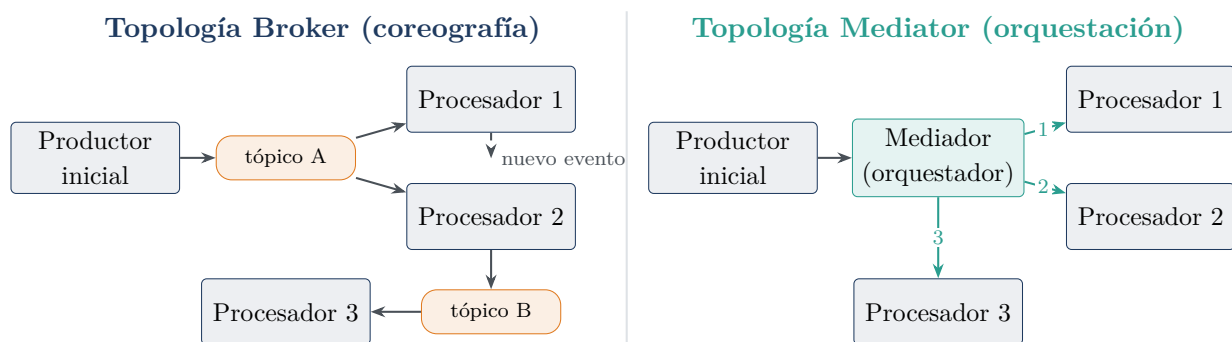


Figura 2.1: Comparación de las topologías Broker y Mediator de EDA (elaboración propia a partir de [1])

2.2.3 Los cuatro significados de “orientado a eventos”

Fowler [7] advierte que el término agrupa cuatro patrones distintos que conviene no confundir, porque cada uno tiene implicaciones diferentes:

Tabla 2.2: Variantes de uso de eventos según Fowler y su uso en el caso

Variante	Descripción	Uso en el caso
Event Notification	El evento solo avisa que algo cambió (ID y tipo); el consumidor consulta al productor si necesita más datos. Muy desacoplado, pero genera llamadas de vuelta.	Parcial: notificaciones al usuario.
Event-Carried State Transfer	El evento transporta el estado necesario para que el consumidor no tenga que consultar al productor. Aumenta autonomía y disponibilidad; duplica datos.	Principal: ReservaCreada lleva evento, localidad, cantidad y precio.
Event Sourcing	El estado de una entidad se deriva del registro ordenado de todos sus eventos.	No como almacén primario; Kafka conserva el historial para <i>replay</i> .
CQRS	Separar el modelo de escritura (comandos) del de lectura (consultas).	Sí: vistas materializadas de búsqueda en Elasticsearch.

2.3 Características principales

Tabla 2.3: Elementos constitutivos del estilo

Elemento	Responsabilidad	Materialización en el stack
Evento	Hecho inmutable con identificador, tipo, marca de tiempo, fuente y datos.	Mensaje JSON con sobre CloudEvents en Kafka.
Productor (<i>emitter</i>)	Detecta un cambio de estado y lo publica, sin conocer a los consumidores.	Servicio .NET con Confluent.Kafka (<i>producer</i> idempotente).
Canal / tópico	Agrupar eventos de un mismo tipo o agregado; define orden y paralelismo (particiones).	Tópicos Kafka, p. ej. reservas.v1 , pagos.v1 .
Broker / bus	Recibe, persiste y distribuye eventos; desacopla en el tiempo.	Clúster Kafka 4.x en modo KRaft.
Consumidor (<i>event processor</i>)	Reacciona a eventos; ejecuta lógica; puede emitir nuevos eventos.	Worker Services .NET en grupos de consumidores.
Almacén propio	Estado local de cada servicio (<i>database per service</i>).	Un índice de Elasticsearch por servicio.
Contrato	Esquema y semántica del evento; versión.	JSON Schema / AsyncAPI documentados en el repositorio.

Las características de calidad que definen el estilo son:

- **Asincronía y no bloqueo:** el productor responde al usuario apenas registra el hecho; el trabajo restante sucede en segundo plano.
- **Bajo acoplamiento:** los servicios se conocen solo a través de contratos de eventos, lo que permite agregar consumidores sin tocar a los productores (principio abierto/cerrado a nivel de sistema).
- **Escalabilidad horizontal por consumidor:** cada tipo de procesador escala de manera independiente según su carga (en Kafka, hasta el número de particiones del tópico).
- **Absorción de picos (*load leveling*):** el broker actúa como amortiguador cuando la tasa de llegada supera la de procesamiento.
- **Tolerancia a fallos parciales:** si un consumidor cae, los eventos esperan en el log y se procesan cuando vuelve.
- **Consistencia eventual y flujos de larga duración:** los procesos de negocio se modelan como secuencias de eventos (sagas) con compensaciones.
- **Trazabilidad natural:** el flujo de eventos es una bitácora del negocio que puede auditarse y re-procesarse.

2.4 Historia y evolución

La idea de reaccionar a sucesos es tan antigua como la computación interactiva (interrupciones de hardware, bucles de eventos de las interfaces gráficas), pero su consolidación como estilo arquitectónico de sistemas distribuidos se dio en varias olas:

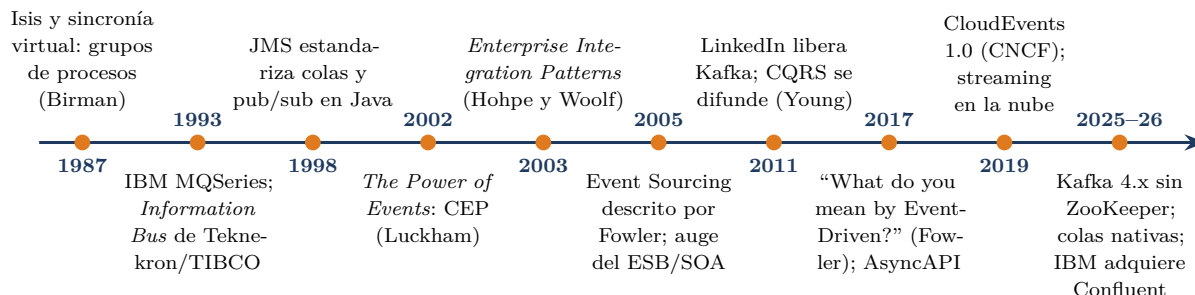


Figura 2.2: Línea de tiempo de la evolución de EDA (elaboración propia)

1. **Orígenes académicos (década de 1980).** La comunicación por grupos de procesos y la entrega ordenada de mensajes (proyecto Isis de Birman en Cornell) sentaron las bases teóricas del publicador-suscriptor [6].
2. **Middleware orientado a mensajes (1990s).** Productos como IBM MQSeries (1993) y el *Information Bus* de Teknekron, luego TIBCO Rendezvous, llevaron colas y pub/sub a bancos y bolsas de valores. En 1998 Java estandarizó la mensajería con JMS.
3. **Integración empresarial y SOA (2000s).** Hohpe y Woolf catalogaron los patrones de mensajería (2003) [3]; Luckham formalizó el procesamiento de eventos complejos (CEP) [5]. Surgió el ESB como bus inteligente, que con el tiempo se volvió un anti-patrón por concentrar lógica (*“smart pipes”*).
4. **Streaming y “tuberías tontas” (2010s).** LinkedIn creó Kafka como un log distribuido de alto rendimiento y lo liberó en 2011; pasó a ser proyecto de primer nivel de Apache en 2012 [8, 15]. El movimiento de microservicios adoptó la consigna de *smart endpoints, dumb pipes*. Greg Young popularizó CQRS y Fowler describió Event Sourcing y aclaró los cuatro significados de EDA (2017) [7].
5. **Estandarización y nube (2017–2022).** AsyncAPI se convirtió en el “OpenAPI de los eventos” y la CNCF publicó CloudEvents 1.0 (2019) para describir eventos de forma interoperable. Los proveedores de nube lanzaron buses gestionados (EventBridge, Event Grid, Pub/Sub).
6. **Madurez operativa e IA (2023–2026).** Kafka eliminó definitivamente ZooKeeper en la versión 4.0 (modo KRaft exclusivo), agregó colas nativas mediante *share groups* (preview en 4.1, disponibilidad general en 4.2, febrero de 2026) [13, 14, 17]. En marzo de 2026 IBM completó la compra de Confluent por unos 11 000 millones de dólares, presentando el streaming de eventos como la base de datos “en vivo” para agentes de IA [18].

2.5 Ventajas y desventajas

Tabla 2.4: Ventajas y desventajas de EDA (topología Broker)

Ventaja	Por qué y en qué condiciones
Desacople fuerte	Productores y consumidores evolucionan, se despliegan y fallan de forma independiente.
Extensibilidad	Nuevas capacidades (analítica, fidelización, antifraude) se agregan como consumidores nuevos sin modificar el código existente.
Escalabilidad y rendimiento	El trabajo se reparte por particiones; cada servicio escala según su carga. Kafka sostiene altos volúmenes con baja latencia.
Capacidad de respuesta	El usuario recibe una respuesta rápida (“reserva recibida”) mientras el resto del proceso continúa en segundo plano.
Resiliencia ante picos	El log amortigua ráfagas (venta de boletas de un concierto) sin perder solicitudes.
Auditoría y <i>replay</i>	El historial de eventos permite reconstruir vistas, depurar incidentes y alimentar nuevos consumidores con datos históricos.
Alineación con el negocio	Los eventos reflejan el lenguaje del dominio (<i>Event Storming</i>), lo que mejora la comunicación con expertos del negocio.
Desventaja	Por qué y cómo se mitiga
Consistencia eventual	No hay transacción global: el usuario puede ver estados intermedios. Mitigación: sagas, estados explícitos (PENDIENTE, CONFIRMADA) y notificaciones.
Complejidad de manejo de errores	Un fallo en el paso 4 no puede “deshacer” automáticamente los pasos 1–3. Mitigación: compensaciones, reintentos y DLQ.
Difícil de probar de extremo a extremo	El flujo está repartido y es asíncrono. Mitigación: pruebas de contrato, Testcontainers y pruebas de integración con Kafka real.
Observabilidad	“¿Dónde quedó mi reserva?” exige trazas distribuidas y ID de correlación en cada evento.
Duplicados y orden	La entrega es “al menos una vez”; el orden solo se garantiza por partición. Mitigación: consumidor idempotente y clave de partición por agregado.
Evolución de contratos	Un cambio de esquema afecta a consumidores desconocidos. Mitigación: versionado, compatibilidad hacia atrás y registro de esquemas.
Costo operativo	Operar Kafka y Elasticsearch en producción requiere experiencia (réplicas, particiones, retención, monitoreo).
Flujo implícito	En coreografía nadie “ve” el proceso completo. Mitigación: catálogo de eventos (AsyncAPI), diagramas dinámicos C4 y trazas.

2.6 Problemas comunes y patrones que los resuelven

La siguiente tabla relaciona los problemas típicos que aparecen al adoptar EDA con los patrones del catálogo del taller que los mitigan.

Tabla 2.5: Problemas recurrentes en EDA y patrones de solución

Problema	Descripción	Patrón / solución
<i>Dual write</i>	El servicio guarda en su base y publica en Kafka; si falla entre ambas operaciones, el sistema queda inconsistente.	Transactional Outbox (en el caso, <i>outbox</i> embebido en el mismo documento de Elasticsearch) + publicador de relevo.
Mensajes duplicados	Reintentos del productor o del consumidor generan entregas repetidas.	Idempotency (consumidor idempotente, tabla/índice de eventos procesados), productor idempotente de Kafka.
Orden de eventos	Eventos de una misma reserva procesados en desorden.	Clave de partición = ID del agregado; secuencia/versión en el evento.
Mensaje envenenado	Un evento mal formado bloquea la partición en reintentos infinitos.	Retry con <i>backoff</i> + Dead Letter Queue (tópico <i>*.dlq</i>).
Transacción distribuida	Reservar asiento y cobrar deben ser “todo o nada”.	Saga coreografiada + Compensating Transactions ; se evita 2PC .
Consultas entre servicios	Un servicio necesita datos de otro sin llamarlo en caliente.	Event-Carried State Transfer + Materialized View / CQRS .
Evolución de esquemas	Cambios en el contrato rompen consumidores.	Versionado (<i>reservas.v1</i>), compatibilidad hacia atrás, <i>Schema Registry</i> , <i>AsyncAPI</i> .
Depuración y trazabilidad	No se sabe por qué un evento no produjo efecto.	Distributed Tracing (OpenTelemetry), Structured Logging , ID de correlación y causalidad.
Contrapresión (<i>back-pressure</i>)	Consumidores más lentos que productores; crece el <i>lag</i> .	Escalar consumidores por partición, Throttling/Rate Limiting en la API, <i>share groups</i> .
Tormentas y ciclos de eventos	Un evento dispara otro que vuelve a disparar el primero.	Diseño de flujo acíclico, eventos con causalidad, revisiones en el catálogo.
Dependencia de servicios externos	La pasarela de pagos es lenta o falla.	Circuit Breaker , Timeout , Retry , Bulkhead (Polly).
Exposición del bus a clientes	La app móvil no debe hablar directamente con Kafka.	API Gateway / BFF y Server-Sent Events para notificaciones.

Problema	Descripción	Patrón / solución
Recuperación de vistas	Un índice de lectura se corrompe o cambia su estructura.	Event Replay desde el log de Kafka (retención adecuada).

2.7 Patrones aplicables y cuándo usarlos

El taller indica que no deben usarse todos los patrones, sino los que el estilo realmente necesita. La tabla 2.6 clasifica los patrones del catálogo por relevancia para EDA y señala cuáles se aplicarán en el caso práctico.

Tabla 2.6: Patrones del catálogo aplicables a EDA y su uso en el caso

Patrón	Categoría	Cuándo usarlo	En el caso
Obligatorios para el estilo			
Publish-Subscribe	Eventos	Siempre: varios consumidores independientes reaccionan al mismo hecho.	✓
Event Streaming	Eventos	Cuando se requiere log durable, orden por clave y reproceso.	✓
Dead Letter Queue	Eventos	Siempre que existan reintentos: aísla mensajes que no pueden procesarse.	✓
Idempotency	Consistencia	Siempre con entrega “al menos una vez”.	✓
Eventual Consistency	Consistencia	Siempre: es una propiedad inherente del estilo.	✓
Saga (coreografía)	Consistencia	Procesos de negocio que abarcan varios servicios.	✓
Compensating Transactions	Consistencia	Cuando un paso posterior falla y hay que revertir efectos (liberar asientos).	✓
Transactional Outbox	Integración	Cuando se guarda estado y se publica un evento en la misma operación.	✓
Database per Service	Datos	Para que cada servicio sea dueño de sus datos; evita <i>Database-as-IPC</i> .	✓
Distributed Tracing y Correlation ID	Observabilidad	Siempre en sistemas asíncronos multi-servicio.	✓
Recomendados			
Event-Carried State Transfer	Eventos	Para evitar consultas síncronas entre servicios.	✓
CQRS + Materialized View	Datos	Cuando las lecturas (búsqueda de eventos) dominan y tienen forma distinta a las escrituras.	✓
Event Notification	Eventos	Para avisos livianos (notificar al usuario).	✓
Event Replay	Eventos	Reconstruir índices o poblar un consumidor nuevo.	✓
API Gateway / BFF	Comunicación	Exponer una única fachada HTTP a la app Flutter.	✓

Patrón	Categoría	Cuándo usarlo	En el caso
Retry, Timeout, Circuit Breaker	Resiliencia	Llamadas a dependencias externas (pasarela de pagos simulada).	✓
Rate Limiting / Throttling	Resiliencia	Proteger la API en aperturas de venta masivas.	✓
Health Checks	Observabilidad	Contenedores y orquestadores necesitan saber si un servicio está vivo y listo.	✓
Repository	Acceso a datos	Encapsular el acceso a Elasticsearch detrás de una interfaz de dominio.	✓
Message Queue (<i>share groups</i>)	Eventos	Trabajo punto a punto sin orden estricto (envío de correos).	Opcional
Patrones de diseño de código (GoF) que emergen			
Observer	Comportamiento	Base conceptual de pub/sub; en Flutter, <i>streams</i> y <i>listeners</i> .	✓
Command	Comportamiento	Representar solicitudes (CrearReserva) antes de convertirlas en eventos.	✓
State	Comportamiento	Ciclo de vida de la reserva (Pendiente → Confirmada / Cancelada / Expirada).	✓
Strategy	Comportamiento	Políticas intercambiables (reintento, precio por localidad).	✓
Chain of Responsibility	Comportamiento	<i>Middleware</i> de consumo: deserializar → validar → deduplicar → manejar.	✓
Adapter	Estructural	Aislar Confluent.Kafka y el cliente de Elasticsearch detrás de puertos propios.	✓
Factory	Creacional	Construcción de eventos con metadatos CloudEvents consistentes.	✓
Mediator	Comportamiento	Despacho interno de manejadores en un servicio (no confundir con la topología Mediator).	Opcional
No aplicables o desaconsejados en este diseño			
Two-Phase Commit	Consistencia	Evitar: bloquea recursos y acopla disponibilidad; Kafka y Elasticsearch no lo ofrecen entre sí.	✗
Shared Database	Datos	Anti-patrón en EDA: acopla servicios por el esquema.	✗
Event Sourcing como almacén primario	Datos	Útil en dominios con auditoría estricta; excede el alcance del taller.	✗
Service Mesh	Comunicación	Aporta en decenas de servicios en Kubernetes; innecesario con Docker Compose.	✗

2.8 Casos de uso: cuándo usar EDA y cuándo no

Tabla 2.7: Criterios de adopción de EDA

Usar EDA cuando...	Evitar EDA cuando...
Varios subsistemas deben reaccionar al mismo hecho de negocio (pedido, pago, reserva).	El sistema es un CRUD sencillo con pocos usuarios y un solo equipo.
La carga es irregular y con picos (ventas flash, apertura de boletería).	Se requiere consistencia fuerte e inmediata entre entidades (p. ej. un libro contable que no tolere estados intermedios).
Se necesita integrar sistemas heterogéneos o legados sin acoplarlos.	El flujo es corto, secuencial y el usuario necesita la respuesta final en la misma solicitud.
Hay procesos de larga duración que dependen de terceros (pagos, envíos).	El equipo no tiene capacidad para operar un broker y observabilidad distribuida.
Se busca procesamiento en tiempo real (telemetría, fraude, IoT).	El dominio aún es muy incierto y cambian los contratos cada semana (costo alto de evolución de eventos).
Se requiere auditoría y posibilidad de reprocesar la historia.	El volumen es tan bajo que un monolito modular con un bus en memoria es suficiente (YAGNI).

🔗 Aplicación al caso: plataforma de reservas de eventos

La venta de tickets cumple los criterios de la izquierda: la apertura de venta genera picos extremos, la reserva involucra inventario, pagos y notificaciones que pueden evolucionar por separado, y la pasarela de pago es un tercero lento. El principal punto de la derecha (consistencia fuerte) se aborda con un *hold* temporal de asientos, control de concurrencia optimista en el índice de inventario y compensaciones por expiración.

2.9 Casos de aplicación en la industria

Tabla 2.8: Casos reales de EDA en la industria

Organización	Sector	Uso de eventos
LinkedIn	Red social	Creó Kafka para unificar la ingesta de actividad (vistas, clics, métricas) en un log central del que leen búsqueda, recomendaciones y analítica [8].
Netflix	Streaming de video	Su plataforma de datos en tiempo real (Keystone) usa Kafka como columna vertebral para mover eventos de reproducción y operación hacia procesamiento y almacenamiento.

Organización	Sector	Uso de eventos
Uber	Movilidad	Usa Kafka como bus de eventos para viajes, precios dinámicos, geolocalización y detección de fraude en tiempo real.
Zalando	Comercio electrónico	Construyó Nakadi, un bus de eventos con API REST sobre Kafka, para que sus equipos de microservicios publiquen y consuman eventos con contratos gobernados.
Nubank	Banca digital (LATAM)	Arquitectura de microservicios comunicados principalmente por mensajería asíncrona con Kafka, relevante por su escala en Brasil, México y Colombia.
Ticketmaster	Venta de boletas	Ha descrito públicamente el uso de Kafka para desacoplar la venta de tickets de los sistemas de inventario y notificación; es el referente más cercano al dominio del caso.
Cientes de Confluent	Transversal	Más de 6 500 empresas, incluido el 40 % del Fortune 500, usaban la plataforma de streaming de Confluent al momento de su adquisición por IBM (2026) [18].

2.10 Anti-patrones de riesgo para este estilo

Del catálogo de anti-patrones del taller, los siguientes son especialmente relevantes para EDA y serán vigilados durante el diseño:

- **Distributed Monolith:** servicios que deben desplegarse juntos porque comparten esquemas o dependen de respuestas síncronas en cadena. Se evita con contratos versionados y sin llamadas HTTP entre servicios en el flujo principal.
- **Database-as-IPC / Shared Database:** usar un mismo índice de Elasticsearch para que dos servicios “se comuniquen”. Se evita con *database per service*.
- **Chatty Microservices:** eventos demasiado finos o consultas de vuelta excesivas tras cada notificación. Se mitiga con *event-carried state transfer*.
- **Lack of Versioning / ausencia de versionado de esquemas:** romper consumidores al cambiar un evento. Se mitiga con versión en el tópico y en el tipo CloudEvents.
- **Lack of Observability:** sin trazas ni correlación el sistema es inoperable. Se mitiga con OpenTelemetry, Aspire Dashboard, Kibana y Kafbat UI.
- **Golden Hammer:** volver todo asíncrono. Las consultas del usuario (buscar eventos, ver su reserva) seguirán siendo síncronas vía REST.
- **Overengineering:** introducir Event Sourcing, Service Mesh o Kubernetes sin necesidad en un proyecto académico; se descartan explícitamente (YAGNI).
- **Works on My Machine / despliegue manual:** se mitiga con Docker Compose como único mecanismo de despliegue.

Investigación del stack tecnológico

Este capítulo analiza cada tecnología del stack con la misma estructura: definición, qué no es, características, historia, ventajas y desventajas, casos de uso y casos de aplicación. Al final se estudia la relación de cada tecnología con EDA y qué tan común es la combinación.

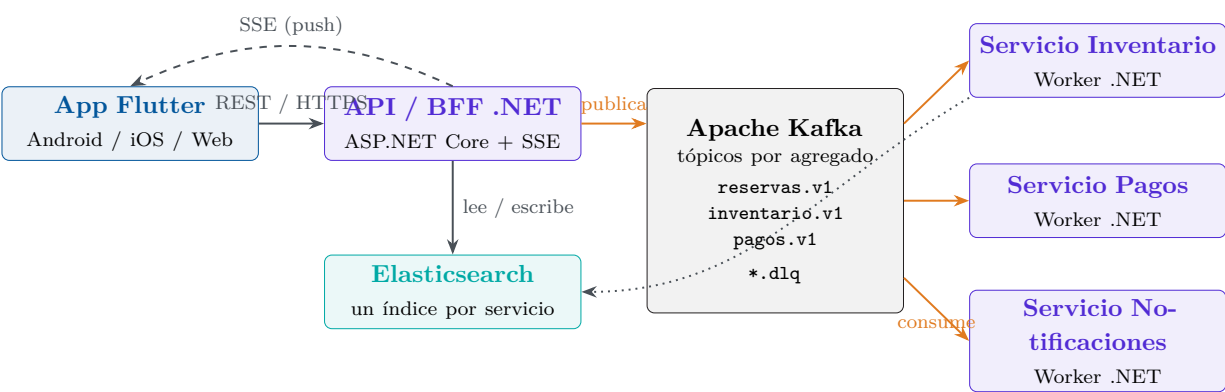


Figura 3.1: Vista preliminar del rol de cada tecnología en la solución (se detallará con el modelo C4 en el Capítulo 5)

3.1 Flutter (frontend)

Flutter – ficha técnica	
Creador / mantenedor	Google y comunidad de código abierto (licencia BSD-3)
Lenguaje	Dart (tipado estático, <i>null safety</i> obligatoria desde Dart 3)
Versión vigente	3.47.1 con Dart 3.13.1 (agosto de 2026) [31]
Plataformas	Android, iOS, Web, Windows, macOS, Linux y embebidos
Motor de render	Impeller (por defecto en móviles y, desde 3.47, en escritorio); Skia como legado

3.1.1 Definición

Definición
Flutter es un kit de desarrollo de interfaces de usuario (<i>UI toolkit</i>) de código abierto que permite construir aplicaciones compiladas para múltiples plataformas a partir de una única base de código en Dart. En lugar de envolver los componentes nativos de cada sistema operativo, Flutter dibuja cada píxel con su propio motor gráfico, lo que garantiza una

interfaz idéntica en todas las plataformas [28].

✗ Qué NO es

- **No es un lenguaje:** el lenguaje es Dart; Flutter es el framework de UI.
- **No es un híbrido basado en WebView** (como Ionic/Cordova): compila a código nativo ARM/x64 en modo *release*.
- **No es un “puente” a widgets nativos** (como React Native): los widgets son propios y se renderizan con Impeller/Skia.
- **No es un backend** ni un cliente de Kafka: en esta arquitectura consume una API HTTP y un flujo de Server-Sent Events.
- **No es la mejor opción para sitios web de contenido con SEO:** su salida web es un lienzo (*canvas*) orientado a aplicaciones.

3.1.2 Características principales

- **Todo es un widget:** la UI se describe de forma declarativa como un árbol inmutable de widgets que se reconstruye cuando cambia el estado.
- **Compilación dual:** JIT en desarrollo (habilita *hot reload* en menos de un segundo) y AOT en producción (arranque rápido y rendimiento cercano al nativo). En web compila a JavaScript o WebAssembly (estable desde la versión 3.22).
- **Motor propio:** Impeller precompila los *shaders* para eliminar los tirones de animación; Skia queda como alternativa.
- **Programación reactiva nativa:** Dart ofrece **Future** y **Stream** en el lenguaje, lo que encaja con notificaciones asíncronas (eventos que llegan por Server-Sent Events).
- **Gestión de estado plural:** Provider, Riverpod, BLoC (basado en *streams* de eventos), entre otros. El patrón BLoC es conceptualmente un EDA en miniatura dentro de la app.
- **Ecosistema pub.dev:** paquetes para HTTP (**dio**, **http**) y lectura de respuestas en *streaming*, suficientes para consumir Server-Sent Events.

3.1.3 Historia y evolución

Tabla 3.1: Hitos de Flutter

Año	Hito
2015	Presentado como “Sky” en la Dart Developer Summit: un motor capaz de renderizar a 120 cuadros por segundo en Android.
2017	Primera versión alfa (v0.0.6, mayo) [28].
2018	Flutter 1.0 estable (diciembre) para Android e iOS.
2021	Flutter 2 (marzo): soporte web estable y <i>null safety</i> [28].
2022	Flutter 3 (mayo) con Dart 2.17: escritorio estable en Windows, macOS y Linux.
2023	Dart 3: <i>null safety</i> obligatoria, <i>records</i> y <i>patterns</i> ; Impeller por defecto en iOS.
2024	WebAssembly estable para web (3.22) y Impeller maduro en Android.
2025	Flutter 3.35 y 3.38: <i>widget previews</i> , <i>dot shorthands</i> de Dart 3.10 y soporte para iOS 26 [30].
2026	Lanzamientos trimestrales 3.41 (feb.), 3.44 (may.) y 3.47 (ago.): Impeller por defecto en escritorio y Material/Cupertino separados en paquetes independientes [29, 31].

3.1.4 Ventajas y desventajas

Tabla 3.2: Ventajas y desventajas de Flutter

Ventajas	Desventajas
Una base de código para seis plataformas; reduce costo y tiempo.	Tamaño de binario mayor que una app nativa mínima.
UI consistente y control total del diseño.	Acceso a APIs nuevas del sistema operativo puede requerir <i>platform channels</i> .
<i>Hot reload</i> y herramientas de depuración maduras.	Dart es menos conocido que JavaScript/Kotlin/Swift (5,9 % de uso en la encuesta de Stack Overflow 2025) [32].
Rendimiento cercano al nativo gracias a AOT e Impeller.	Web: SEO y tamaño inicial de descarga limitados.
Modelo asíncrono (Stream) natural para eventos en tiempo real.	Ecosistema de gestión de estado fragmentado (muchas opciones).

3.1.5 Casos de uso

Cuándo usarlo: productos móviles multiplataforma con equipos pequeños; aplicaciones con UI de marca muy personalizada; MVP que deben llegar a Android e iOS a la vez; apps que reciben actualizaciones en tiempo real (estado de un pedido o una reserva).

Cuándo no usarlo: sitios web de contenido que dependen de SEO; apps que requieren integración muy profunda y temprana con APIs nuevas de iOS/Android; equipos ya especializados en nativo sin necesidad multiplataforma; juegos 3D intensivos (mejor un motor de juegos).

3.1.6 Casos de aplicación en la industria

Google (Google Pay y otras apps propias), BMW (app *My BMW*), Toyota (sistemas de infoentretenimiento), Alibaba (Xianyu, su plataforma de segunda mano) y eBay Motors han comunicado públicamente su uso de Flutter. En Latinoamérica, **Nubank** migró su app móvil a Flutter, y en Colombia hay vacantes específicas para desarrolladores Flutter en el sector financiero [44].

3.2 .NET y C# (backend)

.NET / ASP.NET Core – ficha técnica

Creador / mantenedor	Microsoft y .NET Foundation (licencia MIT)
Lenguajes	C# (principal), F#, Visual Basic
Versión vigente	.NET 10 LTS (11-nov-2025), parche 10.0.12 (8-sep-2026); soporte hasta 14-nov-2028 [20, 22]
Próxima versión	.NET 11 (STS), RC1 publicado el 8-sep-2026; lanzamiento esperado en noviembre de 2026 [20]
Componentes usados	ASP.NET Core (Minimal APIs), Worker Services, Server-Sent Events, YARP, Confluent.Kafka, Elastic.Clients.Elasticsearch, Polly, OpenTelemetry

3.2.1 Definición

Definición

.NET es una **plataforma de desarrollo** gratuita, de código abierto y multiplataforma que incluye un entorno de ejecución administrado (CLR, con compilación JIT y AOT), una amplia biblioteca de clases base, un SDK y frameworks de aplicación. **ASP.NET Core** es su framework para servicios web, APIs y aplicaciones en tiempo real; **C#** es su lenguaje principal, orientado a objetos y con fuerte soporte funcional y asíncrono (**async/await**) [23].

Qué NO es

- **No es .NET Framework:** .NET Framework (4.8) es la versión histórica solo para Windows, en mantenimiento. El .NET moderno (antes .NET Core) es multiplataforma y corre en contenedores Linux.
- **No es un único framework:** es una plataforma sobre la que se montan ASP.NET Core, EF Core, MAUI, Blazor, etc.

- **No es exclusivo de Windows ni de Azure:** se despliega en Linux, Docker, Kubernetes y cualquier nube.
- **No es un broker de mensajes:** se conecta a Kafka mediante el cliente oficial de Confluent (basado en `librdkafka`).

3.2.2 Características principales

- **Alto rendimiento:** el servidor Kestrel y las mejoras continuas del runtime ubican a ASP.NET Core entre los frameworks web más rápidos; soporta *Native AOT* para arranques en milisegundos y menor memoria en contenedores.
- **Asincronía de primera clase:** `async/await`, `IAsyncEnumerable` y `Channels` facilitan consumidores de eventos que no bloquean hilos.
- **Worker Services (`BackgroundService`):** plantilla nativa para procesos de larga duración, ideal para consumidores de Kafka.
- **Inyección de dependencias, configuración y *logging* integrados:** favorecen principios SOLID (DIP) y la configuración por variables de entorno en Docker.
- **Server-Sent Events nativos (.NET 10):** `TypedResults.ServerSentEvents` publica un `IAsyncEnumerable` como `text/event-stream`, suficiente para notificaciones servidor → cliente; `SignalR` queda como opción cuando se requiere comunicación bidireccional.
- **YARP:** *proxy* inverso de Microsoft construido sobre ASP.NET Core, que permite montar un API Gateway con el mismo lenguaje, autenticación y *rate limiting* del resto de servicios.
- **Resiliencia y observabilidad estandarizadas:** `Microsoft.Extensions.Resilience` (Polly) y `OpenTelemetry` con soporte oficial; *health checks* integrados.
- **Tipado fuerte y moderno:** *records* inmutables (ideales para modelar eventos), *pattern matching* y tipos anulables.

3.2.3 Historia y evolución

Tabla 3.3: Hitos de .NET

Año	Hito
2002	.NET Framework 1.0 y C# 1.0 (diseñado por Anders Hejlsberg), solo para Windows.
2004	Mono: implementación abierta y multiplataforma impulsada por la comunidad.
2016	.NET Core 1.0 (27 de junio): reescritura abierta y multiplataforma; nace ASP.NET Core [23].
2019	.NET Core 3.x: Worker Services, gRPC y mejoras de rendimiento.
2020	.NET 5 unifica la línea; ciclo anual cada noviembre, con versiones pares LTS.
2021–2023	.NET 6 (Minimal APIs), .NET 7 y .NET 8 (Native AOT para ASP.NET Core).
2025	.NET 10 LTS (11 de noviembre); TIOBE nombra a C# “lenguaje del año 2025” [39, 40].
2026	.NET 8 y 9 terminan soporte el 10-nov-2026; .NET 11 (STS) en RC [21].

3.2.4 Ventajas y desventajas

Tabla 3.4: Ventajas y desventajas de .NET para servicios orientados a eventos

Ventajas	Desventajas
Rendimiento y eficiencia de memoria altos; imágenes Docker pequeñas con AOT.	El cliente de Kafka es un envoltorio de <code>librdkafka</code> (C): menos idiomático que el cliente Java nativo y sin Kafka Streams oficial.
Plataforma integral: DI, configuración, resiliencia, <i>health checks</i> , OpenTelemetry.	Ciclo de actualización anual exige migraciones periódicas (LTS cada dos años).
Tipado fuerte y <i>records</i> para contratos de eventos.	Percepción histórica de “tecnología Windows” en algunos equipos.
Ecosistema empresarial y de mensajería (MassTransit, NServiceBus, Wolverine).	Varias librerías de mensajería maduras son de licencia comercial para uso empresarial.
Gran demanda laboral en Colombia (sector financiero, seguros, gobierno).	Menos ejemplos de EDA con Kafka que en Java/Spring.

3.2.5 Casos de uso

Cuándo usarlo: APIs y microservicios empresariales, servicios con alta concurrencia de E/S, consumidores y productores de eventos de larga duración, sistemas con requisitos de seguridad y soporte a largo plazo, organizaciones con ecosistema Microsoft/Azure.

Cuándo no usarlo: procesamiento de streams con estado complejo (Kafka Streams/Flink en

JVM son más maduros); *scripts* o prototipos de ciencia de datos (Python domina); funciones *serverless* muy pequeñas cuando no se usa AOT.

3.2.6 Casos de aplicación en la industria

Stack Overflow opera su plataforma sobre .NET; múltiples servicios de Microsoft (Bing, Azure, Teams) usan .NET moderno a gran escala. En Colombia es una de las tecnologías de backend con más vacantes: Computrabajo registra más de mil salarios reportados para “Desarrollador .NET” [41].

3.3 Elasticsearch (persistencia)

Elasticsearch – ficha técnica	
Creador / mantenedor	Shay Banon; Elastic N.V.
Base tecnológica	Java sobre Apache Lucene (índice invertido)
Licencia	Triple licencia: AGPLv3 (código abierto, desde agosto de 2024), SSPL y Elastic License
Versión vigente	9.5 (4-ago-2026), parche 9.5.4; 9.4 con soporte; 8.19 con soporte hasta julio de 2027 [24]
Cliente .NET	Elastic.Clients.Elasticsearch 9.5.x, con proveedor LINQ a ES QL desde 9.3.4 [25, 26]

3.3.1 Definición

Definición
Elasticsearch es un motor distribuido de búsqueda y analítica que almacena documentos JSON sin esquema rígido y los indexa para consultas de texto completo, filtros, agregaciones y búsqueda vectorial en tiempo casi real. Expone todas sus operaciones mediante una API REST y escala horizontalmente dividiendo cada índice en <i>shards</i> replicados entre nodos [27].

✗ Qué NO es
■ No es una base de datos relacional: no tiene <i>joins</i> generales ni integridad referencial; se modela desnormalizando. ■ No ofrece transacciones ACID multi-documento: la atomicidad está garantizada <i>solo por documento</i>. ■ No es consistente de inmediato para búsquedas: un documento indexado aparece en las búsquedas tras el siguiente <i>refresh</i> (1 s por defecto). La lectura por <code>_id</code> (GET) sí es en tiempo real. ■ No es un broker de mensajes ni una caché: aunque sea rápido, no reemplaza a Kafka

ni a Redis.

- **No es (ya) solo “la E del ELK”:** hoy también es base vectorial y motor de relevancia para IA generativa.

3.3.2 Características principales

- **Índice invertido (Lucene):** búsquedas de texto completo con análisis lingüístico (tokenización, *stemming* en español, sinónimos) y relevancia BM25.
- **Distribución:** *shards* primarios y réplicas; tolerancia a la caída de nodos y paralelismo de consultas.
- **Tiempo casi real (NRT):** escritura durable vía *translog* y visibilidad en búsquedas tras el *refresh*.
- **Control de concurrencia optimista:** cada documento tiene `_seq_no` y `_primary_term`; una escritura condicionada con `if_seq_no/if_primary_term` falla con `409 Conflict` si otro proceso lo modificó antes. Es la herramienta clave para el inventario de asientos.
- **Creación idempotente:** `op_type=create` con un `_id` determinista (el ID del evento) rechaza duplicados, lo que implementa deduplicación en el consumidor.
- **Lenguajes de consulta:** Query DSL (JSON), SQL y ES|QL (lenguaje de tuberías).
- **Ecosistema:** Kibana (visualización y administración), Beats/Logstash/Elastic Agent (ingesta) y conector *sink* para Kafka Connect.

3.3.3 Historia y evolución

Tabla 3.5: Hitos de Elasticsearch

Año	Hito
2004	Shay Banon crea Compass, una capa de búsqueda sobre Lucene (origen del proyecto).
2010	Primera versión de Elasticsearch (febrero), diseñada como distribuida desde el inicio.
2012	Se funda la empresa Elasticsearch BV (hoy Elastic); se consolida el stack ELK con Logstash y Kibana.
2018	Elastic sale a bolsa; X-Pack se integra al código principal.
2021	Cambio de licencia de Apache 2.0 a SSPL/Elastic License; AWS crea el <i>fork</i> OpenSearch.
2022	Versión 8.0: seguridad activada por defecto y búsqueda vectorial kNN nativa.
2024	Elastic agrega AGPLv3 como opción: Elasticsearch vuelve a ser código abierto.
2025	Versión 9.0 (abril) sobre Lucene 10; 9.1 y 9.2 en el mismo año [24].
2026	Versiones 9.3 (feb.), 9.4 (may.) y 9.5 (ago.); el cliente .NET agrega un proveedor LINQ que traduce a ES QL [26].

3.3.4 Ventajas y desventajas

Tabla 3.6: Ventajas y desventajas de Elasticsearch como almacén de los servicios

Ventajas	Desventajas
Búsqueda de eventos por texto, fecha, ciudad y categoría con relevancia y facetas.	Sin transacciones multi-documento: obliga a modelar agregados completos en un solo documento.
Esquema flexible: acompaña la evolución de los eventos.	Consistencia de búsqueda casi real (no inmediata).
Escala horizontal y alta disponibilidad con réplicas.	Consumo de memoria alto (JVM); en local requiere ajustar <i>heap</i> .
Concurrencia optimista e inserción idempotente nativas.	Cambios de <i>mapping</i> de campos existentes exigen reindexar.
Agregaciones para analítica (ventas por evento, ocupación).	Uso como base primaria es menos común que PostgreSQL/SQL Server; menos guías de modelado transaccional.
Kibana para inspeccionar datos y logs durante la demostración.	Licenciamiento con cláusulas a revisar para servicios gestionados.

3.3.5 Casos de uso

Cuándo usarlo: catálogos y búsqueda de productos/eventos; observabilidad (logs, métricas y trazas); analítica en tiempo casi real; vistas materializadas (lado de lectura de CQRS); búsqueda semántica y vectorial.

Cuándo no usarlo: sistemas contables o transaccionales con invariantes entre muchas entidades (usar una base relacional); datos altamente relacionales con muchas uniones; como cola o bus de mensajes; cuando no se tolera el retardo del *refresh* en las búsquedas.

🔗 Aplicación al caso: plataforma de reservas de eventos

Elasticsearch es *naturalmente* adecuado para el catálogo de eventos (búsqueda por nombre, artista, ciudad y fecha) y para las vistas de consulta de reservas. Para el inventario de asientos, el grupo aprovechará la atomicidad por documento y el control de concurrencia optimista: cada localidad es un documento con su contador de cupos, y una reserva solo descuenta cupos si la versión leída sigue vigente. Esta decisión, y sus límites, se documentan en el ADR-003 (Capítulo 4).

3.3.6 Casos de aplicación en la industria

Elasticsearch es la base del stack de observabilidad y seguridad (SIEM) de Elastic y se usa ampliamente para búsqueda en comercio electrónico y portales de contenido. GitHub lo empleó durante años para su búsqueda de código antes de construir un motor propio (2023); empresas como Netflix y Uber lo han usado para búsqueda y análisis de logs. En la encuesta de Stack Overflow 2025, el 16,7 % de los encuestados (17,8 % de los profesionales) reportó trabajo extenso con Elasticsearch [32].

3.4 Apache Kafka (integración)

Apache Kafka – ficha técnica	
Creador / mantenedor	Creado en LinkedIn (J. Kreps, N. Narkhede, J. Rao); Apache Software Foundation (licencia Apache 2.0)
Lenguajes	Java y Scala (servidor); clientes en múltiples lenguajes
Versión vigente	4.2.0 (16/17-feb-2026), solo modo KRaft [15, 14]
Novedades 4.x	Eliminación de ZooKeeper, nuevo coordinador de grupos, colas nativas (<i>share groups</i> , KIP-932) en disponibilidad general desde 4.2 [13, 17]
Cliente .NET	Confluent.Kafka (sobre librdkafka)

3.4.1 Definición

Definición

Apache Kafka es una **plataforma distribuida de streaming de eventos**: un *log* de solo-anexar (*append-only*), particionado, replicado y durable, en el que los productores escriben registros y los consumidores los leen a su propio ritmo manteniendo su posición (*offset*). Combina tres capacidades: publicar/suscribirse a flujos de eventos, almacenarlos de forma durable el tiempo que se desee y procesarlos a medida que ocurren [11, 8].

Qué NO es

- **No es una cola tradicional:** los mensajes no se borran al consumirse; permanecen según la política de retención y pueden releerse. (Desde 4.2, los *share groups* añaden semántica de cola, pero sobre el mismo log).
- **No es un ESB:** no transforma ni enruta con reglas de negocio (“tubería tonta”); la lógica vive en los servicios.
- **No es una base de datos para consultas ad hoc:** no se consulta “la reserva 123”; para eso están las vistas en Elasticsearch.
- **No es RPC:** no hay respuesta directa a un mensaje; la “respuesta” es otro evento.
- **No es trivial de operar:** particiones, réplicas y retención deben diseñarse.

3.4.2 Conceptos y características principales

Tabla 3.7: Conceptos de Kafka y su función en EDA

Concepto	Función
Tópico	Canal lógico de eventos del mismo tipo o agregado (<code>reservas.v1</code>).
Partición	Unidad de orden y paralelismo. El orden solo se garantiza dentro de una partición; la clave del mensaje determina la partición.
Offset	Posición de un consumidor en la partición; permite reanudar y reprocesar (<i>replay</i>).
Grupo de consumidores	Reparte las particiones entre instancias de un mismo servicio (escalado horizontal); cada grupo recibe todos los eventos (pub/sub).
<i>Share group</i> (4.x)	Varios consumidores leen la misma partición con confirmación por mensaje y conteo de reintentos (semántica de cola) [13].
Replicación / ISR	Cada partición tiene un líder y réplicas; <code>acks=all</code> con <code>min.insync.replicas</code> evita pérdida de datos.
Retención y compactación	Por tiempo/tamaño o conservando el último valor por clave (tópicos compactados).
Productor idempotente y transacciones	Evitan duplicados por reintento y permiten escritura atómica en varios tópicos (<i>exactly-once</i> dentro de Kafka).
KRaft	Protocolo de consenso Raft interno para metadatos; reemplaza a ZooKeeper desde 4.0.
Kafka Connect / Streams	Integración declarativa con sistemas externos (p. ej. <i>sink</i> a Elasticsearch) y procesamiento de flujos en JVM.

3.4.3 Historia y evolución

Tabla 3.8: Hitos de Apache Kafka

Año	Hito
2010–2011	Creado en LinkedIn para unificar la ingesta de actividad; liberado como código abierto en enero de 2011 [15].
2012	Se gradúa de la incubadora como proyecto de primer nivel de Apache.
2013	Kreps publica “The Log”, manifiesto conceptual del streaming [8].
2014	Sus creadores fundan Confluent.
2016–2017	Kafka Streams (0.10), productor idempotente y transacciones (0.11); versión 1.0.
2021–2022	KRaft en acceso anticipado (2.8) y listo para producción (3.3).
2025	Kafka 4.0 (marzo) elimina ZooKeeper; 4.1 (2-sep-2025) trae colas en <i>preview</i> [16].
2026	Kafka 4.2 (febrero): colas en disponibilidad general, nuevo protocolo de rebalanceo de Streams y DLQ en Streams [14, 17]. IBM completa la adquisición de Confluent (17-mar-2026) [18].

3.4.4 Ventajas y desventajas

Tabla 3.9: Ventajas y desventajas de Kafka como bus de eventos

Ventajas	Desventajas
Alto rendimiento (escritura secuencial, <i>batching</i> , compresión) y baja latencia.	Curva de aprendizaje: particiones, offsets, rebalanceos y garantías de entrega.
Durabilidad y <i>replay</i> : los eventos son la fuente para reconstruir vistas.	Operación exigente en producción (capacidad, monitoreo de <i>lag</i> , réplicas).
Escalado horizontal por particiones y grupos de consumidores.	Orden solo por partición; número de particiones difícil de reducir.
Ecosistema enorme (Connect, Streams, Schema Registry, clientes).	No hay enrutamiento flexible como en RabbitMQ (exchanges) ni prioridades.
Desde 4.x: arquitectura más simple (sin ZooKeeper) y colas nativas.	Riesgo de dependencia de proveedor en servicios gestionados (Confluent/IBM, MSK).
Estándar de facto: más del 80 % de las empresas Fortune 100 lo usan, según Apache [12].	Excesivo para volúmenes bajos o integraciones sencillas.

3.4.5 Casos de uso

Cuándo usarlo: columna vertebral de EDA entre microservicios; integración de datos en tiempo real (CDC, *pipelines*); procesamiento de streams; agregación de logs y métricas; auditoría y *replay*; alimentar modelos y agentes de IA con datos vivos.

Cuándo no usarlo: colas de tareas simples de bajo volumen (RabbitMQ o una cola gestionada son más sencillos); comunicación solicitud-respuesta; mensajería directa con clientes móviles; cuando el equipo no puede operar un clúster y no hay presupuesto para un servicio gestionado.

3.4.6 Casos de aplicación en la industria

LinkedIn (origen), Netflix, Uber, Nubank, Zalando y Ticketmaster (ver Tabla 2.8). La adquisición de Confluent por IBM, con más de 6 500 clientes empresariales, confirma que el streaming de eventos es infraestructura estratégica y no un nicho [18].

3.5 Protocolos de integración y componentes adicionales

Tabla 3.10: Protocolos de comunicación definidos para la solución

Protocolo	Entre	Uso y justificación	Tipo
REST sobre HTTPS (JSON)	App Flutter ↔ API/BFF .NET	Comandos (crear reserva) y consultas (buscar eventos). Universal, fácil de depurar y con OpenAPI.	Síncrono
Server-Sent Events (SSE)	API .NET → App Flutter	Empujar cada cambio de estado de la reserva sin <i>polling</i> ; HTTP estándar, unidireccional y con reconexión.	Tiempo real
Protocolo binario de Kafka sobre TCP	Servicios .NET ↔ Kafka	Publicación y consumo de eventos de dominio. Sobre CloudEvents en formato JSON.	Asíncrono
HTTP/JSON (API REST de Elasticsearch)	Servicios .NET → Elasticsearch	Persistencia y consultas mediante el cliente oficial.	Síncrono

Componentes adicionales. **Docker y Docker Compose** (contenedores obligatorios del taller; Docker es la herramienta de nube más usada, con 71,1 % en Stack Overflow 2025 [32]); **Kibana** para inspeccionar índices y logs; **YARP** como API Gateway; **Kafbat UI** (continuación mantenida del antiguo Kafka UI) para observar tópicos, particiones y *lag*; **OpenTelemetry** con el **Aspire Dashboard** para ver trazas, métricas y logs; **Polly** para reintentos, *timeouts* y *circuit breaker*; **AsyncAPI** y **CloudEvents** para documentar y estandarizar contratos de eventos; **Testcontainers** para pruebas de integración con Kafka y Elasticsearch reales.

3.6 Relación entre el estilo y las tecnologías

Tabla 3.11: Cómo cada tecnología habilita (o tensiona) las propiedades de EDA

Tecnología	Rol en EDA	Propiedades del estilo que habilita	Tensión
Kafka	Broker / log de eventos (el “canal” del estilo)	Desacople temporal y espacial; pub/sub con grupos; orden por clave; durabilidad y <i>replay</i> ; escalado por particiones; colas con <i>share groups</i> .	Entrega al menos una vez; operación; <i>share groups</i> aún sin soporte en el cliente .NET.
.NET	Productores y procesadores de eventos; API/BFF	<i>Worker Services</i> para consumidores de larga duración; <i>async/await</i> ; <i>records</i> inmutables como eventos; Polly y OpenTelemetry para resiliencia y trazas; Server-Sent Events nativos para empujar eventos al cliente.	Sin Kafka Streams nativo.
Elasticsearch	Almacén propio de cada servicio y vistas materializadas (CQRS)	<i>Database per service</i> ; lado de lectura optimizado para búsqueda; inserción idempotente por <i>_id</i> ; concurrencia optimista; esquema flexible para eventos que evolucionan.	Sin transacciones multi-documento.
Flutter	Iniciador de comandos y receptor de notificaciones	Modelo reactivo con Stream ; patrón BLoC (eventos → estados) coherente con el estilo; UI que muestra estados intermedios de la saga (“procesando pago”).	Debe manejar la consistencia eventual en la UX.

★ Idea clave

La coherencia del stack con EDA es alta en tres de sus cuatro piezas (Kafka, .NET y Flutter). Elasticsearch es coherente como *lado de lectura* y como almacén por servicio, pero su falta de transacciones multi-documento *refuerza* la necesidad de los patrones propios de EDA: agregados en un solo documento, *outbox* embebido, idempotencia y sagas. En otras palabras, el estilo compensa la limitación de la base de datos, y la base de datos obliga a aplicar el estilo con disciplina.

3.7 ¿Qué tan común es el stack asignado?

Ninguna fuente pública mide la adopción del stack completo como unidad, por lo que se analiza por **pares de tecnologías**, que es como se integran en la práctica. La calificación (0–5) es una *estimación del grupo* basada en la evidencia citada.

Tabla 3.12: Frecuencia de combinación de las tecnologías del stack (estimación del grupo)

Par	Frecuencia	Evidencia y observaciones
Kafka + Elasticsearch	●●●●●	Combinación clásica: Kafka Connect ofrece un conector <i>sink</i> a Elasticsearch y las arquitecturas de logs (ELK) usan Kafka como amortiguador.
.NET + Kafka	●●●●●	Cliente oficial Confluent.Kafka mantenido por Confluent; frecuente en banca y seguros con ecosistema Microsoft. Menos habitual que Java + Kafka.
.NET + Elasticsearch	●●●●●	Cliente oficial (9.5.x) y <i>sinks</i> de <i>logging</i> (Serilog). Común para búsqueda y observabilidad.
Flutter + .NET	●●●●●	Patrón habitual “app móvil + backend corporativo” en LATAM; la app consume REST y SSE estándar, sin depender de clientes específicos de .NET.
Flutter + Kafka	●●●●●	No se conectan directamente (ni se debe): el móvil no es un consumidor confiable de Kafka. Se integran a través del backend.
Flutter + Elasticsearch	●●●●●	Tampoco se conectan directamente (expondría el clúster). La app consulta a la API.
Elasticsearch como base primaria	●●●●●	Poco común: lo usual es PostgreSQL o SQL Server como fuente de verdad y Elasticsearch como réplica de lectura. Es viable si se diseñan agregados por documento y concurrencia optimista.
Stack completo	●●●●●	Poco frecuente como conjunto, pero cada integración real (móvil → API → Kafka → servicios → índices) está bien documentada.

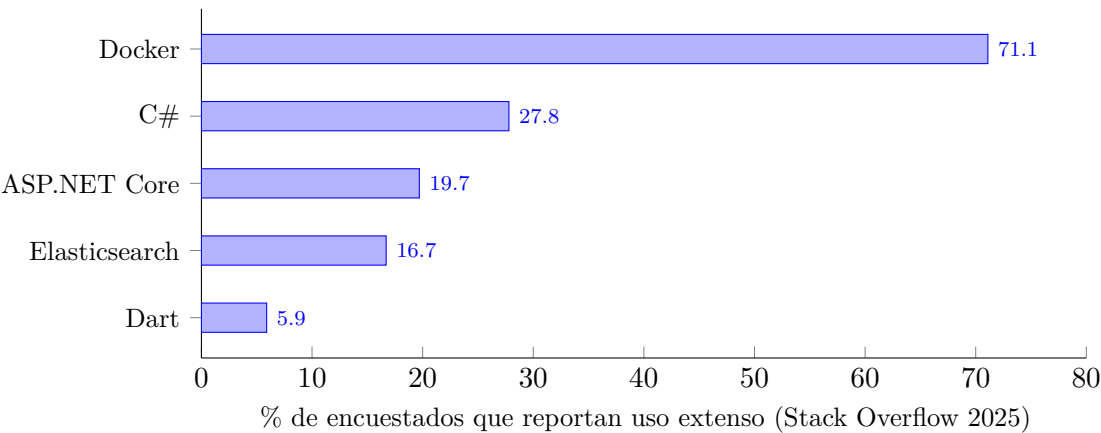


Figura 3.2: Adopción de tecnologías del stack en la encuesta Stack Overflow 2025 (todos los encuestados) [32]. Flutter no aparece en la categoría web de 2025; en 2024 registró 9,4 % en “otros frameworks”.

Análisis arquitectónico

Este capítulo desarrolla las cuatro matrices exigidas por el taller. Cada matriz se evalúa para la topología Broker de EDA y, cuando aplica, se contrasta con el stack concreto. La escala de soporte usada es:

++	soporta fuertemente	+	soporta	±	neutro / depende del diseño
-	limita	--	limita fuertemente		

4.1 Matriz de atributos de calidad vs. estilo

Los atributos se toman del modelo de calidad de producto ISO/IEC 25010:2023 [10], complementado con atributos arquitectónicos de uso común (escalabilidad, elasticidad, despleabilidad, observabilidad y costo) propuestos por Richards y Ford [1].

Tabla 4.1: Matriz de atributos de calidad vs. EDA (topología Broker) y mitigaciones con el stack

Atributo (ISO 25010)	Soporte	Cómo lo soporta el estilo	Cómo lo limita el estilo	Mitigación / aprovechamiento con el stack
Eficiencia de desempeño				
Comportamiento temporal (latencia percibida)	++	El usuario recibe respuesta en cuanto se registra el comando; el trabajo pesado es asíncrono.	La latencia de extremo a extremo del proceso completo aumenta (varios saltos por el broker).	Kafka con <code>linger.ms</code> bajo y compresión; ASP.NET Core responde 202 Accepted; Server-Sent Events notifican el resultado.
Capacidad / <i>throughput</i>	++	Procesamiento paralelo por particiones y consumidores independientes.	El paralelismo máximo por grupo está acotado por el número de particiones.	Definir particiones por tópico según la carga esperada (p. ej. 6 para <code>reservas.v1</code>).
Utilización de recursos	±	Los consumidores ociosos consumen poco; se escala solo lo necesario.	Broker, réplicas y JVM de Kafka y Elasticsearch consumen memoria considerable.	Límites de memoria en Docker Compose; .NET con <i>Native AOT</i> o imágenes <i>chiseled</i> .
Fiabilidad				
Disponibilidad	++	La caída de un consumidor no detiene a los productores; los eventos esperan en el log.	El broker se vuelve componente crítico: si cae Kafka, no se publican eventos.	Réplicas (<code>replication.factor=3</code> , <code>min.insync.replicas=2</code>) en producción; en la demo, un nodo con <i>health checks</i> .
Tolerancia a fallos	+	Aislamiento de fallos por servicio; reintentos y DLQ evitan pérdidas.	Fallos parciales dejan el proceso a medias (reserva sin pago).	Saga con compensaciones; Polly (<i>retry</i> , <i>circuit breaker</i>); tópicos <code>*.dlq</code> .
Recuperabilidad	++	Los consumidores retoman desde su último offset; las vistas pueden reconstruirse con <i>replay</i> .	Requiere retención suficiente y consumidores idempotentes.	Retención de 7 días en tópicos de dominio; índices reconstruibles desde Kafka.

Atributo	Soporte	Soporta	Limita	Stack
Madurez / consistencia de datos	—	—	Consistencia eventual: estados intermedios visibles y posibles lecturas desactualizadas.	Estados explícitos en la entidad Reserva; lecturas por <code>_id</code> (tiempo real) en Elasticsearch para el estado crítico.
Mantenibilidad				
Modularidad	++	Servicios pequeños con responsabilidades claras que se comunican por contratos.	Riesgo de fragmentar en exceso (nano-servicios).	Tres servicios de dominio + API/BFF; límites alineados a subdominios.
Modificabilidad / extensibilidad	++	Agregar un consumidor no afecta a los demás (abierto/cerrado a nivel de sistema).	Cambiar un evento existente impacta a consumidores desconocidos.	Versionado de tópicos y tipos CloudEvents; cambios solo aditivos en v1.
Reusabilidad	+	Un mismo evento sirve a múltiples propósitos (inventario, analítica, notificación).	Eventos demasiado genéricos pierden significado de negocio.	Eventos de dominio con nombres del lenguaje ubicuo.
Analizabilidad	—	El log sirve como bitácora de negocio.	El flujo es implícito; es difícil seguir una reserva a través de servicios.	OpenTelemetry con propagación de contexto en cabeceras Kafka; <code>correlationId</code> en logs; Kibana.
Capacidad de prueba (testabilidad)	—	Cada procesador se prueba aislado con eventos de entrada/salida.	Pruebas de extremo a extremo asíncronas, no deterministas en tiempo.	Testcontainers (Kafka y Elasticsearch reales); pruebas de contrato de eventos; xUnit.
Flexibilidad (portabilidad y escalabilidad)				
Escalabilidad	++	Escalado horizontal independiente de cada procesador.	Particiones calientes si la clave está mal distribuida.	Clave de partición = <code>reservaId</code> / <code>localidadId</code> ; réplicas de servicios en Compose.
Elasticidad	+	El broker amortigua picos mientras se agregan consumidores.	Rebalanceos de grupo pausan el consumo brevemente.	Nuevo protocolo de rebalanceo de Kafka 4.x; <i>share groups</i> para trabajo sin orden.

Atributo	Soporte	Soporta	Limita	Stack
Adaptabilidad / instalabilidad	+	Servicios empaquetados en contenedores independientes.	Más piezas que instalar y configurar.	Un único <code>docker compose up</code> ; configuración por variables de entorno.
Seguridad				
Confidencialidad e integridad	±	Superficie expuesta mínima: solo la API es pública; el bus es interno.	Los datos viajan por más canales y quedan persistidos en el log.	TLS/SASL en Kafka (producción), seguridad activada en Elasticsearch, JWT en la API; no incluir datos de tarjeta en eventos.
Autenticidad y trazabilidad (no repudio, responsabilidad)	+	Cada evento es un registro inmutable con fuente y marca de tiempo.	Si no se firma ni se controla el acceso, cualquiera con acceso al tópico puede publicar.	ACL por tópico en producción; campo <code>source</code> y <code>subject</code> de CloudEvents.
Compatibilidad				
Interoperabilidad	++	Productores y consumidores en tecnologías distintas comparten contratos.	Sin estándar de esquema, cada equipo inventa su formato.	CloudEvents + JSON Schema; documentación AsyncAPI.
Coexistencia	+	Sistemas legados pueden publicar/consumir eventos sin cambiar a los demás.	—	Kafka Connect para integraciones futuras.
Capacidad de interacción (usabilidad) y adecuación funcional				
Capacidad de interacción	±	Respuestas rápidas y notificaciones en tiempo real mejoran la experiencia.	El usuario debe entender estados como “procesando”.	Flutter con BLoC mostrando estados de la saga; Server-Sent Events para empujar cambios.
Adecuación funcional (corrección)	±	Los eventos reflejan hechos del dominio.	Duplicados o desorden pueden producir resultados incorrectos si no se diseñan.	Idempotencia por <code>eventId</code> , concurrencia optimista en inventario.
Atributos arquitectónicos adicionales				
Desplegabilidad	+	Servicios se despliegan de forma independiente.	Coordinación de versiones de contratos.	Imágenes por servicio; compatibilidad hacia atrás.

Atributo	Soporte	Soporta	Limita	Stack
Observabilidad	—	Métricas de <i>lag</i> revelan cuellos de botella.	Sin instrumentación el sistema es una caja negra.	Kafbat UI, Kibana, OpenTelemetry + Aspire Dashboard.
Simplicidad	— —	—	Es de los estilos más complejos de razonar, probar y operar.	Alcance acotado (un caso de uso), YAGNI explícito.
Costo	—	Pago por lo que se escala.	Infraestructura adicional (broker, índices, observabilidad).	Todo local en contenedores para el taller.

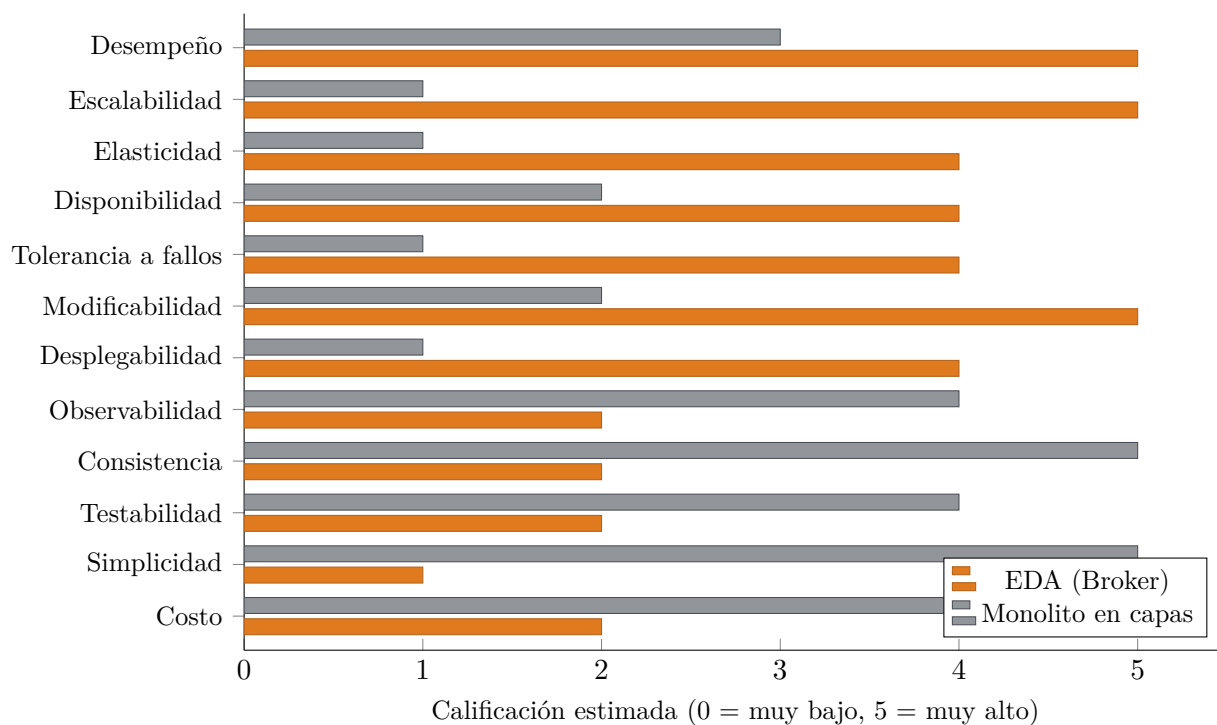


Figura 4.1: Perfil de atributos de EDA frente a un monolito en capas (estimación del grupo inspirada en [1])

★ Idea clave

EDA intercambia **simplicidad, consistencia inmediata y testabilidad** por **escalabilidad, desempeño percibido, disponibilidad y extensibilidad**. En el dominio de reservas este intercambio es favorable, siempre que el diseño gestione explícitamente la consistencia eventual (sagas) y la observabilidad (trazas y correlación).

4.2 Matriz de principios de diseño vs. estilo

La matriz evalúa cómo la topología Broker de EDA *favorece* o *tensiona* cada principio, y cómo se aplica concretamente en .NET y Flutter. STUPID se analiza como conjunto de anti-principios: “cumplir” significa *evitarlos*.

Tabla 4.2: Matriz de principios de diseño vs. EDA (topología Broker)

Principio	Nivel	Cómo lo favorece el estilo	Riesgo o tensión	Aplicación en el stack
SOLID				
S – Responsabilidad única	++	Cada procesador reacciona a un tipo de evento y tiene una sola razón para cambiar (inventario, pagos, notificaciones).	Un “servicio de todo” que consume todos los tópicos (<i>God Service</i>).	Un <code>EventHandler<T></code> por evento; un <i>Worker</i> por servicio; en Flutter, un <code>BLoC</code> por pantalla/flujo.
O – Abierto/cerrado	++	Nuevas funciones se agregan como nuevos consumidores sin modificar productores.	Modificar el esquema de un evento existente viola el principio a nivel de sistema.	Nuevos <i>handlers</i> registrados por DI; eventos versionados (v1, v2).
L – Sustitución de Liskov	+	Cualquier consumidor que respete el contrato del evento puede reemplazar a otro.	Consumidores que dependen de efectos no documentados de otro.	Interfaces <code>IEventPublisher</code> e <code>IReservaRepository</code> con implementaciones Kafka/Elasticsearch o <i>fakes</i> en pruebas.
I – Segregación de interfaces	+	Cada consumidor se suscribe solo a los eventos que necesita.	Eventos “gordos” que obligan a depender de campos irrelevantes.	Eventos específicos (<code>PagoAprobado</code> , <code>PagoRechazado</code>) en vez de un <code>PagoActualizado</code> genérico.
D – Inversión de dependencias	++	Productores y consumidores dependen de una abstracción (el contrato del evento), no entre sí.	Filtrar tipos de <code>Confluent.Kafka</code> al dominio.	Arquitectura hexagonal interna: el dominio define puertos; Kafka y Elasticsearch son adaptadores.
Principios de simplicidad				
KISS	–	Cada procesador es simple por separado.	El sistema en conjunto es complejo (asincronía, fallos parciales).	Alcance mínimo: un flujo de negocio, tres servicios, un bus; sin Service Mesh ni Kubernetes.
DRY	±	Contratos de eventos compartidos como única fuente de verdad.	Duplicación de datos entre servicios (intencional, por <i>event-carried state</i>); riesgo de librería compartida que acople.	Proyecto <code>Contracts</code> con <i>records</i> de eventos (solo DTO, sin lógica); duplicación de datos aceptada y documentada.

Principio	Cumpl.	Favorece	Riesgo	Stack
YAGNI	±	Permite empezar con pocos consumidores y agregar más cuando se necesiten.	Tentación de agregar Event Sourcing, Schema Registry o CQRS completo “por si acaso”.	Se descartan explícitamente Event Sourcing, Kafka Streams y Service Mesh para esta entrega.
Principios de comportamiento y acoplamiento				
PoLA (menor sorpresa)	±	Eventos con nombres del dominio en pasado son autoexplicativos.	Efectos “mágicos”: un evento desencadena acciones que el productor no espera.	Catálogo AsyncAPI; nombres consistentes <Agregado><Verbo en pasado> ; estados de reserva visibles en la UI.
Ley de Demeter	++	Los servicios solo “hablan” con el broker y su propio almacén; no navegan objetos de otros servicios.	Consultas encadenadas a otros servicios tras recibir una notificación.	<i>Event-carried state transfer</i> : el evento trae los datos necesarios.
Composición sobre herencia	++	Los flujos se componen encadenando procesadores y <i>middlewares</i> .	Jerarquías profundas de clases base de eventos.	Pipeline de consumo por composición (deserializar → validar → deduplicar → manejar); widgets de Flutter por composición.
STUPID (anti-principios a evitar)				
S – Singleton	±	El estilo no lo necesita.	Productores Kafka como <i>singletons</i> globales ocultos.	El productor sí debe ser único por proceso (es <i>thread-safe</i>), pero registrado por DI (AddSingleton), no como estático.
T – Tight coupling	++	Desacople espacial y temporal por diseño.	Acoplamiento por esquema de evento.	Contratos versionados y compatibilidad hacia atrás.
U – Untestability	–	Procesadores puros fáciles de probar unitariamente.	Pruebas de integración asíncronas.	Puertos/adaptadores + Testcontainers.
P – Premature optimization	±	—	Afinar particiones, compresión y <i>batching</i> antes de medir.	Configuración por defecto razonable; medir con Kafbat UI antes de optimizar.
I – Indescriptive naming	+	El lenguaje de eventos obliga a nombrar hechos del negocio.	Tópicos como topic1 , events .	Convención <dominio>.<versión> y <Agregado><Hecho> .

Principio	Cumpl.	Favorece	Riesgo	Stack
D – Duplication	±	Contratos compartidos.	Lógica de deduplicación, reintento y trazas repetida en cada servicio.	Librería BuildingBlocks con el <i>pipeline</i> común de consumo.

4.3 Matriz de tácticas vs. estilo y stack, y registros de decisión (ADR)

Las tácticas arquitectónicas son decisiones de diseño que influyen en la respuesta de un atributo de calidad [2]. La matriz 4.3 vincula cada táctica con el mecanismo del estilo y la implementación en el stack; la última columna indica el ADR que la justifica.

Tabla 4.3: Matriz de tácticas vs. estilo y stack

Atributo	Táctica (Bass et al.)	Mecanismo en EDA	Implementación en el stack	ADR
Disponibilidad	Redundancia activa (replicación)	Log replicado en varios brokers.	Kafka <code>RF=3, acks=all, min.insync.replicas=2</code> ; réplicas de índices en Elasticsearch (producción).	002
	Detección: <i>heartbeat</i> / <i>health check</i>	Liveness y readiness de procesadores.	<code>MapHealthChecks</code> en ASP.NET Core; <code>healthcheck</code> en Docker Compose.	008
	Reintento (<i>retry</i>)	Reprocesar eventos fallidos.	Polly con <i>backoff</i> exponencial; tópico de reintento.	004
	Remover de servicio / aislar	Aislar mensajes que siempre fallan.	Tópicos <code>*.dlq</code> con cabeceras de error.	004
	Transacciones / estados	Sagas y compensaciones en vez de 2PC.	Estados de reserva + eventos compensatorios <code>AsientosLiberados</code> .	007
Desempeño	Introducir concurrencia	Consumidores paralelos por partición.	Grupos de consumidores .NET; particiones por tópico.	002
	Mantener múltiples copias de datos	Vistas materializadas (CQRS).	Índice de búsqueda de eventos en Elasticsearch alimentado por eventos.	003
	Programar recursos / cola	<i>Load leveling</i> en el broker.	Kafka absorbe picos de la apertura de venta.	002
	Reducir sobrecarga	Procesamiento asíncrono y por lotes.	<code>async/await</code> ; <i>batching</i> y compresión del productor.	005
Modificabilidad	Usar un intermediario	El broker desacopla productores y consumidores.	Kafka como único canal entre servicios.	001
	Encapsular / abstraer servicios comunes	Contratos de eventos.	Proyecto <code>Contracts</code> ; CloudEvents.	004
	Diferir el enlace (<i>defer binding</i>)	Suscripción en tiempo de despliegue.	Nombres de tópicos y grupos por configuración.	008
	Restringir dependencias	Sin llamadas directas entre servicios.	Hexagonal interno; solo el BFF expone HTTP.	005

Atributo	Táctica	EDA	Stack	ADR
Seguridad	Autenticar y autorizar actores	Solo la API es pública.	JWT en ASP.NET Core; SASL/ACL en Kafka (producción).	006
	Cifrar datos	Canales cifrados.	HTTPS (incluido el <i>stream</i> SSE); TLS en Kafka y Elasticsearch (producción).	006
	Limitar exposición / acceso	Bus interno no expuesto.	Red interna de Docker; <i>rate limiting</i> en la API.	006
Integrabilidad	Adherirse a estándares	Formato común de eventos.	CloudEvents + JSON Schema + AsyncAPI.	004
	Orquestar vs. coreografiar	Coreografía por eventos.	Saga coreografiada (sin orquestador central).	001, 007
Testabilidad	Separar interfaz de implementación	Puertos y adaptadores.	Interfaces de repositorio y publicador; <i>fakes</i> .	005
	Entornos especializados	Infraestructura real efímera.	Testcontainers para Kafka y Elasticsearch.	008
Usabilidad	Retroalimentación del sistema	Notificar el avance de la saga.	SSE → Flutter (BLoC).	006
	Mantener modelo de la tarea	Estados explícitos.	Reserva: PENDIENTE, RETENIDA, CONFIRMADA, RECHAZADA, EXPIRADA, REEMBOLSADA.	007

4.3.1 Registros de decisiones arquitectónicas (ADR)

Los ADR siguen el formato propuesto por Nygard [9]: contexto, decisión, alternativas, consecuencias y estado. Todos se encuentran en estado *Aceptado* para esta entrega y podrán revisarse durante el diseño.

ADR-001 – Adoptar EDA con topología Broker (coreografía)

Contexto. La reserva de tickets involucra inventario, pagos y notificaciones que evolucionan de forma independiente; la apertura de ventas genera picos. El estilo asignado es EDA.

Decisión. Implementar la topología Broker: cada servicio reacciona a eventos y publica nuevos hechos; no existe orquestador central.

Alternativas. (a) Topología Mediator con un orquestador de sagas: más control y visibilidad, pero introduce un punto central y acoplamiento; (b) microservicios síncronos REST: más simples de razonar, pero acoplados en disponibilidad y latencia.

Tácticas. Usar un intermediario; coreografía; restringir dependencias.

Consecuencias. + Extensibilidad y escalabilidad; nuevos consumidores sin cambios. – Flujo implícito: se mitiga con diagrama dinámico C4, catálogo AsyncAPI y trazas.

Estado. Aceptado.

ADR-002 – Apache Kafka 4.x (KRaft) como bus de eventos

Contexto. Se requiere un canal durable, con orden por agregado, pub/sub para varios consumidores y capacidad de reprocesar.

Decisión. Usar Kafka 4.x en modo KRaft. Un tópico por agregado y versión (`reservas.v1`, `inventario.v1`, `pagos.v1`); la clave de partición es el ID del agregado que debe procesar los eventos en orden (la localidad para los eventos que modifican inventario, la reserva para los demás), productor idempotente (`enable.idempotence=true`, `acks=all`), retención de 7 días.

Alternativas. RabbitMQ (mejor enrutamiento, sin *replay* nativo), NATS JetStream (ligero, menor ecosistema .NET), Azure Service Bus (gestionado, dependencia de proveedor).

Tácticas. Redundancia, concurrencia, *load leveling*.

Consecuencias. + Durabilidad, *replay*, escalado por particiones. – Entrega al menos una vez y operación más compleja; se mitiga con ADR-004 y Docker Compose.

Estado. Aceptado.

ADR-003 – Elasticsearch como almacén por servicio con agregados por documento

Contexto. El stack asigna Elasticsearch como persistencia. No existen transacciones multi-documento; la atomicidad es por documento y la búsqueda es casi en tiempo real.

Decisión. (1) *Database per service*: índices `catalogo-eventos`, `inventario-localidades`, `reservas`, `pagos`, sin índices compartidos. (2) Cada agregado cabe en un documento (la reserva con sus datos y su *outbox*; la localidad con sus contadores, retenciones y *outbox*). (3) Un solo escritor por localidad, gracias a la clave de partición `localidadId` (ADR-002), y concurrencia optimista (`if_seq_no` / `if_primary_term`) con reintento ante 409 como salvaguarda durante rebalanceos. (4) Lecturas de estado crítico por `_id` (tiempo real), búsquedas sobre índices de lectura. (5) `refresh=wait_for` solo donde la UX lo requiera.

Alternativas. PostgreSQL como fuente de verdad + Elasticsearch como réplica de lectura (más común en la industria, pero fuera del stack asignado); *scripts* Painless para decremento atómico (válido, pero menos portable y más difícil de probar).

Tácticas. Múltiples copias de datos (CQRS), control de concurrencia.

Consecuencias. + Búsqueda rica del catálogo y esquema flexible. – Modelado cuidadoso de agregados; imposibilidad de invariantes entre documentos (se resuelven con la saga).

Estado. Aceptado.

ADR-004 – Outbox embebido, consumidor idempotente, DLQ y contratos CloudEvents

Contexto. Guardar en Elasticsearch y publicar en Kafka son dos operaciones independientes (*dual write*); Kafka entrega al menos una vez.

Decisión. (1) **Outbox embebido:** el documento del agregado incluye un arreglo **outbox** con los eventos pendientes, escrito atómicamente junto con el estado; un publicador de relevo (*BackgroundService*) los envía a Kafka y los marca como publicados. (2) **Consumidor idempotente:** cada evento procesado se registra con `op_type=create` y `_id = eventId`; un 409 indica duplicado y se descarta. (3) **Reintentos y DLQ:** 3 reintentos con *backoff*; luego se envía a `<tópico>.dlq`. (4) Sobre **CloudEvents** (`id`, `type`, `source`, `time`, `subject`) y JSON Schema versionado.

Alternativas. Transacciones de Kafka (no cubren Elasticsearch); CDC (no aplica a Elasticsearch de forma estándar); publicar primero y guardar después (riesgo de eventos “fantasma”).

Consecuencias. + Ningún evento se pierde ni se aplica dos veces. – Latencia adicional del relevo (milisegundos) y lógica de infraestructura extra, centralizada en **BuildingBlocks**.

Estado. Aceptado.

ADR-005 – .NET 10 LTS con ASP.NET Core Minimal APIs y Worker Services

Contexto. El backend debe exponer HTTP al cliente y ejecutar consumidores de larga duración.

Decisión. .NET 10 LTS (soporte hasta noviembre de 2028). APIs con Minimal APIs; API Gateway con YARP sobre ASP.NET Core (autenticación JWT y *rate limiting* con el mismo *middleware* del resto de servicios); cada servicio de dominio como *Worker Service* con *BackgroundService*; cliente **Confluent.Kafka**; cliente **Elastic.Clients.Elasticsearch**; arquitectura hexagonal interna (dominio sin dependencias de infraestructura).

Alternativas. .NET 11 (STS, sale en noviembre de 2026: soporte más corto); MassTransit (abstracción útil, pero desde la versión 9 es comercial y oculta conceptos que el taller busca evidenciar); Kong, Traefik u Ocelot como gateway (válidos, pero agregan otra tecnología o configuración declarativa para lo que YARP resuelve en C#).

Consecuencias. + Rendimiento, DI, resiliencia y observabilidad integradas. – Sin Kafka Streams; el procesamiento con estado se hace en el servicio.

Estado. Aceptado.

ADR-006 – Flutter con REST para comandos y Server-Sent Events para notificaciones

Contexto. La app no debe conectarse a Kafka ni a Elasticsearch; el resultado de la reserva llega de forma asíncrona y la comunicación en vivo solo va del servidor a la app.

Decisión. Flutter envía comandos por REST (`POST /reservas` responde `202 Accepted` con `reservaId`) y abre un *stream* de Server-Sent Events (`GET /reservas/{id}/eventos`, nativo en .NET 10 con `TypedResults.ServerSentEvents`) por el que recibe cada cambio de estado. Al reconectarse, la app consulta `GET /reservas/{id}` para resincronizarse, sin necesidad de un búfer de reproducción. Gestión de estado con BLoC. Autenticación con JWT y *rate limiting* en el gateway.

Alternativas. SignalR (bidireccional, con grupos y reconexión automática, pero su cliente Dart es comunitario —sin editor verificado— y la necesidad es solo servidor → cliente); WebSocket puro (bidireccionalidad innecesaria); *polling* periódico (más simple, más tráfico y peor UX); GraphQL *subscriptions* (más complejo).

Consecuencias. + UX en tiempo real sobre HTTP estándar, sin librería propietaria, y paso transparente por el gateway. – El `EventSource` del navegador no permite la cabecera `Authorization`, por lo que la app usa un cliente HTTP con lectura en *streaming*; con varias instancias de Reservas, cada instancia debe recibir las notificaciones de las reservas conectadas a ella.

Estado. Aceptado.

ADR-007 – Saga coreografiada con compensaciones y expiración de reservas

Contexto. Reservar asientos y cobrar debe comportarse como “todo o nada” sin transacción distribuida.

Decisión. Flujo: `ReservaSolicitada` → Inventario emite `AsientosRetenidos` o `AsientosNoDisponibles` → Pagos emite `PagoAprobado` o `PagoRechazado` → Reservas emite `ReservaConfirmada` o `ReservaRechazada` → Inventario vende los cupos o los libera (`AsientosVendidos` o `AsientosLiberados`). Toda retención expira a los 10 minutos y genera `ReservaExpirada`. Reservas es la única autoridad sobre el ciclo de vida de la reserva: Inventario reacciona a sus decisiones y no a los eventos de pago, lo que evita vender cupos de una reserva expirada. Los estados solo avanzan, de modo que los eventos que llegan desordenados o repetidos se descartan sin romper la saga.

Alternativas. Orquestación con un *saga orchestrator* (topología Mediator); 2PC/TCC (no soportado por Kafka + Elasticsearch y contrario al estilo).

Consecuencias. + Consistencia eventual controlada y visible para el usuario. – Más eventos y estados que diseñar y probar.

Estado. Aceptado.

ADR-008 – Despliegue con Docker Compose y observabilidad mínima

Contexto. El taller exige contenedores; la demostración debe ser reproducible en cualquier equipo.

Decisión. Un `docker-compose.yml` con Kafka (KRaft, un nodo), Kafbat UI, Elasticsearch, Kibana, el gateway, los servicios y los *workers*; *health checks*, dependencias por estado saludable y configuración por variables de entorno. Trazas, métricas y logs estructurados con OpenTelemetry, exportados por OTLP al Aspire Dashboard.

Alternativas. Kubernetes (excesivo para el alcance: YAGNI); Podman (válido y compatible; se documentará como alternativa); OTel Collector + Jaeger (solo trazas y dos contenedores); Grafana LGTM (completo pero pesado para un equipo de estudiante).

Consecuencias. + Reproducibilidad (evita *Works on My Machine*) y demostración clara.
– Un solo nodo de Kafka y Elasticsearch no demuestra alta disponibilidad real.
Estado. Aceptado.

4.4 Matriz de mercado laboral vs. estilo y stack

4.4.1 Fuentes y método

Se consultaron la encuesta de Stack Overflow 2025 (49 000 respuestas; la edición 2026 se abrió en junio de 2026) [32, 33, 35], el índice TIOBE de septiembre de 2026 [39], el índice PYPL [36], el ranking de IEEE Spectrum 2025 [38], portales de empleo colombianos (Computrabajo, Indeed, elempleo) y la plataforma de contratación Talently [46, 47, 48]. Los salarios se presentan **mensuales en COP** para Colombia y **anuales en USD** como referencia internacional; la conversión usa la TRM de \$ 3.306,86 [49]. Cuando no existe dato directo para una tecnología (Kafka, Elasticsearch), se indica explícitamente y se usa el rol más cercano.

★ Idea clave

Las cifras de portales de empleo son *medias salariales publicadas o reportadas* y varían según ciudad, seniority, inglés y tipo de contrato. Las de Talently reflejan principalmente contratación remota en dólares, por lo que son sensiblemente más altas que el mercado local. Además, IEEE Spectrum advierte que los índices basados en actividad pública pierden precisión: en 2025 las preguntas nuevas en Stack Exchange cayeron al 22 % del nivel de 2024, porque los desarrolladores consultan asistentes de IA en privado [38]. Por eso se combinan varias fuentes y se leen como tendencias, no como cifras exactas.

4.4.2 Popularidad e indicadores de demanda

Tabla 4.4: Indicadores de popularidad del estilo y del stack (septiembre de 2026)

Tecnología	Indicadores	Tendencia
C# / .NET	TIOBE: 5.º lugar con 4,22 % (sept. 2026); “lenguaje del año 2025”. Stack Overflow 2025: C# 27,8 % (29,9 % profesionales); ASP.NET Core 19,7 %, admirado por 61,3 % [39, 40, 32].	Estable-alta; LTS hasta 2028. En búsquedas de tutoriales (PYPL) C# cae cerca de 3 puntos en el año [37], señal de un lenguaje maduro más que de abandono.
Flutter / Dart	Dart 5,9 % (2025); Flutter 9,4 % en “otros frameworks” (2024), por encima de React Native [32, 34]. Lanzamientos trimestrales constantes. PYPL: Dart ≈0,55 % de las búsquedas de tutoriales (abril de 2026) [37].	Creciente en móvil y en LATAM.
Elasticsearch	16,7 % de uso (17,8 % profesionales); 8.ª base de datos más usada en 2025; admirada por 39,4 % [32].	Estable; impulso por búsqueda vectorial e IA.
Apache Kafka / EDA	Más del 80 % del Fortune 100 usa Kafka [12]; Confluent con 6 500 clientes empresariales, adquirida por IBM por unos USD 11 000 millones (2026) [18].	Alta; el streaming se posiciona como infraestructura para IA.
Docker	71,1 % de uso; mayor aumento de un año en la encuesta 2025 (+17 puntos) [32].	Universal (requisito base).

4.4.3 Salarios

Tabla 4.5: Matriz de mercado laboral vs. estilo y stack (salarios y proyección)

Perfil / tecnología	Colombia – mercado local (COP/mes)	Colombia – contratación en USD (USD/mes ≈ COP/mes)	Referencia global (USD/año, mediana SO 2025)	Proyección
Desarrollador .NET (backend)	Media \$ 4.276.026 (Computrabajo, sept. 2026; rango reportado hasta \$ 12.000.000); \$ 3.348.495 (Indeed) [41, 43]	.NET Core: <i>junior</i> 4 400–5 332; <i>mid</i> 5 500–6 666; <i>senior</i> 7 150–8 665 (≈ \$ 14,6–\$ 28,7 millones) [48]	Back-end: 79 742 (mundo); 175 000 (EE. UU.); 87 011 (Alemania) [33]	Alta. .NET 10 LTS hasta 2028; C# lenguaje del año 2025; fuerte en banca y gobierno.
Desarrollador Flutter (móvil)	Vacantes de \$ 5.000.000–\$ 5.500.000 (Bogotá, sector financiero); móvil <i>junior</i> \$ 3.500.000–\$ 4.000.000 [44, 45]	<i>junior</i> 1 360–1 520; <i>mid</i> 1 700–1 900; <i>senior</i> 2 210–2 470 (≈ \$ 4,5–\$ 8,2 millones) [47]	Móvil: 69 609 (mundo); 170 000 (EE. UU.) [33]	Media-alta. Crecimiento en fintech y apps multiplataforma.
Ingeniero de datos / streaming (Kafka)	Sin dato específico público. Se estima en el rango <i>senior</i> de backend/datos (\$ 6–\$ 12 millones) por su especialización (<i>estimación del grupo</i>).	Programador general Colombia: <i>mid</i> 2 599–3 569; <i>senior</i> 5 000–6 000 [46]	Data engineer: 81 210 (mundo); 150 000 (EE. UU.) [33]	Alta. IBM–Confluent y la IA en tiempo real aumentan la demanda.
Especialista en búsqueda / observabilidad (Elasticsearch)	Sin dato específico público; habitualmente incluido en perfiles backend, DevOps o SRE (<i>estimación del grupo</i> : \$ 5–\$ 10 millones).	Ver fila anterior (perfil general).	DevOps: 87 011; DBA: 85 168 (mundo) [33]	Media-alta. Búsqueda vectorial y observabilidad.
Arquitecto de software (EDA)	Perfil <i>senior</i> /líder: por encima de la media de “Desarrollador sénior” (\$ 4,6–\$ 5,0 millones de media, con máximos cercanos a \$ 18 millones) [42]	Perfil <i>senior</i> : 5 000–6 000 [46]	Arquitecto: 102 000 (mundo); 180 000 (EE. UU.); 109 054 (Alemania) [33]	Muy alta. EDA y streaming son competencias diferenciales.
Desarrollador de software (referencia)	Media \$ 3.819.039 (Computrabajo); \$ 2.994.556 (Indeed) [42, 43]	<i>junior</i> 1 155–1 533 [46]	Full-stack: 72 509 (mundo) [33]	Estable.

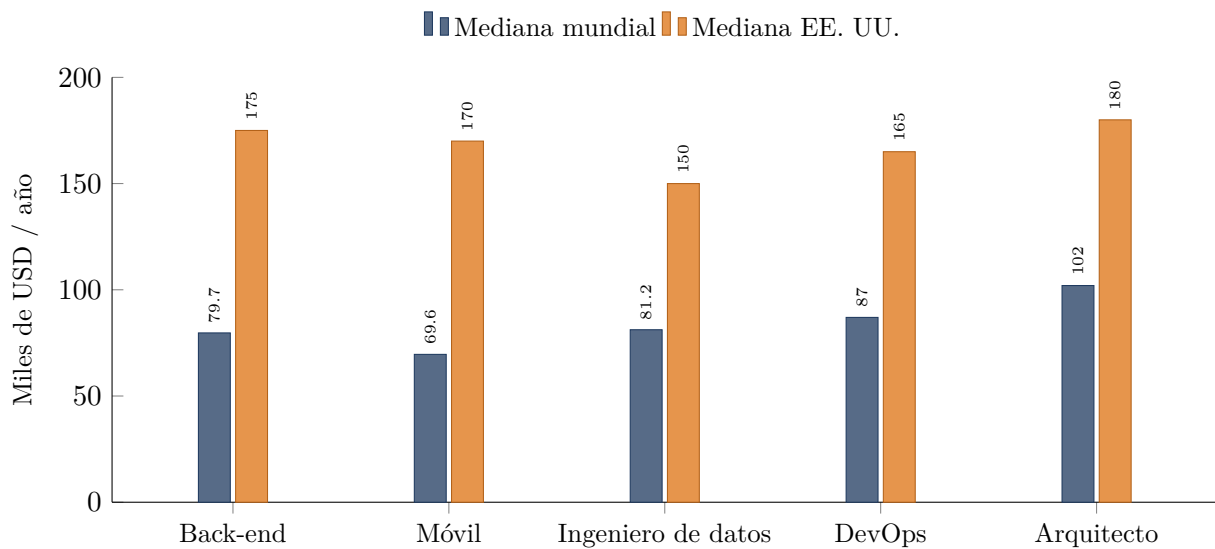


Figura 4.2: Salario anual mediano por rol relacionado con el stack (Stack Overflow Developer Survey 2025) [33]

4.4.4 Proyección en el mercado laboral

1. **.NET/C#:** con .NET 8 y 9 terminando soporte el 10 de noviembre de 2026, muchas organizaciones migrarán a .NET 10 LTS, lo que sostiene la demanda de perfiles con experiencia en versiones modernas [21]. TIOBE muestra a C# estable en el quinto lugar, aunque con una caída de 2,16 puntos interanuales tras su pico de enero de 2026 [39].
2. **Kafka y EDA:** la adquisición de Confluent por IBM y la narrativa de “datos en vivo para agentes de IA” posicionan el streaming de eventos como competencia estratégica. Al mismo tiempo, analistas señalan incertidumbre sobre el rumbo de Confluent dentro de IBM, lo que podría favorecer alternativas compatibles (Redpanda, WarpStream) y aumentar el valor de dominar el protocolo abierto de Kafka más que un producto específico [19].
3. **Elasticsearch:** su vuelta a una licencia de código abierto (AGPL) y la búsqueda vectorial mantienen su relevancia frente a OpenSearch y bases vectoriales especializadas.
4. **Flutter:** sin reemplazo claro en el nicho multiplataforma con UI propia; su demanda local se concentra en fintech y banca digital.
5. **Combinación del stack:** el perfil que combina backend .NET, EDA con Kafka y búsqueda con Elasticsearch es escaso y por eso mejor remunerado; la habilidad transferible más valiosa es el *razonamiento en sistemas distribuidos asíncronos* (idempotencia, sagas, observabilidad), independiente del producto.

Diseño: ejemplo práctico y funcional

Este capítulo modela la solución del caso práctico —una plataforma de reservas y venta de tickets para eventos, que en adelante se denomina **TaquillaEDA**— a partir de una **arquitectura real existente: la de Ticketmaster**, la mayor ticketera del mundo y uno de los usuarios más conocidos de Apache Kafka. Primero se reconstruye la arquitectura de referencia con base en fuentes públicas; después se explica qué se toma de ella y qué no, y finalmente se presentan los diagramas exigidos por el taller (HLD, contexto, contenedores, dinámico y despliegue) y los opcionales (*system landscape*, componentes, código y modelo de datos).

★ Idea clave

El objetivo no es copiar a Ticketmaster —su escala (600 millones de tickets al año) y su historia de casi 50 años no son replicables en un taller—, sino *extraer las decisiones que resolvieron problemas reales* del dominio (picos de demanda, retención temporal de asientos, integración de cientos de sistemas mediante eventos) y trasladarlas a un diseño de tamaño académico, coherente con EDA en topología Broker y con el stack asignado (Flutter, .NET, Elasticsearch y Kafka).

5.1 Arquitectura de referencia: Ticketmaster

5.1.1 Por qué Ticketmaster

Ticketmaster es la referencia natural para este dominio por tres razones: (1) opera exactamente el negocio del caso (inventario limitado, ventas con picos extremos y pagos); (2) su transformación tecnológica está documentada en charlas y casos de estudio públicos en los que Kafka aparece como el centro de su plataforma de datos; y (3) sus incidentes públicos, en particular la preventa del *Eras Tour* de 2022, dejaron métricas concretas sobre lo que ocurre cuando la demanda supera la capacidad. La Tabla 5.1 resume los hechos que se usan como base del diseño.

Tabla 5.1: Hechos públicos sobre la arquitectura de Ticketmaster usados como referencia

Aspecto	Hecho documentado	Fuente
Escala actual	Unos 600 millones de tickets al año ($\approx$ 2 millones diarios), 2 000 millones de visitas web anuales, operación en más de 35 países; fundada en 1976.	[TM1]
Herencia tecnológica	“Cada versión de cada pieza de software creada en los últimos 40 años corre en algún lugar de la compañía”, incluido un sistema VAX de los años setenta que se ejecuta emulado; 21 sistemas de ticketing distintos, más de 250 productos y más de 65 equipos de producto (2016).	[TM2]

Aspecto	Hecho documentado	Fuente
Microservicios	Migró de aplicaciones monolíticas a microservicios hacia 2015; en 2019 operaba cerca de 300 aplicaciones en más de 4 500 máquinas virtuales, entre centros de datos propios y AWS.	[TM3]
Kafka como centro	Kafka (con soporte de Confluent) funciona como <i>hub</i> central de datos: los productores publican en tópicos y los consumidores leen del clúster central. Uso de <i>Schema Registry</i> para evolucionar esquemas con compatibilidad hacia atrás.	[TM3, TM4]
Stream de inventario	“A los fans les importan los asientos disponibles, los costos y los paquetes; a los recintos, la demografía y la conversión. Ponemos todo esto en un <i>stream</i> de inventario para diversos usos” (Chris Smith, VP de Ingeniería y Ciencia de Datos).	[TM5]
Consumidores en tiempo real	Detección de abuso y fraude en tiempo real, ticketing basado en identidad, monitoreo de asistencia, deslistado automático de reventa al ingresar al recinto y gestión de solicitudes de privacidad, todos alimentados por Kafka.	[TM4]
Procesamiento de streams	Uso de Kafka Streams y KSQL para reducir la fricción en productos de aprendizaje automático.	[TM6]
Picos de demanda	“Cada vez que ponemos tickets a la venta es, en esencia, un ataque de denegación de servicio distribuido que lanzamos a propósito contra nosotros mismos.”	[TM4]
Sala de espera virtual	<i>Smart Queue</i> : la sala de espera abre unos 30 minutos antes; el orden en la fila no depende de la hora de llegada; cuando llega el turno, el comprador tiene 10 minutos para empezar a comprar; el ingreso a la venta se dosifica para minimizar errores de pago.	[TM7, TM8]
Incidente <i>Eras Tour</i> (nov. 2022)	3,5 millones de registros en Verified Fan, 1,5 millones de códigos enviados, 3 500 millones de solicitudes al sistema (4 veces el pico anterior), cerca del 15 % de las interacciones con problemas; se ralentizaron las filas para estabilizar el sistema; aun así se vendieron 2 millones de tickets en un día.	[TM9]
Observabilidad	Telemetría centralizada en Kafka con marcas de tiempo para correlación; trazas distribuidas (inicialmente Jaeger, luego LightStep).	[TM3]
Infraestructura	Híbrida: centros de datos propios más AWS (EKS, Local Zones y <i>cloud bursting</i> en los picos de venta); Kubernetes con <i>health checks</i> de pods para recuperación automática.	[TM1, TM2]

5.1.2 Vista reconstruida de la arquitectura de referencia

La Figura 5.1 integra los hechos anteriores en una sola vista. **No es un diagrama oficial de Ticketmaster**: es una reconstrucción del grupo a partir de las fuentes citadas, útil para identificar los patrones que se trasladan al diseño.

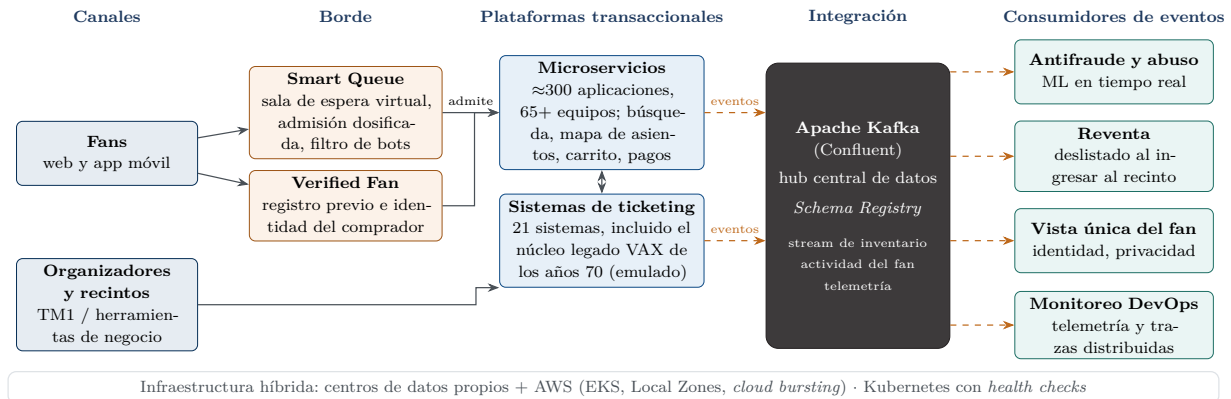


Figura 5.1: Vista reconstruida de la arquitectura de Ticketmaster (elaboración propia a partir de [TM1]–[TM9]; no es un diagrama oficial)

5.1.3 Lecciones de la referencia que se trasladan al diseño

La Tabla 5.2 traduce cada práctica de Ticketmaster a una decisión concreta de TaquillaEDA y la enlaza con el ADR correspondiente del Capítulo 4.

Tabla 5.2: De la arquitectura de referencia al diseño de TaquillaEDA

Práctica en Ticketmaster	Problema que resuelve	Adaptación en TaquillaEDA	ADR
Kafka como <i>hub</i> central entre cientos de sistemas	Integración punto a punto n^2 entre sistemas heterogéneos	Kafka es el único canal entre servicios; ningún servicio llama a otro por HTTP en el flujo principal.	001, 002
<i>Stream</i> de inventario compartido	Múltiples interesados necesitan la disponibilidad sin consultar al sistema dueño	El tópico <code>inventario.v1</code> transporta el estado de la localidad (<i>event-carried state transfer</i>); Catálogo y Reservas lo consumen.	003
<i>Schema Registry</i> con compatibilidad hacia atrás	Deuda técnica por cambios de esquema no gobernados	Contratos CloudEvents + JSON Schema versionados en el repositorio; tópicos con sufijo de versión; solo cambios aditivos en v1.	004
<i>Smart Queue</i> con admisión dosificada	Picos tipo DDoS “autoinfligidos” en la apertura de ventas	Sala de espera simplificada en el API Gateway: <i>rate limiting</i> por evento con respuesta 429 + <i>Retry-After</i> ; la app muestra la fila y reintenta.	006

Práctica	Problema	Adaptación	ADR
Turno de 10 minutos para comprar	Asientos bloqueados indefinidamente por compradores que abandonan	Retención de cupos con expiración de 10 minutos y compensación automática (<i>ReservaExpirada</i> → <i>AsientosLiberados</i>).	007
Consumidores de fraude, reventa y analítica agregados sobre el mismo <i>stream</i>	Evolución del negocio sin tocar los sistemas núcleo	El diseño deja explícitos consumidores futuros (antifraude, analítica) que se conectarían a los tópicos existentes sin modificar ningún servicio.	001
Trazas distribuidas y telemetría centralizada	Diagnosticar fallos en cientos de servicios	OpenTelemetry con propagación de contexto en las cabeceras de Kafka; en el taller, Aspire Dashboard para ver trazas, métricas y logs juntos; en producción, un backend persistente como Jaeger (la herramienta con la que empezó Ticketmaster).	008
Kubernetes con <i>health checks</i>	Recuperación automática en picos	Docker Compose con <i>health checks</i> y reinicio automático; se documenta el mapeo a Kubernetes para producción.	008
Lección del <i>Eras Tour</i> : ralentizar la fila para estabilizar	El sistema colapsa si admite más de lo que puede procesar	La admisión se limita en el borde y Kafka amortigua lo que ya entró; la capacidad se mide con el <i>lag</i> de los consumidores.	002, 006

Lo que deliberadamente *no* se replica. La infraestructura híbrida con *Local Zones*, los 21 sistemas de ticketing, el núcleo legado VAX, el procesamiento con Kafka Streams/KSQL y los modelos de aprendizaje automático exceden el alcance del taller y contradicen el principio YAGNI (Sección 4.2). Tampoco se implementa un mapa de asientos numerados: el inventario se modela por *localidad con cupos* (p. ej. “General”, “Preferencial”, “VIP”), una simplificación que conserva el problema esencial de concurrencia sobre un recurso limitado.

Supuestos donde la información pública no alcanza. Las fuentes no detallan cómo Ticketmaster implementa internamente la retención de asientos. Los diseños de referencia del dominio suelen usar un candado distribuido con tiempo de vida (p. ej. Redis con TTL) [TM10]. Como Redis no forma parte del stack asignado, TaquillaEDA implementa la retención dentro del documento de la localidad en Elasticsearch, con **control de concurrencia optimista** y un campo de expiración revisado por un proceso periódico (ADR-003 y ADR-007).

5.1.4 Fuentes específicas de la arquitectura de referencia

[TM1] M. Fuller (Ticketmaster), S. Marupaka y S. Lu (AWS), “Ticketmaster: Enhancing live event experiences for fans with AWS (SPF206)”, AWS re:Invent 2025. Resumen: https://dev.to/kazuya_dev/aws-reinvent-2025-ticketmaster-enhancing-live-event-experiences-for-fan

s-with-aws-spf206-14

- [TM2] Linux.com, “Ticketmaster Chooses Kubernetes to Stay Ahead of Competition”, 21 dic. 2016. <https://www.linux.com/news/ticketmaster-chooses-kubernetes-stay-ahead-competition/>
- [TM3] TechTarget, “Ticketmaster unifies DevOps monitoring with Confluent Kafka”, 11 oct. 2019. <https://www.techtarget.com/searchitoperations/news/252472217/Ticketmaster-unifies-DevOps-monitoring-with-Confluent-Kafka>
- [TM4] CIO, “Ticketmaster tackles tech debt with streaming data platform”, 24 oct. 2019. <https://www.cio.com/article/217748/ticketmaster-tackles-tech-debt-with-streaming-data-platform.html>
- [TM5] Confluent, “Ticketmaster: Reduce Development Friction and Boost Machine Learning” (caso de cliente). <https://www.confluent.io/customers/ticketmaster/>
- [TM6] C. Smith (Ticketmaster) y D. Traphagen (Confluent), “An Introduction to KSQL & Kafka Streams Processing with Ticketmaster”, Confluent Online Talks. <https://www.confluent.io/online-talks/introduction-to-ksql-and-kafka-streams-processing-with-ticketmaster/>
- [TM7] Ticketmaster Business, “Smart Queue”. <https://business.ticketmaster.com/smart-queue/>
- [TM8] Ticketmaster Australia, “Ticketmaster’s Guide to Smart Queue”. <https://discover.ticketmaster.com.au/guide-to/ticketmasters-guide-to-smart-queue-15286>
- [TM9] Ticketmaster Business, “Taylor Swift | The Eras Tour Onsale Explained”, 19 nov. 2022. <https://business.ticketmaster.com/press-release/taylor-swift-the-eras-tour-onsale-explained/>
- [TM10] Hello Interview, “Design a Ticket Booking Site Like Ticketmaster” (diseño de referencia del dominio, no oficial). <https://www.hellointerview.com/learn/system-design/problem-breakdowns/ticketmaster>

5.2 Alcance del ejemplo práctico

El alcance se define para que el ejemplo sea pequeño pero completo: un único caso de uso de extremo a extremo que atraviesa todas las tecnologías del stack y que obliga a resolver los problemas característicos del estilo (consistencia eventual, duplicados, compensaciones y picos de demanda).

5.2.1 Requisitos funcionales

Tabla 5.3: Requisitos funcionales de TaquillaEDA

ID	Requisito	Servicio responsable
RF-01	El organizador publica un evento con sus localidades (nombre, precio y capacidad).	Catálogo → Inventario
RF-02	El comprador busca eventos por texto libre, ciudad, categoría y rango de fechas, y ve la disponibilidad por localidad.	Catálogo
RF-03	El comprador solicita una reserva de 1 a 6 cupos de una localidad; el sistema responde de inmediato con un identificador de reserva.	Reservas
RF-04	El sistema retiene los cupos por 10 minutos si hay disponibilidad, o rechaza la reserva si no la hay.	Inventario
RF-05	El sistema cobra el valor de la reserva a través de una pasarela de pagos (simulada).	Pagos
RF-06	El comprador recibe en la app, en tiempo real, cada cambio de estado de su reserva, y un correo de confirmación o rechazo.	Reservas, Notificaciones
RF-07	Si la reserva no se paga en 10 minutos, expira y los cupos vuelven a estar disponibles.	Reservas → Inventario
RF-08	En horas de alta demanda, el comprador entra a una sala de espera antes de poder reservar.	API Gateway

5.2.2 Escenarios de atributos de calidad

Los escenarios siguen la plantilla de seis partes de Bass *et al.* [2] y convierten los atributos priorizados en la matriz de la Sección 4.1 en criterios verificables durante la demostración.

Tabla 5.4: Escenarios de calidad del ejemplo práctico

Atributo	Estímulo y entorno	Respuesta esperada	Medida
Desempeño	200 compradores solicitan reservas en el mismo minuto (apertura de venta), operación normal.	La API registra la solicitud y responde sin esperar a inventario ni pagos.	202 Accepted en menos de 300 ms (p95).
Consistencia (no sobreventa)	100 solicitudes concurrentes compiten por una localidad con 50 cupos.	Solo se retienen los cupos existentes; el resto recibe AsientosNoDisponibles .	0 cupos vendidos por encima de la capacidad.
Disponibilidad y recuperabilidad	El contenedor de Pagos se detiene durante 2 minutos en plena venta.	Los eventos se acumulan en Kafka; al reiniciar, Pagos procesa los pendientes.	0 eventos perdidos; reservas completadas tras el reinicio.

Atributo	Estímulo y entorno	Respuesta esperada	Medida
Tolerancia a fallos	La pasarela responde con errores o tarda más de 3 s.	Reintentos con <i>backoff</i> y <i>circuit breaker</i> ; si persiste, pago rechazado y compensación.	Ninguna reserva queda “colgada” más de 10 minutos.
Modificabilidad	Se necesita un nuevo consumidor (p. ej. analítica de ventas).	Se agrega como servicio nuevo suscrito a <code>reservas.v1</code> .	0 líneas modificadas en los servicios existentes.
Usabilidad	El comprador espera el resultado de su reserva.	La app muestra cada estado (retenida, pagando, confirmada) sin recargar.	Notificación en la app menos de 2 s después del evento.
Observabilidad	Un comprador reporta que su reserva “no avanzó”.	Se localiza el flujo completo con un solo identificador.	Traza de extremo a extremo y logs de cada servicio visibles en Aspire Dashboard con el <code>traceId</code> .

5.2.3 Entidades de negocio

El taller exige un mínimo de tres entidades relacionadas; el diseño usa cinco, cada una con un único servicio dueño (*database per service*).

Tabla 5.5: Entidades de negocio, relaciones y servicio dueño

Entidad	Atributos principales	Relaciones	Servicio / índice
Evento	id, nombre, artista, categoría, ciudad, recinto, fecha, estado	1 Evento – N Localidades	Catálogo / <code>catalogo-eventos</code>
Localidad	id, eventoId, nombre, precio, capacidad, disponibles, retenidos, vendidos, retenciones	N Localidades – 1 Evento; 1 Localidad – N Reservas	Inventario / <code>inventario-localidades</code>
Reserva	id, clienteId, eventoId, localidadId, cantidad, total, estado, expiraEn	N Reservas – 1 Localidad; 1 Reserva – 0..1 Pago	Reservas / <code>reservas</code>
Pago	id, reservaId, monto, estado, referenciaPasarela, intentos	1 Pago – 1 Reserva	Pagos / <code>pagos</code>
Cliente	id, nombre, correo	1 Cliente – N Reservas	Identidad simplificada (<i>claims</i> del JWT)

5.2.4 Caso de uso de extremo a extremo

CU-01 – Reservar y pagar un ticket

Actor principal: Comprador. **Actores secundarios:** Pasarela de pagos, Servidor de correo.

Precondiciones: el comprador está autenticado (JWT); el evento está publicado y su localidad tiene cupos.

Flujo principal:

1. El comprador busca el evento, elige una localidad y la cantidad de cupos.
2. La app envía POST `/api/reservas`; el API Gateway valida el token y la admisión (sala de espera).
3. Reservas registra la reserva en estado PENDIENTE y responde 202 Accepted con el `reservaId`.
4. Inventario retiene los cupos y emite `AsientosRetenidos`; la reserva pasa a RETENIDA.
5. Pagos cobra en la pasarela y emite `PagoAprobado`.
6. Reservas confirma la reserva y emite `ReservaConfirmada`; Inventario convierte la retención en venta.
7. La app muestra “Reserva confirmada” y Notificaciones envía el correo.

Flujos alternos: **A1** sin cupos → `AsientosNoDisponibles`, reserva RECHAZADA; **A2** pago rechazado → `PagoRechazado`, reserva RECHAZADA e Inventario libera los cupos (compensación); **A3** vencen los 10 minutos → `ReservaExpirada`, Inventario libera los cupos; **A4** evento duplicado → el consumidor lo detecta y lo descarta (idempotencia); **A5** pago aprobado después de la expiración → `ReembolsoSolicitado` y Pagos reembolsa; **A6** demanda superior a la admisión → 429 y sala de espera en la app; **A7** evento que falla repetidamente → tras 3 reintentos pasa a la DLQ; **A8** `PagoAprobado` llega antes que `AsientosRetenidos` → la reserva se confirma directamente. El detalle de cada flujo está en la Tabla 5.11.

Postcondición: la reserva termina en un estado final (CONFIRMADA, RECHAZADA, EXPIRADA o REEMBOLSADA) y el inventario es consistente con ese estado.

5.3 Diagrama de alto nivel (HLD)

El HLD muestra la solución por capas de responsabilidad: clientes, borde, servicios de dominio con sus datos, columna vertebral de eventos, sistemas externos y observabilidad. Cada servicio aparece junto a su propio índice para hacer visible el principio *database per service*; físicamente todos los índices residen en un mismo clúster de Elasticsearch.

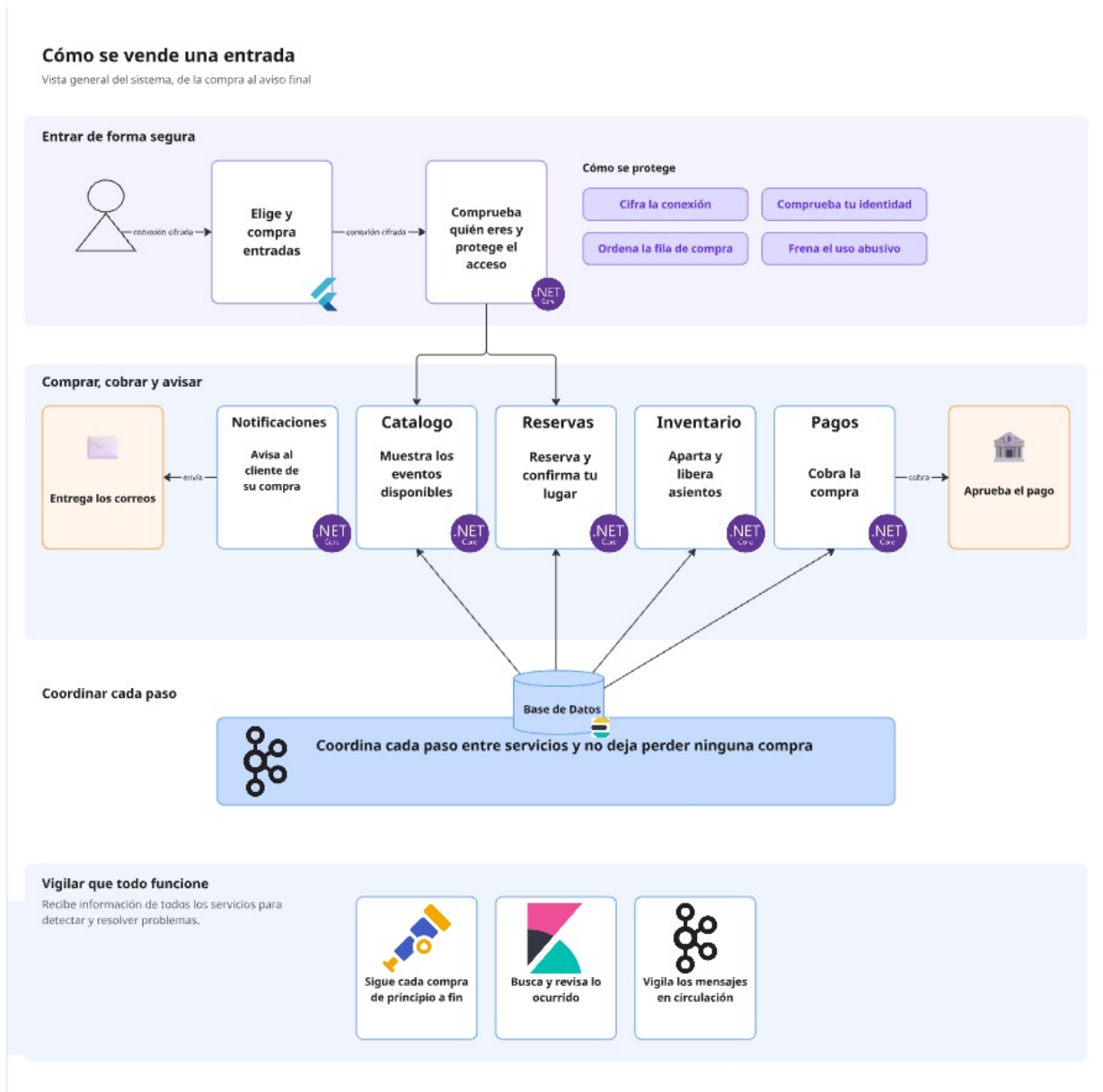


Figura 5.2: HLD

Tabla 5.6: Bloques del HLD y su justificación

Bloque	Responsabilidad y justificación	Decisión
App Flutter	Única interfaz para compradores y organizadores; envía comandos por REST y recibe estados por Server-Sent Events. Nunca habla con Kafka ni con Elasticsearch.	ADR-006
API Gateway	Punto de entrada único: TLS, validación de JWT, <i>rate limiting</i> por evento (sala de espera), enrutamiento a Catálogo y Reservas y paso del <i>stream</i> SSE. Equivale, en pequeño, al borde de Smart Queue. Se implementa con YARP para mantener un solo lenguaje y reutilizar el <i>middleware</i> de ASP.NET Core.	ADR-005, 006
Catálogo	Lado de lectura (CQRS): búsqueda de eventos con texto completo y disponibilidad proyectada desde <code>inventario.v1</code> . Publica <code>EventoPublicado</code> .	ADR-003
Reservas	Dueño del agregado Reserva y de la saga desde el punto de vista del comprador; única autoridad sobre su ciclo de vida; publica los cambios de estado por SSE y expira reservas vencidas.	ADR-004, 007
Inventario	Dueño de los cupos; única fuente de verdad sobre disponibilidad; escritor único por localidad, con concurrencia optimista como	ADR-002, 003

5.4 Diagrama de contexto (C4 – Nivel 1)

El diagrama de contexto ubica a TaquillaEDA frente a sus usuarios y a los sistemas externos con los que interactúa. A este nivel el sistema es una caja negra: no se distinguen servicios, tópicos ni índices.

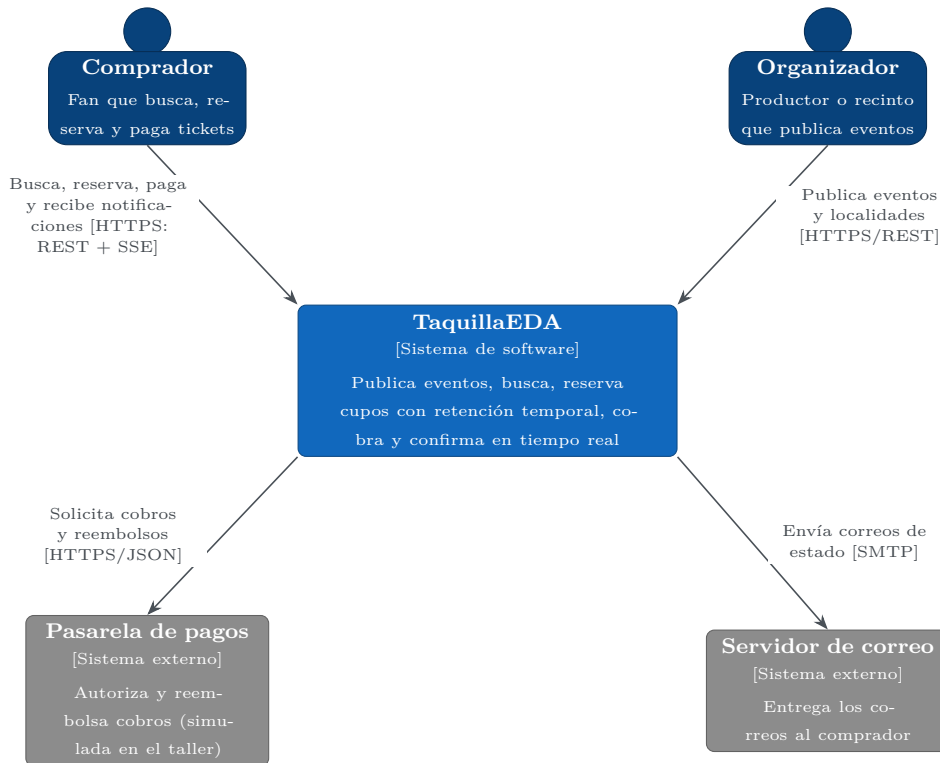


Figura 5.3: Diagrama de contexto (C4 nivel 1) de TaquillaEDA

Tabla 5.7: Elementos del diagrama de contexto

Elemento	Tipo	Interacción
Comprador	Persona	Usa la app para buscar, reservar y pagar; recibe estados en tiempo real y un correo final.
Organizador	Persona	Usa la app en modo organizador para publicar eventos con sus localidades (RF-01).
TaquillaEDA	Sistema en alcance	Sistema orientado a eventos objeto del taller.
Pasarela de pagos	Sistema externo	Autoriza cobros y reembolsos. En el taller es un contenedor simulado con latencia y tasa de fallo configurables para probar la resiliencia.
Servidor de correo	Sistema externo	Recibe correos por SMTP. En el taller se usa Mailpit, que captura los correos y los muestra en una interfaz web.

5.5 Diagrama de contenedores (C4 – Nivel 2)

El diagrama de contenedores abre la caja de TaquillaEDA. Por encima de Kafka están los servicios que el comprador alcanza de forma síncrona (a través del gateway); por debajo, los procesadores de eventos puramente asíncronos. Los índices se dibujan junto a su servicio dueño para enfatizar que ningún servicio lee los datos de otro.

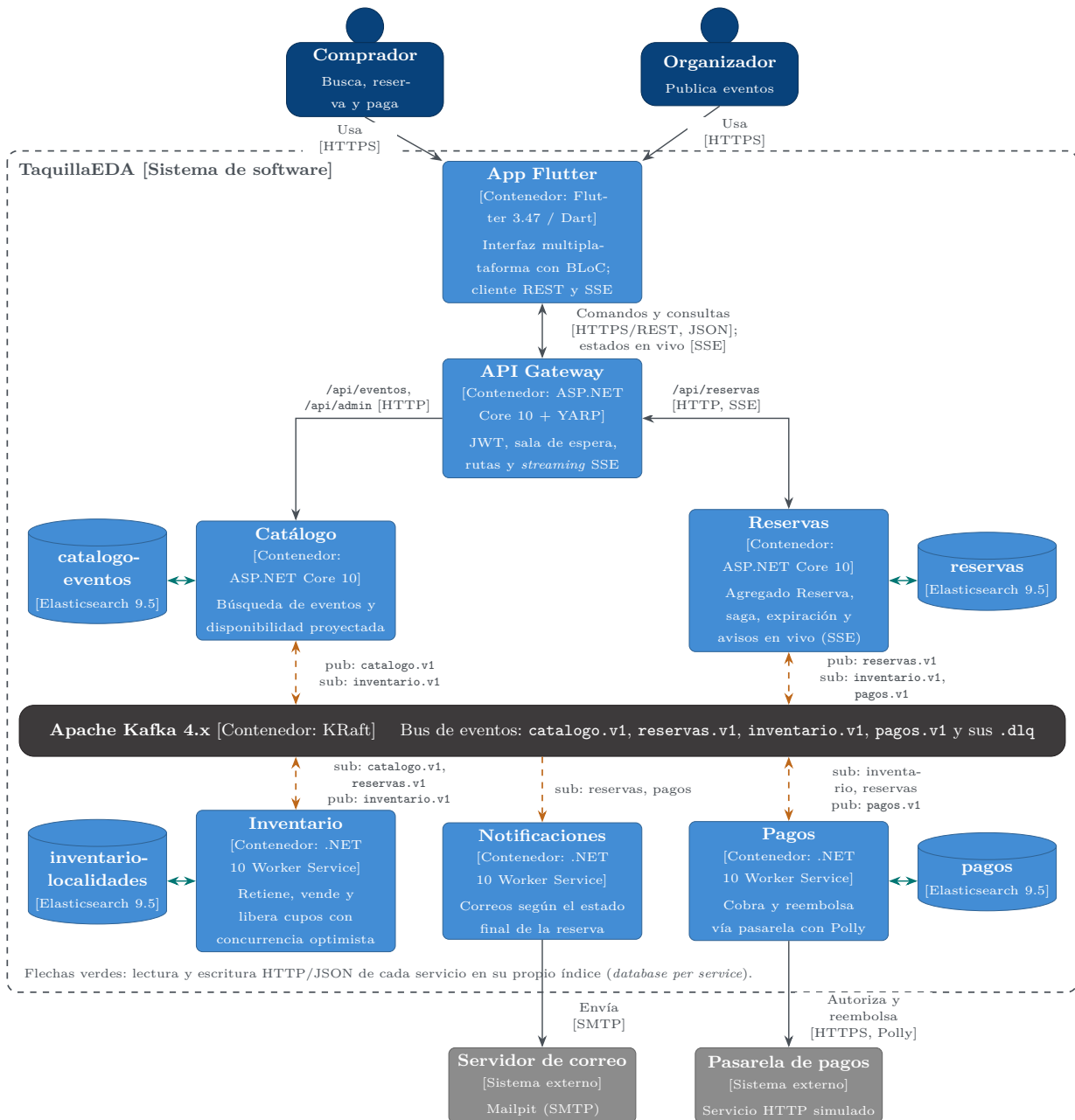


Figura 5.4: Diagrama de contenedores (C4 nivel 2) de TaquillaEDA. Discontinuas naranjas: publicación (pub) y suscripción (sub) de eventos

Tabla 5.8: Catálogo de contenedores

Contenedor	Tecnología	Responsabilidades	Interfaces
App Flutter	Flutter 3.47, Dart 3.13, dio (REST y lectura SSE en <i>streaming</i>), flutter_bloc	Pantallas de búsqueda, detalle, reserva, sala de espera y estado; BLoC que traduce notificaciones en estados de UI; al reconectarse, resincroniza con GET.	REST, SSE
API Gateway	ASP.NET Core 10, YARP, Rate Limiter	Terminación TLS, validación JWT, límite de admisión por evento (ventana deslizante), rutas y paso del <i>stream</i> SSE sin <i>buffering</i> .	:8000
Catálogo	ASP.NET Core 10 Minimal APIs, Elastic.Clients.Elasticsearch, Confluent.Kafka	GET /api/eventos, GET /api/eventos/{id}, POST /api/admin/eventos; proyecta disponibilidad desde inventario.v1.	REST; pub catalogo.v1
Reservas	ASP.NET Core 10 (SSE nativo), Worker interno	POST /api/reservas, GET /api/reservas/{id}, GET /api/reservas/{id}/eventos (SSE); máquina de estados; relevo del <i>outbox</i> ; expiración cada 30 s.	REST, SSE; pub reservas.v1
Inventario	.NET 10 Worker Service (2 réplicas)	Crear localidades, retener, confirmar venta y liberar cupos siguiendo las decisiones de Reservas; escritor único por localidad; idempotencia por reservaId.	pub inventario.v1
Pagos	.NET 10 Worker Service, HttpClient + Polly	Cobro y reembolso; reintentos, <i>timeout</i> de 3 s, <i>circuit breaker</i> .	pub pagos.v1
Notificaciones	.NET 10 Worker Service, MailKit	Plantillas de correo para confirmación, rechazo y expiración.	SMTP
Kafka	Apache Kafka 4.2 (KRaft, 1 nodo en el taller)	Tópicos de dominio (6 particiones) y DLQ; retención de 7 días.	:9092
Elasticsearch	Elasticsearch 9.5 (1 nodo en el taller)	Un índice por servicio más índices de eventos procesados.	:9200

5.6 Diagrama dinámico (flujo principal)

El diagrama dinámico describe la ejecución del CU-01 en su camino feliz. Se presenta en forma de secuencia (variante admitida por C4 para diagramas dinámicos) porque en una coreografía el *orden temporal* de los eventos es lo que el lector necesita seguir. Kafka aparece como participante para hacer explícito que ningún servicio llama directamente a otro.

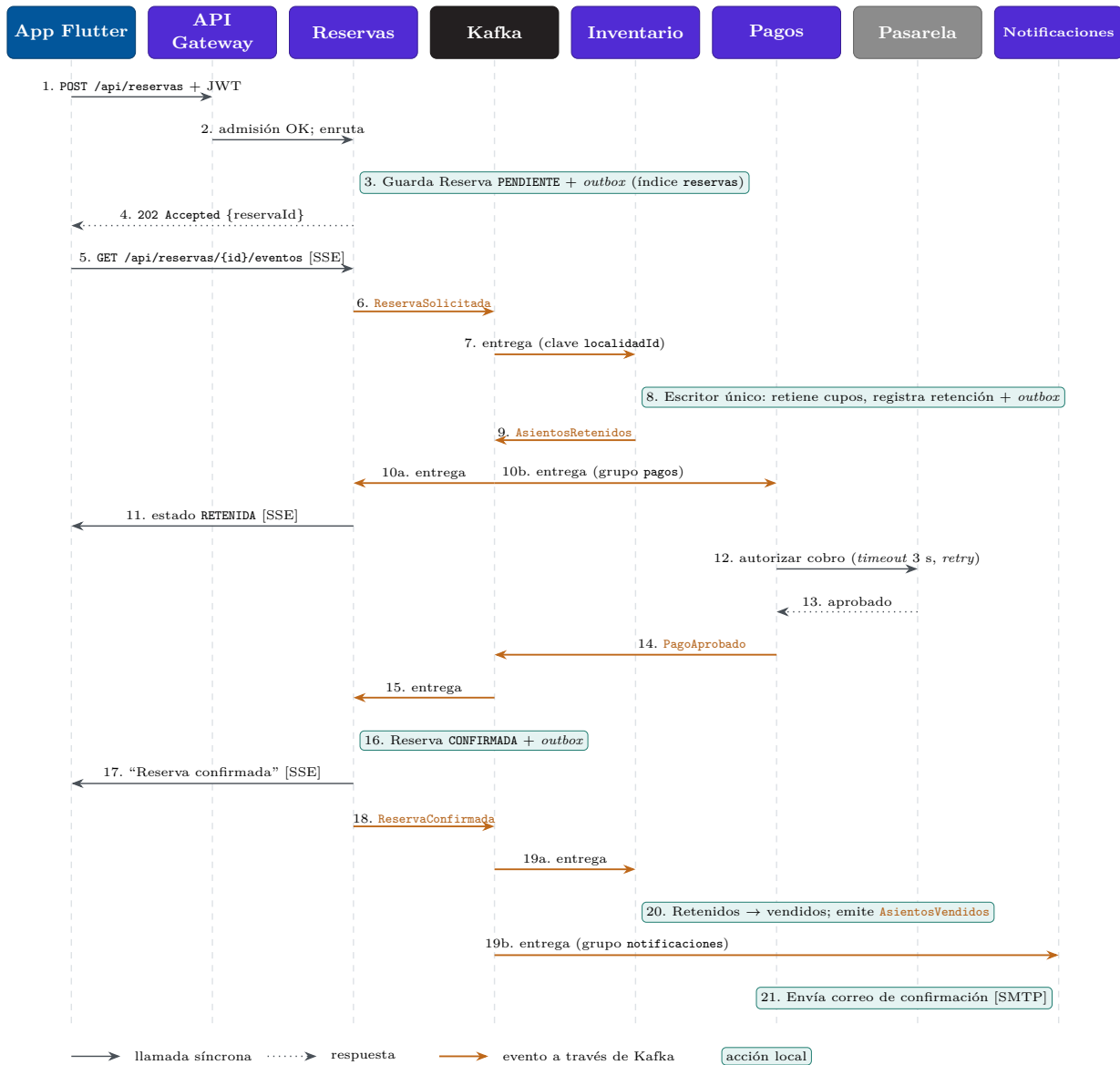


Figura 5.5: Diagrama dinámico del CU-01 “Reservar y pagar un ticket” (camino feliz)

Tabla 5.9: Pasos del flujo principal, tópicos y patrones aplicados

Paso	Descripción	Tópico / clave	Patrón o táctica
1–2	El gateway valida el JWT y la cuota de admisión del evento; si se supera, responde 429 con Retry-After (flujo A6).	–	API Gateway, <i>rate limiting</i>

Paso	Descripción	Tópico / clave	Patrón o táctica
3–4	Reservas valida la solicitud (cantidad 1–6, evento vigente), guarda la reserva y el evento pendiente en el <i>mismo documento</i> y responde sin esperar.	–	Transactional Outbox, respuesta asíncrona
5	La app abre el <i>stream</i> SSE de su reserva (con JWT en la cabecera). Si la conexión se cae, primero consulta <code>GET /api/reservas/{id}</code> para resincronizarse y luego reabre el <i>stream</i> .	–	Notificación <i>push</i>
6	El relevo del <i>outbox</i> publica con productor idempotente y marca el evento como publicado.	<code>reservas.v1 / localidadId</code>	Outbox relay, idempotencia del productor
7–9	Todos los eventos de la localidad llegan a la misma partición, así que una sola instancia de Inventario los procesa en orden (escritor único). Deduplica, descuenta disponibles y guarda condicionado a la versión; el 409 solo aparece si un rebalanceo deja dos instancias sobre la partición.	<code>inventario.v1 / reservaId</code>	Escritor único, concurrencia optimista, consumidor idempotente
10–11	Reservas y Pagos reciben el mismo evento en grupos de consumidores distintos (pub/sub).	<code>inventario.v1</code>	Publish–Subscribe
12–13	Pagos invoca la pasarela con <i>timeout</i> , reintentos y <i>circuit breaker</i> ; la clave de idempotencia de la pasarela es el <code>reservaId</code> .	–	Retry, Timeout, Circuit Breaker
14–17	Reservas, única autoridad sobre el ciclo de vida de la reserva, recibe el resultado del pago, confirma y avisa a la app.	<code>pagos.v1 / reservaId</code>	Saga coreografiada
18–20	Inventario convierte la retención en venta al recibir la <i>decisión</i> de Reservas, no el evento de pago; así nunca vende cupos de una reserva que Reservas ya dio por expirada.	<code>reservas.v1 / localidadId</code>	Saga coreografiada, fuente única de decisión
20	AsientosVendidos actualiza la disponibilidad proyectada en Catálogo (no dibujado).	<code>inventario.v1</code>	CQRS, vista materializada
19b–21	El comprador recibe el correo de confirmación.	<code>reservas.v1</code>	Event Notification

5.6.1 Coreografía completa y compensaciones

En la topología Broker ningún componente conoce el flujo completo; por eso se documenta explícitamente *quién reacciona a qué*. La Tabla 5.10 es la “partitura” de la saga: cada fila es un evento y cada columna la reacción de un servicio.

Tabla 5.10: Matriz de coreografía: reacción de cada servicio a cada evento

Evento	Tópico	Catálogo	Reservas	Inventario	Pagos	Notificaciones
EventoPublicado	catalogo.v1	productor	–	Crea un documento por localidad con sus cupos	–	–
ReservaSolicitada	reservas.v1	–	productor	Intenta retener cupos	–	–
AsientosRetenidos	inventario.v1	Actualiza disponibles	Estado RETENIDA (si sigue PENDIENTE); notifica por SSE	productor	Inicia el cobro	–
AsientosNoDisponibles	inventario.v1	–	Estado RECHAZADA; notifica	productor	–	–
PagoAprobado	pagos.v1	–	Estado CONFIRMADA desde PENDIENTE o RETENIDA; si ya expiró, pide reembolso	–	productor	–
PagoRechazado	pagos.v1	–	Estado RECHAZADA; notifica	–	productor	–
AsientosVendidos / AsientosLiberados	inventario.v1	Actualiza disponibles	–	productor	–	–
ReservaConfirmada	reservas.v1	–	productor	Retenidos → vendidos	–	Correo de confirmación
ReservaRechazada	reservas.v1	–	productor	Compensa: libera cupos si estaban retenidos	–	Correo de rechazo
ReservaExpirada	reservas.v1	–	productor	Compensa: libera cupos	No inicia cobros pendientes	Correo de expiración
ReembolsoSolicitado	reservas.v1	–	productor	–	Reembolsa en la pasarela	–

Evento	Tópico	Catálogo	Reservas	Inventario	Pagos	Notificaciones
PagoReembolsado	pagos.v1	–	Estado REEMBOLSADA	–	<i>productor</i>	Correo de reembolso

5.6.2 Máquina de estados de la reserva

La consistencia eventual se hace visible para el usuario mediante estados explícitos (táctica “mantener el modelo de la tarea”). Solo las transiciones de la Figura 5.6 son válidas y los estados solo avanzan. Como **AsientosRetenidos** y **PagoAprobado** viajan por tópicos distintos, Kafka no garantiza su orden relativo: si el pago llega primero, la reserva pasa directamente a **CONFIRMADA** (Pagos solo cobra después de una retención) y la retención que llega tarde se ignora. Un evento que ya fue superado se registra y se descarta, lo que vuelve al consumidor tolerante a desorden y duplicados. Reservas es la única autoridad sobre el ciclo de vida: Inventario vende o libera cupos siguiendo sus decisiones, nunca los eventos de pago.

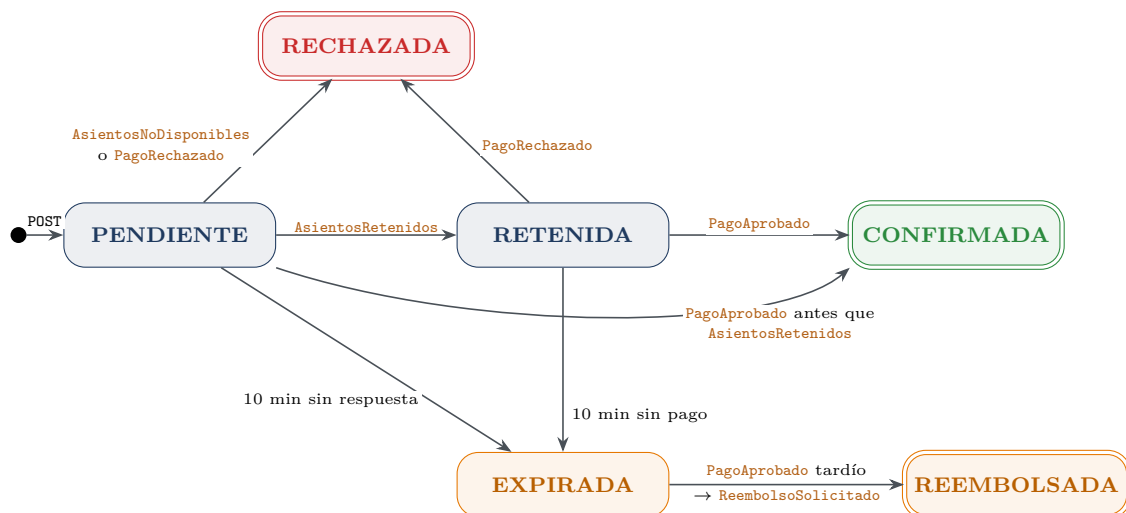


Figura 5.6: Máquina de estados del agregado Reserva (doble borde: estados finales)

Tabla 5.11: Flujos alternos y mecanismo que los resuelve

Flujo	Qué ocurre	Mecanismo
A1 Sin cupos	Inventario no puede retener; emite AsientosNoDisponibles y la reserva termina RECHAZADA sin cobro.	Evento de rechazo
A2 Pago rechazado	Pagos emite PagoRechazado ; Reservas marca RECHAZADA y emite ReservaRechazada ; Inventario devuelve los cupos retenidos.	Transacción compensatoria
A3 Expiración	Un proceso periódico de Reservas (cada 30 s) busca reservas con expiraEn vencido, las marca EXPIRADA y emite el evento; Inventario libera y Pagos no inicia cobros pendientes.	Compensación por tiempo
A4 Duplicado	Un evento reentregado se detecta por su id (índice de procesados) o por el reservaId ya registrado en la localidad, y se ignora.	Consumidor idempotente
A5 Pago tardío	El pago se aprueba después de la expiración; Reservas no confirma y emite ReembolsoSolicitado .	Compensación sobre un tercero
A6 Sobredemanda	El gateway rechaza con 429 + Retry-After ; la app muestra la sala de espera y reintenta con <i>jitter</i> .	<i>Rate limiting</i> , sala de espera
A7 Mensaje envenenado	Tras 3 reintentos con <i>backoff</i> el evento pasa a <tópico>.dlq con la causa en sus cabeceras; se revisa en Kafbat UI.	Retry + DLQ
A8 Eventos cruzados	PagoAprobado llega antes que AsientosRetenidos porque viajan por tópicos distintos; Reservas confirma directamente e ignora la retención tardía.	Estados que solo avanzan

5.7 Diagrama de despliegue

Todo el sistema se despliega con Docker Compose en un único equipo anfitrión (ADR-008), lo que garantiza que la demostración sea reproducible con un solo comando. Los contenedores se comunican por nombre DNS dentro de la red **taquilla-net**; solo se publican hacia el anfitrión los puertos necesarios para usar y observar el sistema y los que necesitan las réplicas que corren en otros equipos (Kafka en el *listener* externo 9094, Elasticsearch, la pasarela simulada y la entrada OTLP de Aspire). Las imágenes propias se construyen con un único **Dockerfile** multietapa, parametrizado con el nombre del servicio.

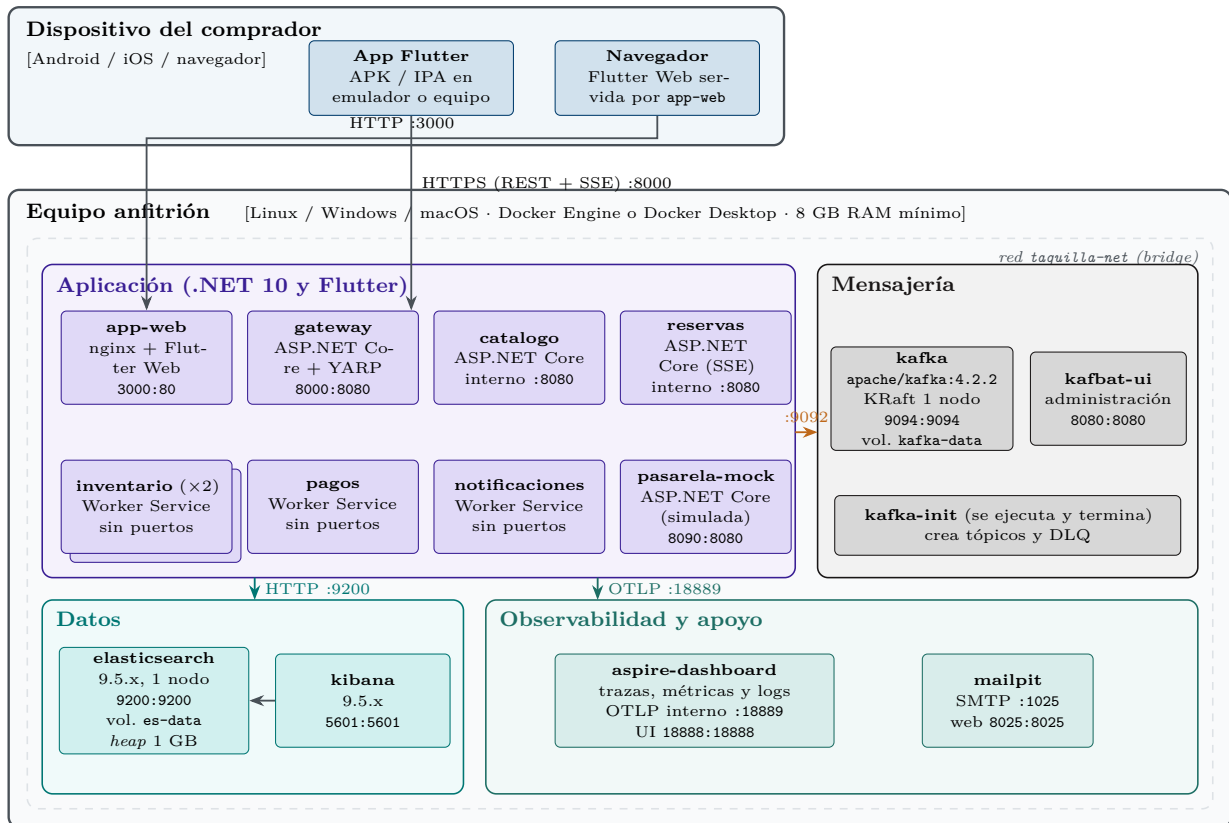


Figura 5.7: Diagrama de despliegue de TaquillaEDA con Docker Compose

Tabla 5.12: Configuración de despliegue por contenedor

Servicio	Imagen	Puerto host	Rép.	Arranca cuando... / <i>health check</i>
kafka	apache/kafka:4.2.2	9094	1	<i>Health check</i> : kafka-broker-api-versions.sh
kafka-init	apache/kafka:4.2.2	—	1	kafka saludable; crea 4 tópicos (6 particiones) y 4 DLQ y termina
elasticsearch	elasticsearch:9.5.x	9200	1	<i>Health check</i> : _cluster/health distinto de red
kibana	kibana:9.5.x	5601	1	elasticsearch saludable
gateway	taquilla/gateway (propia)	8000	1	catálogo y reservas saludables; /health/ready
catalogo, reservas	taquilla/<servicio> (propias)	—	1	kafka-init completado y elasticsearch saludable; /health/live y /health/ready
inventario	taquilla/inventario (propia)	—	2	igual que el anterior; las 2 réplicas comparten el grupo inventario

Servicio	Imagen	Puerto host	Rép.	Arranca cuando... / <i>health check</i>
pagos, notificaciones	taquilla/<servicio> (propias)	–	1	igual que el anterior; notificaciones además requiere mailpit
pasarela- mock	taquilla/ pasarela-mock	8090	1	variables LATENCIA_MS, TASA_FALLO, TASA_RECHAZO y MONTO_MAXIMO para pruebas de resiliencia; el puerto se publica para que Pagos de otros equipos cobre en la misma pasarela
app-web	nginx:alpine + <i>build</i> de Flutter Web	3000	1	gateway saludable
aspire- dashboard, mailpit, kafbat-ui	mcr.microsoft.com/ dotnet/ aspire-dashboard, axllent/mailpit, ghcr.io/kafbat/ kafka-ui	18888, 8025, 8080	1	sin dependencias de negocio

Extracto del archivo `docker-compose.yml`. Muestra los tres patrones de despliegue relevantes: Kafka en modo KRaft sin ZooKeeper, creación explícita de tópicos (la creación automática se desactiva para que los tópicos sean parte del contrato) y arranque ordenado por estado saludable.

`docker-compose.yml` (extracto)

```
services:
  kafka:
    image: apache/kafka:4.2.2
    environment:
      KAFKA_NODE_ID: 1
      KAFKA_PROCESS_ROLES: broker,controller          # KRaft: sin ZooKeeper
      KAFKA_LISTENERS: PLAINTEXT://:9092,CONTROLLER://:9093,EXTERNAL://:9094
      KAFKA_ADVERTISED_LISTENERS: PLAINTEXT://kafka:9092,EXTERNAL://${IP_ANFITRION:-localhost}:9094
      KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: CONTROLLER:PLAINTEXT,PLAINTEXT:PLAINTEXT,EXTERNAL:PLAINTEXT
      KAFKA_CONTROLLER_LISTENER_NAMES: CONTROLLER
      KAFKA_CONTROLLER_QUORUM_VOTERS: 1@kafka:9093
      KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
      KAFKA_AUTO_CREATE_TOPICS_ENABLE: "false"        # los topicos son parte del contrato
    volumes: [kafka-data:/var/lib/kafka/data]
    ports: ["9094:9094"]
    healthcheck:
      test: ["CMD-SHELL", "/opt/kafka/bin/kafka-broker-api-versions.sh --bootstrap-server localhost:9092"]
      interval: 10s
      retries: 12

  kafka-init:
    image: apache/kafka:4.2.2
    depends_on: { kafka: { condition: service_healthy } }
    entrypoint: ["/bin/sh", "-c", "for t in catalogo.v1 reservas.v1 inventario.v1 pagos.v1; do
```

```

/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --create --if-not-exists --topic $$t
--partitions 6 --replication-factor 1 --config retention.ms=604800000;
/opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 --create --if-not-exists --topic $$t.
dlq --partitions 1 --replication-factor 1; done"]

elasticsearch:
  image: docker.elastic.co/elasticsearch/elasticsearch:9.5.4
  environment:
    discovery.type: single-node
    xpack.security.enabled: "false" # solo para el taller
    ES_JAVA_OPTS: -Xms1g -Xmx1g
  volumes: [es-data:/usr/share/elasticsearch/data]
  ports: ["9200:9200"]
  healthcheck:
    test: ["CMD-SHELL", "curl -s localhost:9200/_cluster/health | grep -vq '\"status\": \"red\"'"]
    interval: 10s
    retries: 12

inventario:
  build: { context: ., dockerfile: src/Dockerfile, args: { SERVICIO: Inventario } }
  image: taquilla/inventario:1.0.0
  deploy: { replicas: 2 } # mismo group.id: Kafka reparte particiones
  environment:
    Kafka__BootstrapServers: kafka:9092
    Kafka__GroupId: inventario
    Elastic__Url: http://elasticsearch:9200
    OTEL_EXPORTER_OTLP_ENDPOINT: http://aspire-dashboard:18889
  depends_on:
    kafka-init: { condition: service_completed_successfully }
    elasticsearch: { condition: service_healthy }
  restart: on-failure

volumes:
  kafka-data:
  es-data:

```

5.7.1 Del taller a producción

La Tabla 5.13 muestra cómo evolucionaría el despliegue hacia un entorno productivo, tomando como guía las prácticas de la arquitectura de referencia (Kubernetes con *health checks*, escalado en picos y alta disponibilidad).

Tabla 5.13: Despliegue del taller frente a un despliegue productivo

Aspecto	Taller (Docker Compose)	Producción (referencia)
Orquestación	Compose en un equipo, reinicio on-failure	Kubernetes gestionado (p. ej. EKS, como Ticketmaster) con sondas <i>liveness/readiness</i> y varias zonas de disponibilidad
Kafka	1 nodo combinado <i>broker</i> /controlador, RF=1	≥ 3 <i>brokers</i> y 3 controladores KRaft, RF=3, <code>min.insync.replicas=2</code> , TLS + SASL y ACL por tópico
Elasticsearch	1 nodo, seguridad desactivada	≥ 3 nodos, una réplica por <i>shard</i> , TLS, usuarios por servicio y <i>snapshots</i> programados
Borde	Gateway con <i>rate limiting</i> en memoria	CDN + WAF + balanceador; sala de espera dedicada con estado distribuido y filtro de bots
Escalado	Réplicas fijas	Escalado automático por <i>lag</i> de consumidores (p. ej. KEDA) y <i>cloud bursting</i> para aperturas de venta
Observabilidad	Aspire Dashboard (en memoria), Kibana	Backend persistente (p. ej. Jaeger o Tempo para trazas, Prometheus y Grafana para métricas) con alertas sobre <i>lag</i> , DLQ y errores
Secretos	Variables de entorno	Gestor de secretos y rotación de credenciales

5.8 Diagramas opcionales

El taller marca como opcionales cuatro vistas adicionales. Se incluyen todas porque cada una responde una pregunta distinta del diseño: dónde encaja el sistema en su ecosistema (*landscape*), cómo se evita la sobreventa dentro del servicio más crítico (componentes), cómo se traduce eso a clases (código) y cómo se organizan y versionan los datos (modelo de datos).

5.8.1 System landscape

La vista de paisaje ubica a TaquillaEDA dentro del ecosistema en el que operaría. Los sistemas punteados no se construyen en el taller: representan capacidades que, como en Ticketmaster (antifraude, reventa, vista única del fan), se agregarían como *nuevos consumidores* de los tópicos existentes sin modificar ningún servicio. Es la demostración visual del atributo de extensibilidad que justifica el estilo.

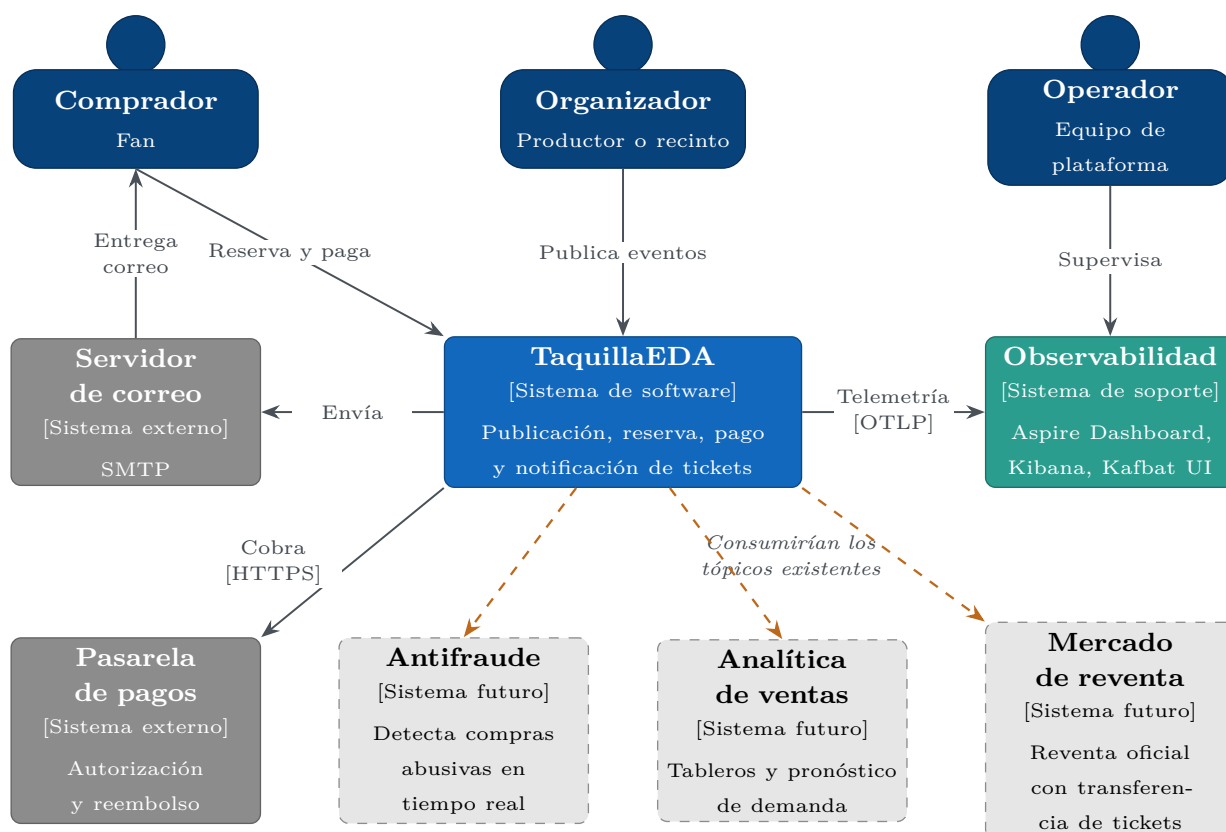


Figura 5.8: System landscape de TaquillaEDA (punteado: sistemas futuros que consumirían los eventos existentes)

5.8.2 Diagrama de componentes (C4 – Nivel 3): servicio de Inventario

Se detalla el servicio de Inventario porque concentra el riesgo principal del dominio: la **sobreventa** bajo concurrencia. Internamente sigue una arquitectura hexagonal: el dominio (agregado Localidad) no conoce a Kafka ni a Elasticsearch; los adaptadores implementan los puertos definidos por el dominio.

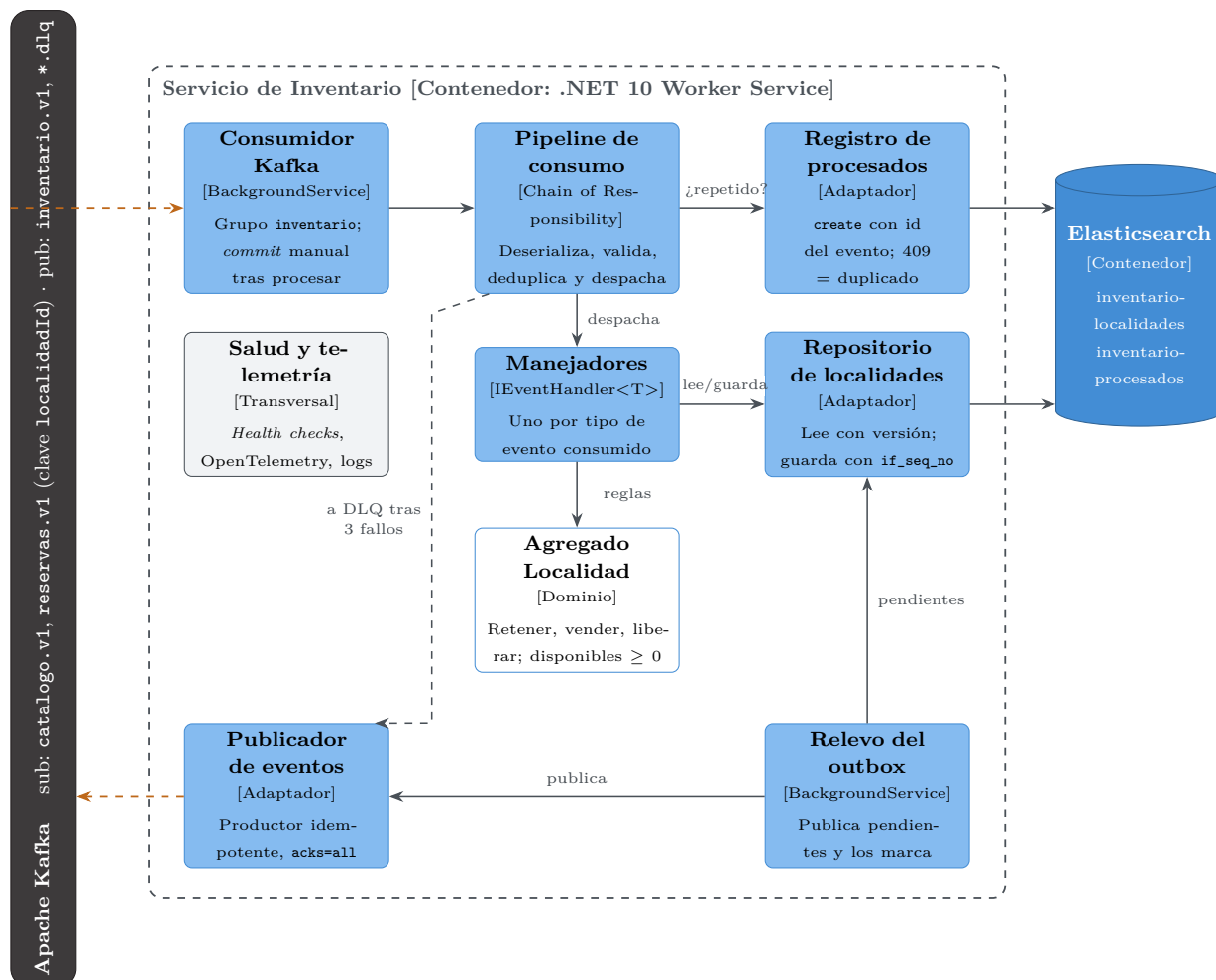


Figura 5.9: Diagrama de componentes (C4 nivel 3) del servicio de Inventario

Por qué el diseño evita la sobreventa. Se combinan dos mecanismos. El principal es el **escritor único**: los eventos que modifican una localidad viajan por `reservas.v1` con clave `localidadId`, así que todos caen en la misma partición y una sola instancia de Inventario los procesa, en orden. Como un documento de Elasticsearch no admite escrituras paralelas útiles, procesar en serie no cuesta rendimiento, y además permite agrupar varias solicitudes de la misma localidad en una sola escritura. El segundo es la **concurrency optimista** como salvaguarda: si durante un rebalanceo dos instancias procesan brevemente la misma partición, la segunda escritura recibe `409 Conflict`, relee y vuelve a evaluar la regla. Como todo el estado de la localidad (contadores, retenciones y *outbox*) vive en un solo documento, la invariante “`disponibles ≥ 0`” se cumple sin transacciones distribuidas. Esta es la adaptación, al stack asignado, de la retención temporal de asientos del dominio de referencia.

5.8.3 Diagrama de código

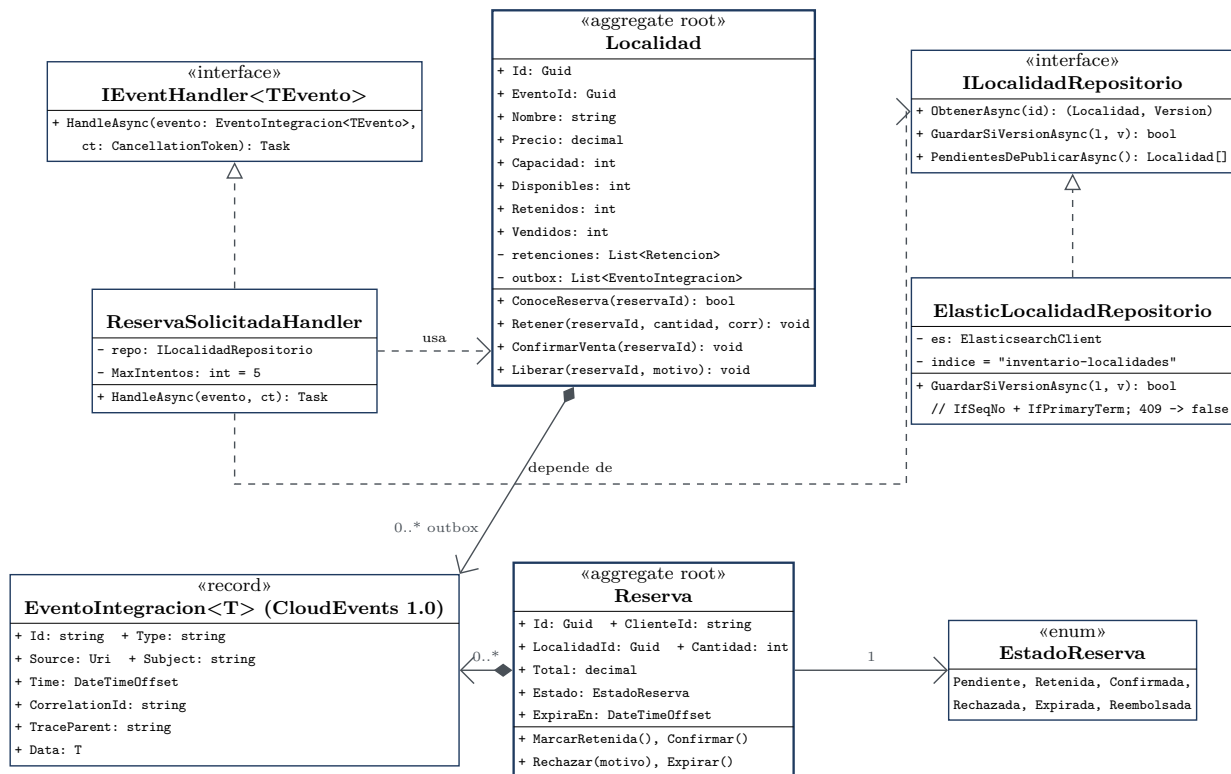


Figura 5.10: Diagrama de código (clases principales del servicio de Inventario y del agregado Reserva)

El siguiente esbozo muestra cómo el manejador combina las tres garantías del diseño —idempotencia, concurrencia optimista y *outbox*— en pocas líneas. Es ilustrativo: en el código implementado, la deduplicación, los reintentos y el envío a la DLQ viven en el *pipeline* de consumo común de *BuildingBlocks* (una cadena de pasos: deserializar, trazar, filtrar, reintentar o DLQ, deduplicar y despachar), y el manejador solo aplica la regla de negocio a través de un actualizador que relea y guarda condicionado a la versión.

ReservaSolicitadaHandler.cs y ElasticLocalidadRepositorio.cs (esbozo)

```

public sealed class ReservaSolicitadaHandler : IEventHandler<ReservaSolicitada>
{
    private const int MaxIntentos = 5;
    private readonly ILocalidadRepositorio _repo;

    public ReservaSolicitadaHandler(ILocalidadRepositorio repo) => _repo = repo;

    public async Task HandleAsync(EventoIntegracion<ReservaSolicitada> evento, CancellationToken ct)
    {
        var datos = evento.Data;
        for (var intento = 1; intento <= MaxIntentos; intento++)
        {
            // 1. Leer el agregado junto con su version (_seq_no, _primary_term)
            var (localidad, version) = await _repo.ObtenerAsync(datos.LocalidadId, ct);

            // 2. Idempotencia natural: esta reserva ya fue procesada
            if (localidad.ConoceReserva(datos.ReservaId)) return;

            // 3. Regla de negocio: retiene o rechaza y agrega el evento al outbox
        }
    }
}
  
```

```

// (AsientosRetenidos o AsientosNoDisponibles) en el MISMO documento
localidad.Retener(datos.ReservaId, datos.Cantidad, evento.CorrelationId);

// 4. Guardado condicional: devuelve false (409) si otro proceso escribio antes
if (await _repo.GuardarSiVersionAsync(localidad, version, ct)) return;
}
// Agotados los intentos: el pipeline reintenta con backoff y luego envia a la DLQ
throw new ConflictoConcurrenciaException(datos.LocalidadId);
}
}

public async Task<bool> GuardarSiVersionAsync(Localidad l, Version v, CancellationToken ct)
{
    var r = await _es.IndexAsync(l, i => i
        .Index("inventario-localidades").Id(l.Id)
        .IfSeqNo(v.SeqNo).IfPrimaryTerm(v.PrimaryTerm), ct);

    if (r.ApiCallDetails.HttpStatusCode == 409) return false; // conflicto optimista
    if (!r.IsValidResponse) throw new ErrorPersistenciaException(r.DebugInformation);
    return true;
}

```

5.8.4 Modelo de datos

Modelo conceptual. Las relaciones entre entidades de distintos servicios se mantienen **por identificador**, nunca por claves foráneas ni consultas cruzadas; la coherencia entre ellas la garantizan los eventos. Cada recuadro punteado es la frontera de datos de un servicio.

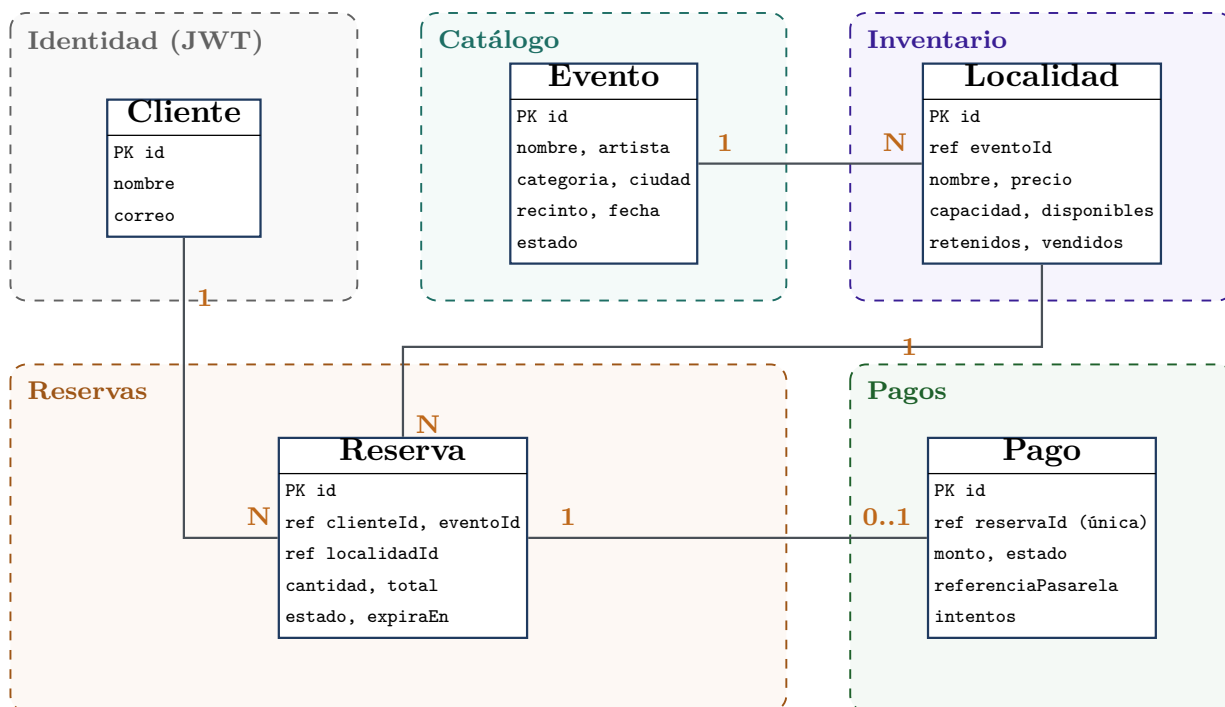


Figura 5.11: Modelo conceptual de datos con fronteras de propiedad por servicio (ref: referencia por identificador sin clave foránea)

Modelo físico en Elasticsearch. Los campos que solo se leen como parte del documento completo (retenciones y *outbox*) se declaran con **enabled: false**: se guardan pero no se indexan,

lo que evita el crecimiento descontrolado del *mapping*. El campo booleano `tieneOutboxPendiente` sí se indexa para que el relevo encuentre rápidamente los documentos con eventos por publicar.

Tabla 5.14: Mappings principales de los índices

Campo	Tipo	Uso
catalogo-eventos (Catálogo) – lado de lectura		
nombre, artista	text (analizador <code>spanish</code>) + keyword	Búsqueda de texto completo con <i>stemming</i> y ordenamiento exacto
categoria, ciudad	keyword	Filtros y facetas (agregaciones)
fecha	date	Rango de fechas y orden
localidades	nested	Nombre, precio y <code>disponibles</code> proyectados por localidad; <code>versionInventario</code> para aplicar solo eventos más recientes
inventario-localidades (Inventario) – fuente de verdad de cupos		
eventoId	keyword	Localidades de un evento
capacidad, disponibles, retenidos, vendidos	integer	Contadores; invariante <code>disponibles + retenidos + vendidos = capacidad</code>
retenciones	object, enabled: false	Lista { <code>reservaId</code> , cantidad, estado, <code>expiraEn</code> }; soporte de idempotencia y compensación
outbox	object, enabled: false	Eventos pendientes y publicados (se depuran periódicamente)
tieneOutbox Pendiente	boolean	Consulta del relevo del <i>outbox</i>
reservas (Reservas)		
clienteId, localidadId, estado	keyword	Consultas del comprador y del proceso de expiración
expiraEn	date	Búsqueda de reservas vencidas (<code>estado</code> ∈ {PENDIENTE, RETENIDA} y <code>expiraEn</code> < ahora)
total	scaled_float (factor 100)	Valor monetario sin errores de redondeo
historial, outbox	object, enabled: false	Traza de transiciones y eventos pendientes
pagos (Pagos)		

Campo	Tipo	Uso
<code>_id = reservaId</code>	–	Un pago por reserva: la creación con <code>op_type=create</code> impide cobros duplicados
<code>estado,</code> <code>referenciaPasarela</code>	keyword	Conciliación y reembolsos
<servicio>-procesados (cada servicio consumidor)		
<code>_id = id del evento</code>	–	Consumidor idempotente: <code>op_type=create</code> ; un 409 indica evento repetido
<code>procesadoEn</code>	date	Depuración por antigüedad (más allá de la retención de Kafka)

Ejemplo de documento y de evento. El documento de la localidad muestra cómo el estado, la retención y el evento pendiente viajan juntos en una sola escritura atómica; el evento muestra el sobre CloudEvents con la trazabilidad (`correlationid` y `traceparent`).

inventario-localidades/_doc/6f1c... y evento AsientosRetenidos

```
{ "eventoId": "e-2026-rock-al-parque", "nombre": "Preferencial", "precio": 250000,
  "capacidad": 500, "disponibles": 412, "retenidos": 8, "vendidos": 80,
  "retenciones": [
    { "reservaId": "r-91a2", "cantidad": 2, "estado": "RETENIDA", "expiraEn": "2026-10-03T15:10:00Z" } ],
  "tieneOutboxPendiente": true,
  "outbox": [
    { "id": "evt-5c1e", "type": "co.taquilla.inventario.asientos-retenidos.v1", "publicado": false } ] }

{ "specversion": "1.0",
  "id": "evt-5c1e",
  "type": "co.taquilla.inventario.asientos-retenidos.v1",
  "source": "/servicios/inventario",
  "subject": "reservas/r-91a2",
  "time": "2026-10-03T15:00:02Z",
  "datacontenttype": "application/json",
  "correlationid": "r-91a2",
  "traceparent": "00-4bf92f3577b34da6a3ce929d0e0e4736-00f067aa0ba902b7-01",
  "data": { "reservaId": "r-91a2", "localidadId": "6f1c", "cantidad": 2,
    "precioUnitario": 250000, "disponibles": 412, "versionInventario": 1873 } }
```

Tabla 5.15: Tópicos de Kafka, claves de partición y grupos de consumidores

Tópico	Part.	Clave	Productor	Grupos de consumidores
catalogo.v1	6	eventoId	Catálogo	inventario
reservas.v1	6	localidadId	Reservas	inventario, pagos, notificaciones
inventario.v1	6	reservaId	Inventario	catalogo, reservas, pagos
pagos.v1	6	reservaId	Pagos	reservas, notificaciones
<tópico>.dlq	1	la original	pipeline de cada consumidor	Revisión manual en Kafbat UI y reinyección

Configuración de tópicos. Cada tópico usa como clave el agregado que debe procesar sus eventos en orden. `reservas.v1` usa `localidadId` porque su consumidor crítico es `Inventario`, que modifica el documento de la localidad; como cada reserva pertenece a una sola localidad, los eventos de una misma reserva también quedan ordenados entre sí. `inventario.v1` y `pagos.v1` usan `reservaId` para repartir entre todas las particiones el trabajo de Pagos y Reservas, que modifican un documento distinto por reserva: si Pagos se particionara por localidad, todos los cobros de la localidad más demandada esperarían a la pasarela en un solo consumidor. La contrapartida es que la localidad más solicitada concentra su carga de inventario en una partición —el problema de toda gran apertura de venta—; se acepta porque el cuello de botella real es el documento, no la partición, y se mitiga agrupando escrituras. Catálogo tolera el desorden porque solo aplica una actualización si su `versionInventario` es mayor que la almacenada. Las colas nativas de Kafka (*share groups*), que encajarían con Pagos, no se usan porque en 2026 su cliente .NET aún no está disponible.

5.9 Trazabilidad entre decisiones y diseño

La Tabla 5.16 cierra el ciclo entre el análisis (Capítulo 4) y el diseño: cada ADR se materializa en elementos concretos de los diagramas, y cada elemento se puede verificar en la demostración.

Tabla 5.16: Trazabilidad ADR – elementos de diseño – evidencia

ADR	Dónde se ve en el diseño	Evidencia en la demo
001 EDA Broker	Figuras 5.4 y 5.5: sin orquestador; Tabla 5.10 de coreografía	Agregar un consumidor nuevo sin tocar código existente
002 Kafka 4.x	Contenedor Kafka (KRaft), Tabla 5.15 de tópicos y claves	Kafbat UI: tópicos, particiones, <i>lag</i> por grupo
003 Elasticsearch por servicio	Índices junto a cada servicio (Figuras 5.3 y 5.4); Tabla 5.14; OCC en la Figura 5.9	Prueba de 100 reservas concurrentes sin sobreventa
004 Outbox, idempotencia, DLQ	Relevo del <i>outbox</i> y registro de procesados (Figura 5.9); flujos A4 y A7	Reenviar un evento y verificar que no se duplica; ver la DLQ
005 .NET 10	Contenedores ASP.NET Core y Worker Services; diagrama de código	Imágenes construidas con <code>docker compose build</code>
006 Flutter + REST + SSE	App, gateway YARP y <i>stream</i> SSE de Reservas; pasos 1, 5, 11 y 17 del flujo	Estados en vivo en la app; 429 y sala de espera

ADR	Dónde se ve en el diseño	Evidencia en la demo
007 Saga coreografiada	Máquina de estados (Figura 5.6); Tablas 5.10 y 5.11	Pasarela con TASA_FALLO alta: cupos liberados
008 Docker Compose + observabilidad	Figura 5.7; Tabla 5.12	Traza y logs completos en Aspire Dashboard con un solo <code>traceId</code>

5.10 Guía de implementación y repositorio

La implementación sigue la estructura del diseño: un proyecto por contenedor, contratos de eventos compartidos como esquemas y como tipos de datos sin lógica, y una librería transversal (`BuildingBlocks`) que concentra el *pipeline* de consumo, el relevo del *outbox*, la idempotencia, la DLQ, la telemetría y los *health checks*, para no repetir esa lógica en cada servicio. El código está publicado en <https://github.com/CarlitosPinzon/EDA-Proyect>.

5.10.1 Estructura del repositorio

github.com/CarlitosPinzon/EDA-Proyect

```

EDA-Proyect/
|-- README.md                descripcion, tecnologias, despliegue y escenarios
|-- GUIA-DEMO-2PCS.md        demostracion en dos equipos y escenarios de caida
|-- ENCENDER-NODO-WINDOWS.md encender un nodo remoto en cualquier Windows
|-- docker-compose.yml       despliegue completo (ADR-008)
|-- docker-compose.nodo-remoto.yml replicas de Inventario y Pagos en otro equipo
|-- iniciar-nodo.cmd, detener-nodo.cmd  nodo remoto con doble clic (Windows)
|-- .env.example              configuracion de la demo (expiracion, sala de espera, fallas)
|-- docs/
|   |-- adr/                  ADR-001 ... ADR-008 en Markdown
|   |-- asyncapi.yaml         catalogo de eventos y topicos
|   |-- gitflow.md            flujo de trabajo con ramas
|-- contracts/schemas/       JSON Schema de cada evento (v1) y del sobre CloudEvents
|-- src/
|   |-- Dockerfile            imagen multietapa comun a todos los servicios .NET
|   |-- Contracts/            sobre CloudEvents y records de eventos (solo DTO)
|   |-- BuildingBlocks/       pipeline de consumo, outbox, idempotencia, DLQ, OTel, JWT
|   |-- Gateway/              YARP + JWT + sala de espera
|   |-- Catalogo/             API de busqueda + proyeccion de disponibilidad + semilla
|   |-- Reservas/             API + SSE + saga + expiracion
|   |-- Inventario/           Worker: Dominio / Aplicacion / Infraestructura
|   |-- Pagos/                Worker + Polly
|   |-- Notificaciones/       Worker + MailKit
|   |-- PasarelaMock/         pasarela simulada (latencia, fallas y rechazos)
|-- app/taquilla_app/        Flutter (BLoC, REST, SSE) + Dockerfile con nginx
|-- scripts/                  prueba de concurrencia, vista de grupos, nodo remoto
|-- tests/
|   |-- Unit/                 dominio y saga coreografiada completa en memoria
|   |-- Integration/          Testcontainers: Kafka y Elasticsearch reales
|-- .github/workflows/ci.yml  integracion continua: compila y ejecuta las pruebas

```

Diferencias entre el diseño y la implementación. La implementación respetó los ocho ADR; los cambios fueron de detalle y se resumen en la Tabla 5.17.

Tabla 5.17: Ajustes entre el diseño y la implementación

Aspecto	Diseño	Implementación
Contratos de eventos	JSON Schema en el repositorio.	Se mantiene el JSON Schema como contrato público y se agrega el proyecto Contracts con los <i>records</i> de cada evento (solo datos, sin lógica) para serializar con tipos.
Imágenes de contenedor	Un Dockerfile por servicio.	Un único Dockerfile multietapa con el argumento SERVICIO ; la restauración de paquetes queda en caché mientras no cambien los .csproj .
Datos de ejemplo	Carga manual con POST /api/admin/eventos .	Catálogo carga seis eventos de ejemplo al arrancar si el índice está vacío (variable CARGAR_SEMILLA).
Versión de Kafka	4.2.0.	4.2.2, parche de la misma serie.
Despliegue	Un solo equipo anfitrión.	Además, un modo distribuido: réplicas de Inventario y Pagos en otros equipos (docker-compose.nodo-remoto.yml), con la pasarela publicada en el puerto 8090 y la IP del equipo principal en IP_ANFITRION .
Observabilidad del reparto	Trazas y logs en Aspire Dashboard.	Además, scripts/ver-grupos.sh (miembros, particiones y <i>lag</i> por grupo) y un log que indica en qué nodo se hizo cada cobro.
Pruebas	Unitarias de dominio e integración.	Además, una prueba de la saga completa en memoria y dos scripts de carga (prueba-concurrencia.sh y .ps1).

5.10.2 Pasos de despliegue

1. Requisitos: Docker Desktop (Windows o macOS) o Docker Engine (Linux) con al menos 8 GB de RAM disponibles. En Linux, Elasticsearch necesita `vm.max_map_count ≥ 262144`.
2. Clonar el repositorio: `git clone https://github.com/CarlitosPinzon/EDA-Project.git`.
3. Levantar todo el sistema: `docker compose up -d --build`. La primera vez tarda entre 5 y 15 minutos porque descarga imágenes y compila los servicios; después arranca en segundos.
4. Verificar el estado con `docker compose ps`: todos los servicios *healthy* y *kafka-init* en *exited (0)*. Catálogo carga automáticamente seis eventos de ejemplo.
5. Usar la app web en `http://localhost:3000` (roles Comprador y Organizador) y observar el flujo en Kafbat UI (:8080), Aspire Dashboard (:18888), Kibana (:5601) y Mailpit (:8025).

6. Ajustar la demostración (opcional): copiar `.env.example` como `.env` para cambiar los minutos de expiración, el cupo de la sala de espera o las fallas de la pasarela simulada.
7. Modo distribuido (opcional): en el equipo principal, definir `IP_ANFITRION` con su IP en la red local y ejecutar `docker compose up -d`; en cada equipo adicional, ejecutar `docker compose -f docker-compose.nodo-remoto.yml up -d -build o`, en Windows, hacer doble clic en `iniciar-nodo.cmd`, que pide la IP y el nombre del nodo, comprueba la conexión con los puertos 9094, 9200, 8090 y 18889 y espera a que los servicios estén *healthy*. Los detalles están en `GUIA-DEMO-2PCS.md` y `ENCENDER-NODO-WINDOWS.md`.

5.10.3 Criterios de aceptación de la implementación

Tabla 5.18: Criterios de aceptación ligados a los requisitos del taller

Requisito del taller	Cómo se cumple
Mínimo 3 entidades relacionadas	Evento, Localidad, Reserva, Pago y Cliente (Figura 5.11).
Flujo funcional de extremo a extremo	CU-01 desde la app Flutter hasta el correo de confirmación (Figura 5.5).
Persistencia real	Cuatro índices de dominio en Elasticsearch y un índice de eventos procesados por grupo de consumidores.
Manejo de errores y validaciones	Validación en la API y en el contrato del evento; flujos alternos A1–A8; Polly; reintentos y DLQ.
Contenedores	Todo el sistema con <code>docker compose up</code> , con <i>health checks</i> y arranque ordenado (Figura 5.7).
Integración entre componentes	REST, Server-Sent Events y Kafka (Figura 5.4).
Repositorio público	https://github.com/CarlitosPinzon/EDA-Proyecto , con README, guías de despliegue y de demostración; la versión entregada se marca con el <i>tag</i> v1.0.0.

5.10.4 Evidencias de funcionamiento

Las pruebas se ejecutaron con el sistema completo en el equipo principal (Linux Mint) y un segundo equipo (Windows con Docker Desktop) que corría una réplica de Inventario y otra de Pagos. Así, el grupo de consumidores `inventario` tenía tres miembros (dos en el equipo principal y uno en el segundo), con cuatro particiones cada uno, y el grupo `pagos` tenía dos miembros, con seis particiones cada uno. El reparto se observó en vivo con `scripts/ver-grupos.sh` y en Kafbat UI. La Tabla 5.19 registra los resultados observados.

Tabla 5.19: Pruebas ejecutadas y resultados observados

Prueba	Procedimiento	Resultado observado	Escenario
E1. No sobreventa con dos equipos	<code>bash scripts/prueba-concurrencia.sh 100 50</code> : publica un evento con una localidad de 50 cupos y lanza 100 reservas concurrentes de un cupo; las que reciben 429 esperan el <code>Retry-After</code> y reintentan con la misma <code>Idempotency-Key</code> .	100 reservas admitidas (202), 850 respuestas 429 de la sala de espera durante la prueba y 0 errores. Resultado final: 50 CONFIRMADA, 50 RECHAZADA, 0 cupos disponibles, ninguna reserva en curso y 0 cupos vendidos por encima de la capacidad.	Consistencia (no sobreventa)
E2. Caída del Inventario de un equipo	En el equipo principal, <code>docker compose stop inventario</code> (detiene sus dos réplicas); se hacen reservas en la app; luego <code>docker compose start inventario</code> .	La réplica del segundo equipo asumió las 12 particiones del grupo <code>inventario</code> y las reservas siguieron hasta CONFIRMADA. Al volver las réplicas del equipo principal, Kafka repartió de nuevo las particiones entre los tres miembros.	Disponibilidad
E3. Caída de Pagos en uno y en los dos equipos	Se detiene Pagos en el equipo principal y se reserva; después se detiene también en el segundo equipo y se reserva otra vez; por último se enciende solo el Pagos del segundo equipo.	Con Pagos apagado en el equipo principal, el segundo equipo hizo los cobros (log <code>cobro ... APROBADO en el nodo pc2</code>). Con Pagos apagado en los dos, la reserva quedó en RETENIDA y el grupo <code>pagos</code> mostró <i>lag</i> 1 en <code>inventario.v1</code> : el evento esperó en Kafka. Al encender Pagos en el segundo equipo, la reserva pasó sola a CONFIRMADA y el <i>lag</i> volvió a 0.	Disponibilidad y recuperabilidad
E4. Caída de un equipo completo bajo carga	Se repite E1 y, durante la admisión de las reservas, se desconecta el segundo equipo de la red.	Kafka mantuvo al segundo equipo en los grupos hasta que venció el <code>session.timeout</code> (30 s); entonces reasignó sus particiones a las réplicas del equipo principal y el <i>lag</i> acumulado se procesó. El resultado final fue el mismo de E1: 50 confirmadas, 50 rechazadas, 0 disponibles y ninguna reserva en curso.	Tolerancia a fallos, consistencia

Procedimientos de verificación adicionales. Los escenarios de la Tabla 5.20 están implementados y documentados en el README; se describen como procedimiento de verificación porque su resultado se comprueba durante la sustentación.

Tabla 5.20: Procedimientos de verificación de los demás escenarios

Escenario	Cómo se ejecuta	Resultado esperado
Tolerancia a fallos (pasarela inestable)	<code>TASA_FALLLO=0.8</code> en <code>.env</code> y <code>docker compose up -d pasarela-mock</code> .	Reintentos con <i>backoff</i> ; si la falla persiste, pago rechazado y cupos liberados (A2).
Pago rechazado (A2)	Reservar 6 cupos VIP del evento de ejemplo <i>Festival Andino de Rock</i> , que superan <code>MONTO_MAXIMO</code> .	Reserva RECHAZADA con “Fondos insuficientes” y cupos devueltos.
Sin cupos (A1)	Dos sesiones reservan 3 cupos cada una del <i>Palco</i> (4 cupos) de <i>Noche Sinfónica de los Andes</i> .	La segunda reserva termina RECHAZADA (“Solo queda 1 cupo”) sin cobro.
Expiración (A3) y pago tardío (A5)	<code>MINUTOS_EXPIRACION=1</code> , Pagos detenido, reserva en la app, espera de 1,5 minutos y Pagos encendido de nuevo.	La reserva pasa a EXPIRADA y libera los cupos; si el cobro alcanzó a hacerse, se reembolsa (REEMBOLSADA).
Mensaje envenenado (A7)	En Kafbat UI, producir en <code>reservas.v1</code> un mensaje con valor <code>hola</code> .	Cada consumidor lo envía a <code>reservas.v1.dlq</code> con la causa en las cabeceras <code>dlq-*</code> .
Modificabilidad	Consumidor de consola con un grupo nuevo (<code>analitica</code>) sobre <code>reservas.v1</code> .	Recibe todos los eventos sin modificar ningún servicio.
Observabilidad	Aspire Dashboard, pestaña <i>Traces</i> , sobre una reserva.	Una sola traza atraviesa gateway, reservas, Kafka, inventario, pagos y notificaciones.

Pruebas automatizadas. El repositorio incluye pruebas unitarias del dominio (*Localidad*, *Reserva*, *Catálogo* y *Pagos*) y una prueba de la saga coreografiada completa en memoria (`dotnet test tests/Unit`), pruebas de integración con Kafka y Elasticsearch reales mediante Testcontainers (`tests/Integration`) y pruebas de la app Flutter (`flutter test`). El *workflow* de GitHub Actions compila y ejecuta las pruebas en cada *push*.

Lecciones aprendidas

Este capítulo recoge lo que el grupo aprendió durante la investigación, el análisis, el diseño y la implementación. Cada lección se apoya en una evidencia concreta: una decisión que se tomó, una alternativa que se descartó, un error que se detectó al revisar el diseño o un resultado de las pruebas del sistema construido. Las lecciones propias de la implementación están en la Sección 6.6.

6.1 Sobre el estilo arquitectónico

6.1.1 En una coreografía, el flujo hay que documentarlo porque el código no lo muestra

En la topología Broker ningún servicio conoce el proceso completo. Al diseñar la saga de reserva, el grupo notó que ni el diagrama de contenedores ni el código de cada servicio responden la pregunta “¿qué pasa después de que se aprueba el pago?”. Por eso fue necesario construir artefactos específicos: la matriz de coreografía (Tabla 5.10), la máquina de estados (Figura 5.6) y el diagrama dinámico (Figura 5.5).

Lección: en EDA el diagrama dinámico no es opcional. Sin una “partitura” de quién reacciona a qué, el sistema se vuelve difícil de explicar, de probar y de modificar, aunque cada servicio por separado sea simple.

6.1.2 Kafka ordena por partición, no por proceso de negocio

La primera versión de la saga suponía que `AsientosRetenidos` siempre llegaría a Reservas antes que `PagoAprobado`. Al revisar el diseño se encontró que ambos eventos viajan por tópicos distintos y que Kafka no garantiza ningún orden entre ellos: si Pagos procesa rápido, la aprobación puede llegar primero y la reserva quedaría atascada. La corrección fue diseñar estados que solo avanzan y aceptar la transición directa `PENDIENTE` → `CONFIRMADA` (flujo alterno A8).

Lección: el orden garantizado por el broker es más limitado que el orden “lógico” del negocio. Todo consumidor debe tolerar eventos desordenados y repetidos, y la máquina de estados es la herramienta para hacerlo explícito.

6.1.3 Cada agregado necesita una sola autoridad sobre su ciclo de vida

En la primera versión, Inventario convertía cupos retenidos en vendidos al recibir `PagoAprobado`, mientras Reservas podía, al mismo tiempo, dar la reserva por expirada. En esa carrera, Inventario vendía cupos de una reserva expirada y los otros servicios quedaban en desacuerdo. La solución fue que Reservas sea la única que decide el estado final y que Inventario siga sus decisiones (`ReservaConfirmada`, `ReservaRechazada`, `ReservaExpirada`), nunca los eventos de pago.

Lección: la coreografía no significa que “todos deciden”. Cada decisión del negocio debe tener un dueño; los demás servicios reaccionan a ella. Cuando dos servicios pueden decidir sobre el mismo hecho, aparecen carreras que no se ven en el camino feliz.

6.1.4 La consistencia eventual se diseña también en la interfaz

El usuario no puede ver “casi reservado”. Los estados intermedios (PENDIENTE, RETENIDA) tuvieron que convertirse en pantallas concretas de la app, con notificaciones en vivo y una regla clara para reconectarse (consultar el estado actual antes de reabrir el *stream*).

Lección: la consistencia eventual no es solo un problema del backend; si la interfaz no la muestra, el usuario la percibe como un error.

6.1.5 Los patrones del estilo no son opcionales

Al comparar la lista de patrones del catálogo del taller con los problemas del caso (Tabla 2.5), el grupo comprobó que *outbox*, idempotencia, DLQ, sagas y trazas distribuidas no son “buenas prácticas extra”: sin cualquiera de ellos, el sistema pierde o duplica reservas en escenarios normales, como un reinicio de contenedor o un reintento de red.

Lección: adoptar EDA implica adoptar su conjunto de patrones completo. Elegir el estilo sin ellos produce un sistema distribuido frágil.

6.2 Sobre el stack tecnológico

6.2.1 Elasticsearch como almacén principal obliga a modelar por agregados

Elasticsearch no tiene transacciones entre documentos, y esa limitación terminó guiando buena parte del diseño: cada agregado cabe en un documento, el *outbox* se guarda dentro del mismo documento para que estado y evento se escriban juntos, y el control de concurrencia optimista protege los cupos. Si el grupo hubiera usado una base relacional, varias de estas decisiones habrían sido innecesarias, pero también menos visibles.

Lección: las limitaciones de la base de datos no se esconden; se convierten en decisiones de diseño que deben justificarse. Un ADR es el lugar para dejar explícito por qué se aceptan (ADR-003).

6.2.2 La clave de partición es una decisión de diseño, no de configuración

La primera versión particionaba todos los tópicos por `reservaId`. Al analizar cómo se escribe el documento de una localidad se vio que eso generaba escrituras paralelas sobre el mismo documento, con conflictos y reintentos. Cambiar la clave de `reservas.v1` a `localidadId` convirtió a Inventario en escritor único por localidad, mientras Pagos y Reservas conservaron `reservaId` para repartir su carga.

Lección: la clave de partición define qué se procesa en orden, quién compite con quién y dónde se concentra la carga. Se elige a partir del agregado que se escribe, no por costumbre.

6.2.3 Evaluar el ecosistema, no solo las funciones

Tres decisiones cambiaron al revisar el estado real de las herramientas y no sus características:

- El cliente de SignalR para Flutter es comunitario, sin editor verificado; como la comunicación en vivo solo va del servidor a la app, se reemplazó por Server-Sent Events, nativos en .NET 10.
- La herramienta Kafka UI original dejó de mantenerse y tiene vulnerabilidades conocidas; se reemplazó por Kafbat UI, su continuación mantenida.
- MassTransit pasó a licencia comercial en su versión 9, y las colas nativas de Kafka (*share groups*) aún no tienen cliente .NET; ambas se descartaron para este alcance.

Lección: antes de adoptar una librería hay que verificar quién la mantiene, cuándo fue su última versión, su licencia y si existe para el lenguaje del stack. Una opción técnicamente mejor puede ser la peor si nadie la mantiene.

6.2.4 Una herramienta más es un contenedor más que operar

Para el API Gateway se evaluaron Kong, Traefik, Nginx, Envoy y Ocelot. Varias son más completas que YARP, pero cada una agrega un lenguaje o un formato de configuración distinto. YARP resolvió lo que el diseño necesitaba (JWT, sala de espera, enrutamiento, *streaming*) con el mismo lenguaje y el mismo *middleware* del resto de servicios. Por la misma razón, OpenTelemetry Collector + Jaeger se sustituyó por Aspire Dashboard, que muestra trazas, métricas y logs en un solo contenedor.

Lección: en un sistema distribuido, la complejidad operativa crece con cada pieza. La mejor herramienta es la que cubre la necesidad real con el menor número de tecnologías nuevas.

6.3 Sobre la arquitectura de referencia

6.3.1 Distinguir entre hechos documentados y supuestos

La información pública sobre Ticketmaster describe su uso de Kafka, su sala de espera y sus incidentes, pero no cómo retiene internamente los asientos. El grupo separó explícitamente los hechos verificables (Tabla 5.1) de los supuestos del diseño, y adaptó la retención con TTL de los diseños de referencia a las herramientas del stack asignado.

Lección: una arquitectura de referencia sirve para extraer decisiones y problemas reales, no para copiarla. Su escala y su historia (casi 50 años de sistemas heredados) no se trasladan a un proyecto académico, y decirlo explícitamente hace más creíble el diseño.

6.3.2 Los incidentes enseñan más que los casos de éxito

La preventa del *Eras Tour* (3 500 millones de solicitudes, cuatro veces el pico anterior) dejó una lección que se incorporó directamente al diseño: si el sistema admite más demanda de la que puede procesar, colapsa. Por eso la admisión se limita en el borde y Kafka amortigua lo que ya entró.

Lección: los atributos de calidad se entienden mejor a partir de fallas documentadas que de descripciones ideales.

6.4 Sobre el proceso de trabajo

6.4.1 Los ADR hacen posible cambiar de opinión

Varias decisiones cambiaron durante el diseño (notificaciones, observabilidad, clave de partición, autoridad en la saga). Como cada una estaba registrada en un ADR con su contexto y sus alternativas, fue posible revisarlas sin rehacer el documento completo y ubicar exactamente qué capítulos debían actualizarse.

Lección: un ADR no sirve solo para justificar; sirve para saber qué depende de cada decisión cuando esta cambia.

6.4.2 Revisar el diseño es más barato que corregir el código

Los dos errores de la saga (orden entre tópicos y autoridad sobre la reserva) se encontraron recorriendo el diagrama dinámico y la matriz de coreografía, antes de escribir código. En la implementación se habrían manifestado como fallas intermitentes, difíciles de reproducir.

Lección: en sistemas asíncronos conviene “ejecutar” el diseño en papel, evento por evento, incluyendo los caminos de error y los tiempos límite.

6.4.3 Los datos de mercado se leen como tendencia

Las fuentes de salarios mostraron diferencias grandes: un desarrollador .NET en Colombia gana en promedio unos \$4,3 millones al mes según portales locales, pero más de \$14 millones en contratos remotos en dólares. A esto se suma que los índices basados en actividad pública pierden precisión por el uso de asistentes de IA.

Lección: para un análisis de mercado honesto hay que combinar fuentes, indicar qué mide cada una y no presentar una sola cifra como verdad.

6.5 Qué haríamos diferente

Tabla 6.1: Decisiones que el grupo reconsideraría en un proyecto real

Tema	En este taller	En un proyecto real
Fuente de verdad de los datos	Elasticsearch para todo, porque es la base asignada.	Base relacional (PostgreSQL o SQL Server) como fuente de verdad del inventario y Elasticsearch como lado de lectura alimentado por eventos.
Topología de la saga	Coreografía (Broker), por ser el estilo asignado.	Evaluar orquestación (Mediator) para la saga de reserva y pago, que tiene muchos caminos de error; coreografía para el resto de reacciones.
Punto de partida del diseño	Entidades y servicios primero, eventos después.	Empezar con un taller de <i>Event Storming</i> con expertos del negocio para descubrir eventos, comandos y límites antes de dibujar contenedores.
Contratos de eventos	JSON Schema versionado en el repositorio.	<i>Schema Registry</i> con validación de compatibilidad en el pipeline de integración continua, como hace Ticketmaster.
Sala de espera	<i>Rate limiting</i> en memoria dentro del gateway.	Servicio dedicado con estado distribuido, orden aleatorio de entrada y filtro de bots.
Pruebas	Pruebas unitarias e integración con Testcontainers.	Añadir pruebas de contrato entre productores y consumidores y pruebas de carga que reproduzcan una apertura de venta.
Alta disponibilidad	Servicios replicados en dos equipos, pero Kafka y Elasticsearch en un solo nodo del equipo principal.	Clúster de Kafka (3 <i>brokers</i> , RF=3) y de Elasticsearch (3 nodos) repartidos entre equipos, para que ningún equipo sea un punto único de falla.

6.6 Lecciones de la implementación

Estas lecciones salen de construir el sistema y de probarlo en uno y en dos equipos (Sección 5.10.4). Siguen la misma estructura que las anteriores: la situación, la evidencia y lo que el grupo aprendió.

6.6.1 El *outbox* cumple su promesa a cambio de entregar eventos más de una vez

El relevo del *outbox* publica los eventos pendientes de un documento y después los marca como publicados. Si el proceso cae entre esos dos pasos, el evento se vuelve a publicar al reiniciar. Además, el relevo de cada réplica barre cada dos segundos los documentos con `tieneOutboxPendiente = true`, sin importar qué réplica los escribió: si una réplica cae con eventos sin publicar, otra los publica. En las pruebas de caída (E2 a E4) ninguna reserva se perdió ni quedó sin respuesta, y la

razón es que todos los consumidores descartan los duplicados con el índice `<grupo>-procesados`.

Lección: el *outbox* no elimina los duplicados; los hace seguros. La entrega “al menos una vez” es un contrato de todo el sistema y solo funciona si todos los consumidores son idempotentes, sin excepciones.

6.6.2 Agregar réplicas no reparte la carga de una localidad muy demandada

En la prueba E1 las 100 reservas eran para una sola localidad. Como `reservas.v1` usa `localidadId` como clave, todas cayeron en la misma partición y las procesó una sola réplica de Inventario, aunque había tres en dos equipos. Eso es justamente lo que garantiza que no haya sobreventa (escritor único), pero también significa que más réplicas no aceleran la venta de un evento muy popular: solo reparten localidades distintas.

Lección: la clave de partición fija a la vez la consistencia y el límite de paralelismo. Para un evento muy demandado, la capacidad la define la admisión en el borde (sala de espera) y no el número de réplicas; si hiciera falta más, habría que partir el inventario en unidades más pequeñas (por ejemplo, bloques de asientos).

6.6.3 La sala de espera solo funciona con clientes que cooperan

Para admitir 100 reservas, la sala de espera respondió 850 veces `429 Too Many Requests`: en promedio, cada comprador lo intentó más de ocho veces. El sistema se mantuvo correcto porque el cliente de prueba respeta la cabecera `Retry-After` y reintenta con la misma `Idempotency-Key`. Sin esa clave, un reintento después de un tiempo de espera agotado podría crear dos reservas para la misma compra.

Lección: limitar la admisión traslada trabajo al cliente. La API debe devolver cuándo reintentar y aceptar claves de idempotencia, y la app debe usarlas; de lo contrario, la protección contra picos genera duplicados.

6.6.4 Detectar una falla toma tiempo, y ese tiempo se configura

En las pruebas de caída hubo dos comportamientos distintos. Al detener un servicio de forma ordenada (`docker compose stop`), la réplica avisa a Kafka que sale del grupo y sus particiones se reasignan en pocos segundos. Al desconectar el equipo de la red (E4), Kafka no recibe ningún aviso y espera a que venza el `session.timeout` (30 s) antes de reasignar; mientras tanto, los eventos de esas particiones se acumulan como *lag*. Ninguno se pierde, pero las reservas afectadas se demoran.

Lección: la detección de fallas es un compromiso. Un *timeout* corto recupera más rápido, pero expulsa réplicas sanas ante una pausa de red o de recolección de basura y provoca rebalanceos innecesarios. El valor debe elegirse y documentarse, porque define cuánto se demora el sistema en recuperarse.

6.6.5 En Kafka, la dirección de conexión la decide el *broker*

Para conectar el segundo equipo no bastó con abrir el puerto 9094. El cliente se conecta primero a la dirección configurada, pero después usa la que el *broker* le anuncia (`advertised.listeners`); con el valor por defecto `localhost`, el segundo equipo terminaría intentando conectarse a sí mismo. La solución fue anunciar la IP del equipo principal (`IP_ANFITRION`) en el *listener* externo y mantener `kafka:9092` para la red interna de Docker. Cada cambio de red obliga a actualizar esa variable y recrear el contenedor de Kafka.

Lección: la red también es parte de la arquitectura. En un despliegue con contenedores hay al menos dos redes (la interna y la del equipo) y cada una necesita su propio *listener*; asumir “localhost” funciona hasta que aparece el primer cliente externo.

6.6.6 Replicar los servicios no replica la infraestructura

El segundo equipo asumió el trabajo de Inventario y Pagos cada vez que se detuvieron en el equipo principal. Sin embargo, Kafka y Elasticsearch corren en un solo nodo dentro del equipo principal: si ese equipo se apaga, todo el sistema se detiene, aunque los servicios del segundo equipo sigan encendidos.

Lección: la alta disponibilidad se decide componente por componente, y el eslabón débil suele ser el que guarda el estado. Los servicios sin estado ya están listos para repartirse entre equipos; la infraestructura necesita su propio clúster (Tabla 5.13).

6.6.7 Muchas fallas aparecen en la configuración entre componentes

Tres problemas de la implementación, documentados en los comentarios de `docker-compose.yml` y `nginx.conf`, no estaban en el código de los servicios. (1) Los contenedores arrancan con `sh`, que descarta en silencio las variables de entorno cuyo nombre tiene guion o punto; el gateway ignoraba su configuración de destinos, usaba `localhost` y respondía 502. (2) `nginx` guardaba la IP del gateway resuelta al arrancar; si el gateway se recreaba con otra IP, la app recibía 502 hasta reiniciar `nginx`, lo que se resolvió con `resolver 127.0.0.11 valid=10s`. (3) Para que el *stream* SSE llegue en vivo a la app, `nginx` debe desactivar su *buffering* y ampliar su tiempo de lectura (una hora); con la configuración por defecto, los cambios de estado llegarían agrupados o la conexión se cortaría.

Lección: en un sistema distribuido, parte de las fallas está en la configuración que une los componentes: nombres de variables, DNS y *proxies*. Cada salto de una comunicación en vivo debe configurarse para ella, y conviene documentar esas reglas junto al archivo donde se aplican, como se hizo en `docker-compose.yml` y `nginx.conf`.

6.6.8 Un despliegue reproducible necesita verificaciones previas, no solo un comando

Al montar el segundo equipo en Windows aparecieron errores que no eran de la arquitectura: un `docker compose up` sin `-f` levantó el sistema completo en Windows (Kafka, Elasticsearch, Kibana y los demás) en lugar de solo el nodo remoto, y la descarga de imágenes base falló por un *timeout* de TLS en una red lenta. A eso se suma un riesgo conocido de Windows: un `.env`

creado con `echo` en PowerShell 5 queda en UTF-16 y Docker Compose no lo lee. La respuesta fue un script (`iniciar-nodo.cmd`) que verifica Docker, valida la IP y el nombre del nodo, guarda el `.env` en ASCII, prueba los puertos 9094, 9200, 8090 y 18889 antes de levantar nada, reutiliza las imágenes si ya existen y espera a que los servicios estén *healthy*.

Lección: la facilidad de despliegue es un atributo de calidad que también se diseña. Un comando único no basta si depende de que la red, la configuración y el entorno sean los correctos; verificar esas condiciones antes de arrancar convierte errores confusos en mensajes claros.

6.6.9 Lo que no se ve no se puede demostrar

Las trazas de Aspire Dashboard muestran bien el recorrido de una reserva, pero no dicen qué equipo tiene cada partición ni cuántos eventos esperan. Para demostrar el reparto y la recuperación hubo que agregar dos herramientas: `scripts/ver-grupos.sh`, que muestra en vivo los miembros de cada grupo, su IP, sus particiones y el *lag*, y un log en Pagos que indica en qué nodo se hizo cada cobro. Con ellas, las pruebas E2 a E4 se pudieron observar mientras ocurrían: el *lag* en 1 con Pagos apagado fue la evidencia de que el evento seguía guardado en Kafka.

Lección: en EDA la observabilidad no termina en las trazas. El estado de los grupos de consumidores y el *lag* son la forma de ver la parte asíncrona del sistema, y deben estar disponibles desde el principio, no solo cuando algo falla.

Conclusiones

1. **El estilo es adecuado para el dominio.** La Arquitectura Orientada a Eventos en topología Broker responde a los riesgos principales de una plataforma de reservas: picos de demanda en las aperturas de venta, dependencia de una pasarela de pagos lenta o inestable y funcionalidades que crecen por separado. La arquitectura de Ticketmaster, construida alrededor de Kafka, confirma que estas decisiones se toman en la industria para el mismo problema.
2. **El costo del estilo es real y se paga con patrones.** La consistencia eventual, la complejidad de los errores parciales y la baja visibilidad del flujo no desaparecen: se gestionan con saga coreografiada, *outbox*, idempotencia, DLQ, estados explícitos y trazas distribuidas. Los ocho ADR convierten cada uno de esos patrones en una decisión concreta y trazable hasta los diagramas.
3. **El stack es coherente con el estilo, con una tensión bien identificada.** Kafka, .NET 10 y Flutter encajan de forma natural con EDA. Elasticsearch, usado como almacén principal, es la pieza que más exige: obliga a modelar agregados en un solo documento, a guardar el *outbox* junto al estado y a diseñar la escritura con un único escritor por localidad y concurrencia optimista. Esa exigencia se documentó como una decisión consciente y no como un defecto oculto.
4. **El diseño se implementó y cumple los requisitos del taller.** El sistema tiene cinco entidades relacionadas, un caso de uso de extremo a extremo con ocho flujos alternos, persistencia real por servicio, manejo de errores y validaciones, despliegue en contenedores con un solo comando y todos los diagramas obligatorios (HLD, contexto, contenedores, dinámico y despliegue) y opcionales (*landscape*, componentes, código y modelo de datos). El código está publicado en un repositorio público con las guías para desplegarlo y demostrarlo.
5. **Las pruebas confirmaron las decisiones centrales y mostraron sus límites.** Con réplicas en dos equipos, 100 solicitudes concurrentes por 50 cupos terminaron en exactamente 50 ventas y ninguna sobreventa; al apagar Inventario o Pagos en un equipo, o desconectarlo de la red en plena carga, el otro asumió su trabajo, y con Pagos apagado en ambos la reserva esperó en Kafka hasta que un servicio volvió. Las mismas pruebas dejaron ver los límites: el escritor único por localidad fija el paralelismo de un evento muy demandado, la detección de una caída abrupta tarda lo que dure el `session.timeout`, y Kafka y Elasticsearch en un solo nodo siguen siendo un punto único de falla.
6. **Revisar el diseño antes de implementarlo evitó errores costosos.** Recorrer la saga evento por evento reveló dos fallas de coordinación (orden entre tópicos y autoridad sobre la reserva) que en el código habrían aparecido como errores intermitentes. La revisión del ecosistema, además, reemplazó herramientas sin mantenimiento o con licencias restrictivas por alternativas vigentes.
7. **El mercado valora la combinación.** C# y .NET mantienen una demanda estable y alta, en particular en Colombia; Flutter crece en el sector financiero; y el streaming de eventos se consolida como infraestructura estratégica, como muestra la compra de Confluent por

IBM. El perfil que combina backend .NET, EDA con Kafka y búsqueda con Elasticsearch es escaso y mejor remunerado, y su habilidad más transferible es razonar sobre sistemas distribuidos asíncronos.

Trabajo futuro. Desplegar Kafka y Elasticsearch como clústeres de tres nodos repartidos entre equipos para eliminar el punto único de falla; ejecutar pruebas de carga que reproduzcan una apertura de venta y midan los tiempos de respuesta de la Tabla 5.4; agregar pruebas de contrato entre productores y consumidores con un *Schema Registry*; y evaluar el escalado automático de los consumidores según el *lag*.

Referencias

- [1] M. Richards y N. Ford, *Fundamentals of Software Architecture: An Engineering Approach*, 2.^a ed. Sebastopol, CA: O'Reilly Media, 2025.
- [2] L. Bass, P. Clements y R. Kazman, *Software Architecture in Practice*, 4.^a ed. Boston: Addison-Wesley, 2021.
- [3] G. Hohpe y B. Woolf, *Enterprise Integration Patterns: Designing, Building, and Deploying Messaging Solutions*. Boston: Addison-Wesley, 2003.
- [4] M. Kleppmann, *Designing Data-Intensive Applications*. Sebastopol, CA: O'Reilly Media, 2017.
- [5] D. Luckham, *The Power of Events: An Introduction to Complex Event Processing in Distributed Enterprise Systems*. Boston: Addison-Wesley, 2002.
- [6] P. Th. Eugster, P. A. Felber, R. Guerraoui y A.-M. Kermarrec, "The many faces of publish/subscribe," *ACM Computing Surveys*, vol. 35, no. 2, pp. 114–131, 2003.
- [7] M. Fowler, "What do you mean by 'Event-Driven'?", martinowler.com, feb. 2017. [En línea]. Disponible: <https://martinfowler.com/articles/201701-event-driven.html>
- [8] J. Kreps, "The Log: What every software engineer should know about real-time data's unifying abstraction," LinkedIn Engineering, dic. 2013. [En línea]. Disponible: <https://engineering.linkedin.com/distributed-systems/log-what-every-software-engineer-should-know-about-real-time-datas-unifying>
- [9] M. Nygard, "Documenting Architecture Decisions," Cognitect Blog, nov. 2011. [En línea]. Disponible: <https://cognitect.com/blog/2011/11/15/documenting-architecture-decisions>
- [10] ISO/IEC, *ISO/IEC 25010:2023 Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model*. Ginebra: ISO, 2023.
- [11] Apache Software Foundation, "Apache Kafka Documentation." [En línea]. Disponible: <https://kafka.apache.org/documentation/>
- [12] Apache Software Foundation, "Apache Kafka." [En línea]. Disponible: <https://kafka.apache.org/>
- [13] Apache Software Foundation, "Upgrading to 4.1 – Apache Kafka 4.1 Documentation." [En línea]. Disponible: <https://kafka.apache.org/41/getting-started/upgrade>
- [14] Apache Software Foundation, "Release 4.2.0," GitHub, feb. 2026. [En línea]. Disponible: <https://github.com/apache/kafka/releases/tag/4.2.0>
- [15] "Apache Kafka," *Wikipedia*. Consultado: sept. 2026. [En línea]. Disponible: https://en.wikipedia.org/wiki/Apache_Kafka
- [16] OpenLogic by Perforce, "Kafka 4.1: Enhancements in the Latest Release," 2025. [En línea]. Disponible: <https://www.openlogic.com/blog/kafka-4-1-overview>
- [17] Instaclustr, "Apache Kafka: Architecture, deployment and ecosystem [2026 guide]," 2026. [En línea]. Disponible: <https://www.instaclustr.com/education/apache-kafka/>

-
- [18] IBM Newsroom, “IBM Completes Acquisition of Confluent, Making Real Time Data the Engine of Enterprise AI and Agents,” 17 mar. 2026. [En línea]. Disponible: <https://newsroom.ibm.com/2026-03-17-ibm-completes-acquisition-of-confluent,-making-real-time-data-the-engine-of-enterprise-ai-and-agents>
- [19] Factor House, “What the IBM-Confluent deal means for Kafka users,” sept. 2026. [En línea]. Disponible: <https://factorhouse.io/articles/what-the-ibm-confluent-acquisition-means-for-kafka-users/>
- [20] Microsoft, “.NET and .NET Core official support policy.” Consultado: sept. 2026. [En línea]. Disponible: <https://dotnet.microsoft.com/en-us/platform/support/policy/dotnet-core>
- [21] Microsoft .NET Blog, “.NET 8 and .NET 9 will reach End of Support on November 10, 2026,” 2026. [En línea]. Disponible: <https://devblogs.microsoft.com/dotnet/dotnet-8-9-end-of-support/>
- [22] endoflife.date, “Microsoft .NET.” Consultado: sept. 2026. [En línea]. Disponible: <https://endoflife.date/dotnet>
- [23] “.NET,” *Wikipedia*. Consultado: sept. 2026. [En línea]. Disponible: <https://en.wikipedia.org/wiki/.NET>
- [24] endoflife.date, “Elasticsearch.” Consultado: sept. 2026. [En línea]. Disponible: <https://endoflife.date/elasticsearch>
- [25] NuGet Gallery, “Elastic.Clients.Elasticsearch.” Consultado: sept. 2026. [En línea]. Disponible: <https://www.nuget.org/packages/Elastic.Clients.Elasticsearch>
- [26] Elastic, “Elasticsearch .NET Client release notes.” [En línea]. Disponible: <https://www.elastic.co/docs/release-notes/elasticsearch/clients/dotnet>
- [27] “Elasticsearch,” *Wikipedia*. Consultado: sept. 2026. [En línea]. Disponible: <https://en.wikipedia.org/wiki/Elasticsearch>
- [28] “Flutter (software),” *Wikipedia*. Consultado: sept. 2026. [En línea]. Disponible: [https://en.wikipedia.org/wiki/Flutter_\(software\)](https://en.wikipedia.org/wiki/Flutter_(software))
- [29] Flutter, “What’s new in the docs.” Consultado: sept. 2026. [En línea]. Disponible: <https://docs.flutter.dev/release/whats-new>
- [30] Flutter Blog, “What’s new in Flutter 3.38,” nov. 2025. [En línea]. Disponible: <https://flutter.dev/blog/whats-new-in-flutter-3-38>
- [31] Shorebird, “Flutter Versions.” Consultado: sept. 2026. [En línea]. Disponible: <https://docs.shorebird.dev/getting-started/flutter-version/>
- [32] Stack Overflow, “2025 Developer Survey – Technology.” [En línea]. Disponible: <https://survey.stackoverflow.co/2025/technology>
- [33] Stack Overflow, “2025 Developer Survey – Work (Salary).” [En línea]. Disponible: <https://survey.stackoverflow.co/2025/work>
- [34] Stack Overflow, “2024 Developer Survey – Technology (Other frameworks and libraries).” [En línea]. Disponible: <https://survey.stackoverflow.co/2024/technology>
- [35] Stack Overflow Blog, “The 2026 Developer Survey is now open,” jun. 2026. [En línea]. Disponible: <https://stackoverflow.blog/2026/06/23/the-2026-developer-survey-is-now-open-for-human-developers-only/>

-
- [36] P. Carbonnelle, “PYPL PopularitY of Programming Language Index.” [En línea]. Disponible: <https://pypl.github.io/PYPL.html>
- [37] StatisticsTimes, “Top Computer Languages 2026” (datos PYPL y TIOBE), abr. 2026. [En línea]. Disponible: <https://statisticstimes.com/tech/top-computer-languages.php>
- [38] IEEE Spectrum, “The Top Programming Languages 2025,” sept. 2025. [En línea]. Disponible: <https://spectrum.ieee.org/top-programming-languages-2025>
- [39] TIOBE Software, “TIOBE Index for September 2026.” [En línea]. Disponible: <https://www.tiobe.com/tiobe-index/>
- [40] FOSS Post, “C# Statistics 2026,” sept. 2026. [En línea]. Disponible: <https://fosspost.org/c-statistics/>
- [41] Computrabajo Colombia, “Salario de Desarrollador .NET en Colombia.” Consultado: sept. 2026. [En línea]. Disponible: <https://co.computrabajo.com/salarios/desarrollador-net>
- [42] Computrabajo Colombia, “Salario de Desarrollador/a de software en Colombia.” Consultado: sept. 2026. [En línea]. Disponible: <https://co.computrabajo.com/salarios/desarrolladora-de-software>
- [43] Indeed Colombia, “Sueldo de Desarrollador/a de software en Colombia,” act. 29 ago. 2026. [En línea]. Disponible: <https://co.indeed.com/career/desarrollador-de-software/salaries>
- [44] Indeed Colombia, “Empleos de Flutter.” Consultado: sept. 2026. [En línea]. Disponible: <https://co.indeed.com/q-flutter-empleos.html>
- [45] elempleo.com, “Ofertas de empleo Desarrollador Flutter,” ago. 2026. [En línea]. Disponible: <https://www.elempleo.com/co/ofertas-empleo/trabajo-desarrollador-flutter>
- [46] Talently, “Salarios de Developers en Colombia 2026.” [En línea]. Disponible: <https://talently.tech/herramientas/colombia/salario>
- [47] Talently, “Salario de un programador de Flutter en Colombia,” 2026. [En línea]. Disponible: <https://talently.tech/herramientas/colombia/salario/skill/flutter>
- [48] Talently, “Cuánto gana un programador Net Core en Colombia,” 2026. [En línea]. Disponible: <https://talently.tech/herramientas/colombia/salario/skill/net-core>
- [49] Banco de la República, “Tasa de cambio representativa del mercado (TRM),” 27 sept. 2026. [En línea]. Disponible: <https://www.banrep.gov.co/es/glosario/tasa-cambio-trm>